



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

CAMILLE: un Chatbot per l'Identificazione e la Rimozione degli ML-Specific Code Smells

RELATORE

Prof. Fabio Palomba

Dott. Gilberto Recupito

Dott. Giammaria Giordano

Università degli Studi di Salerno

CANDIDATO

Francesco Alessandro Pinto

Matricola: 0512113216

Anno Accademico 2023-2024

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

A nonno Mimì.

Questa tesi la dedico a te, con tutto il mio cuore.

Abstract

Negli ultimi anni, le tecniche di Machine Learning (ML) hanno acquisito una notevole popolarità, trovando impiego in una vasta gamma di contesti, sia quotidiani che professionali. Nonostante ciò, nel campo dell'intelligenza artificiale (AI), persiste una scarsa attenzione verso i Code Smells, i quali, se non adeguatamente gestiti, possono incrementare significativamente il Technical Debt, minando la manutenibilità e l'estensibilità del software. Al fine di affrontare queste problematiche, è stato ideato e realizzato CAMILLE, un Conversational Agent (Chatbot) progettato per assistere i Machine Learning Engineers nelle diverse fasi delle pipeline di ML. Sviluppato mediante il framework Rasa e integrato come estensione in Visual Studio Code, CAMILLE fornisce supporto agli sviluppatori offrendo informazioni dettagliate ed esempi riguardanti gli ML-Specific Code Smells conosciuti allo stato dell'arte. Questo strumento consente inoltre l'analisi di repository su GitHub, generando report sui Code Smells identificati e fornendo suggerimenti per la loro correzione, elaborati attraverso il Large Language Model (LLM) GPT-4o mini. L'analisi condotta sui suggerimenti di refactoring proposti, ha rivelato che CAMILLE è capace di ridurre autonomamente il numero dei Code Smells identificati del 62.97%, evidenziando come questi suggerimenti possano risultare ancora più efficaci se integrati con l'esperienza e la conoscenza contestuale dello sviluppatore. Contribuendo efficacemente alla rimozione dei Code Smells, CAMILLE migliora indirettamente aspetti legati alla qualità del codice, quali efficienza e robustezza, e contribuisce alla riduzione del Technical Debt, consolidandosi come uno strumento prezioso nel suo ambito di applicazione.

Indice

Elenco delle Figure	iv
Elenco delle Tabelle	vi
1 Introduzione	1
1.1 Contesto Applicativo	1
1.2 Motivazione e Obiettivi	2
1.3 Risultati Ottenuti	3
1.4 Struttura della Tesi	4
2 Background e Stato dell'Arte	5
2.1 Technical Debt	5
2.1.1 Classificazione dei Technical Debt	6
2.1.2 AI Technical Debt	6
2.1.3 ML-Specific Technical Debt	7
2.1.4 Strategie di Gestione per AITD	8
2.2 Code Smells	9
2.2.1 Tassonomia dei Code Smells	9
2.2.2 Rapporto tra Code Smells e Technical Debt	10
2.2.3 Tool di Analisi	11
2.2.4 ML-Specific Code Smells	12

2.3	Chatbot	16
2.3.1	Storia dei Chatbot	16
2.3.2	Applicazioni dei Chatbot	17
2.3.3	Tipologie di Chatbot	18
2.3.4	Tecnologie di Base e Architettura dei Chatbot	20
2.3.5	Bot e Chatbot nel Software Engineering	22
3	CAMILLE: Conversational Agent for MachIne Learning smeLLs Elimination	24
3.1	Architettura Generale di CAMILLE	25
3.1.1	Tecnologie Utilizzate	25
3.1.2	Panoramica dell'Architettura	27
3.2	Modifiche al Tool di Analisi	29
3.2.1	Analisi delle repository GitHub	29
3.2.2	Recupero del corpo di una funzione	31
3.2.3	Definizione API	32
3.3	Sviluppo del Chatbot	34
3.3.1	Introduzione al Framework Rasa	34
3.3.2	Architettura del Chatbot	35
3.3.3	Database	37
3.3.4	Domain	44
3.3.5	Natural Language Understanding	47
3.3.6	Dialogue Management	53
3.3.7	Custom Actions	57
3.3.8	Endpoints	67
3.4	Sviluppo dell'Estensione per Visual Studio Code	67
3.4.1	Setup Iniziale	68
3.4.2	Configurazione dell'estensione	69
3.4.3	Realizzazione della WebView	73
4	Valutazione Preliminare dell'Efficacia di CAMILLE	79
4.1	Metodologia di Testing	79
4.1.1	Dataset	80
4.1.2	Procedimento	80

4.1.3	Refactoring del Codice	81
4.2	Analisi dei Risultati	82
4.2.1	Distribuzione dei Code Smells	83
4.2.2	Tasso di Riduzione dei Code Smells	84
4.2.3	Miglioramento della Qualità del Codice	87
4.3	Considerazioni Finali	89
5	Conclusioni e Sviluppi Futuri	91
5.1	Conclusioni	91
5.2	Sviluppi Futuri	92
	Bibliografia	93

Elenco delle figure

2.1	Tassonomia dei Code Smells [1, 2].	11
2.2	Conversazione tra PARRY e ELIZA.	17
2.3	Architettura di un Chatbot [3].	21
3.1	Deployment Diagram di CAMILLE.	28
3.2	Funzione "analyze_github_repository".	30
3.3	Funzione "get_function_body".	31
3.4	Endpoint API per richiedere l'analisi di una repository.	33
3.5	Endpoint API per richiedere il corpo di una funzione.	34
3.6	Architettura di un Chatbot Rasa.	37
3.7	File "mongo_db.py"	39
3.8	Funzione "build_field_projection"	40
3.9	Funzione "process_example_fields".	41
3.10	Funzione "get_all_code_smells".	42
3.11	Funzione "get_code_smell_by_id".	43
3.12	Funzione "get_code_smell_by_name".	43
3.13	Definizione degli Slots.	46
3.14	Response per salutare l'utente.	47
3.15	Esempio di dati di training per il modello di NLU.	48
3.16	Regex per l'entity 'project_repository'.	49

3.17	Lookup Table per l'entity 'code_smell_name'.	50
3.18	Funzione 'code_smell_name_matcher'.	51
3.19	Pipeline di NLU.	52
3.20	Rule per la richiesta del report.	54
3.21	Story 'Project analysis story'.	55
3.22	Policies del Chatbot.	56
3.23	Custom Action 'action_get_code_smells_list'.	59
3.24	Custom Action 'action_provide_code_smell_details'.	60
3.25	Custom Action 'action_provide_code_smell_example'.	61
3.26	Custom Action 'action_project_analysis'.	62
3.27	Custom Action 'action_analysis_report'.	64
3.28	Codice della Custom Action 'action_suggest_fix'.	65
3.29	Funzione 'complete_text'.	66
3.30	Estensione di CAMILLE all'interno di VS Code.	68
3.31	Funzione "GetWebViewContent".	69
3.32	Attivazione dell'estensione e configurazione WebView.	71
3.33	Gestione Messaggi della WebView.	72
3.34	Gestione del Global Unique Identifier (GUID).	74
3.35	Invio del messaggio al Chatbot all'interno della WebView.	76
3.36	Formattazione dei blocchi di codice per "highlight.js".	77
3.37	Download del report HTML.	78
4.1	Modulo "function_replacer.py".	82
4.2	Report sulla distribuzione dei Code Smells prima e dopo il refactoring.	84
4.3	Report sulle percentuali di riduzione dei Code Smells.	85
4.4	Report sui miglioramenti qualitativi riscontrati.	89

Elenco delle tabelle

2.1	22 ML-Specific Code Smells [4].	14
3.1	Campi della collection "codesmells".	38
3.2	Intents del Chatbot.	44
3.3	Custom Actions del Chatbot.	45
3.4	Entities del Chatbot.	46

CAPITOLO 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni, le tecniche di Machine Learning (ML) hanno rivoluzionato numerosi settori grazie alla loro capacità di apprendere dai dati storici e di prendere decisioni intelligenti guidate da tali informazioni. Settori come la cybersecurity, l'Internet of Things (IoT), l'healthcare, l'e-commerce e l'agricoltura sostenibile hanno visto trasformazioni significative grazie a queste tecnologie avanzate [5]. Per soddisfare la crescente domanda del mercato e rispettare tempi di sviluppo relativamente brevi, i team di progetti basati su ML devono includere figure professionali con competenze in diversi ambiti, come software engineer e data scientist [6], ed è fondamentale che queste figure collaborino in maniera efficace per la buona riuscita dell'intero progetto.

Tuttavia, i data scientist spesso non possiedono una solida formazione in ingegneria del software e tendono a seguire standard di programmazione differenti [6]. Sebbene la presenza di team eterogenei possa essere vantaggiosa da un certo punto di vista, poiché facilita la realizzazione di sistemi basati su ML in modo più rapido ed efficiente, scelte di progettazione subottimali e la non aderenza alle buone pratiche dell'ingegneria del software, soprattutto nel contesto dell'intelligenza artificiale, pos-

sono condurre all'accumulo di Technical Debt. Questo termine viene comunemente utilizzato per riferirsi all'insieme di compromessi tecnici che vengono fatti per velocizzare o facilitare lo sviluppo, ma che comportano un aumento della complessità del codice e una crescita esponenziale dei costi e della difficoltà di manutenzione a lungo termine [7].

Una strategia comune per mitigare il Technical Debt accumulato consiste nel migliorare la qualità del codice eliminando i Code Smells e gli anti-pattern al suo interno [4]. Purtroppo, gli studi sui Code Smell specifici del Machine Learning sono limitati [4], così come gli strumenti sviluppati per la loro identificazione e correzione.

1.2 Motivazione e Obiettivi

Come fatto notare nella sezione precedente (1.1), l'adozione del Machine Learning in vari settori ha portato a progressi notevoli ma ha anche introdotto complessità significative nella manutenzione del software. I team di sviluppo affrontano sfide crescenti legate ai Technical Debt, specialmente quando i data scientist o altre figure professionali analoghe, spesso privi di formazione formale in ingegneria del software, contribuiscono al codice. Questo infatti può risultare nell'introduzione di Code Smells che contribuiscono ad aumentare il Technical Debt, rendendo i sistemi più difficili da mantenere e aggiornare. Nonostante l'importanza critica di queste problematiche, ancora più accentuate nel contesto dell'AI e del ML, gli strumenti attuali per identificare e correggere tali difetti sono pochi e spesso limitati.

L'obiettivo di questa tesi consiste nello sviluppare un Conversational Agent che assista i Machine Learning Engineers nell'identificazione e nella correzione degli ML-Specific Code Smell. Questo tool, progettato per essere semplice, intuitivo e facilmente integrabile, sarà reso disponibile attraverso un'estensione per un Ambiente di Sviluppo Integrato (IDE), garantendo accessibilità e adozione nel flusso di lavoro quotidiano degli sviluppatori. Il Chatbot non solo fornirà informazioni dettagliate ed esempi dei vari code smells, ma offrirà anche consigli di correzione personalizzati attraverso un Large Language Model (LLM), permettendo agli sviluppatori di correggere più facilmente i Code Smell rilevati nel proprio codice e di ridurre il Technical Debt all'interno del sistema. Di fondamentale importanza sarà quindi valutare la

potenziale utilità del tool, comprendendo in che misura i suggerimenti personalizzati forniti da esso attraverso il LLM risultano effettivamente validi per rimuovere i Code Smells rilevati all'interno del codice.

1.3 Risultati Ottenuti

Lo sviluppo di CAMILLE ha dato vita a uno strumento estremamente utile, caratterizzato da un design modulare che ne facilita ulteriori miglioramenti ed estensioni. Attraverso le sue funzionalità, CAMILLE consente agli sviluppatori di rilevare più facilmente i Code Smells e offre un supporto efficace nella loro eliminazione. Questo contribuisce a ridurre la suscettibilità del codice agli errori e a migliorare aspetti qualitativi cruciali come la leggibilità, l'efficienza e la robustezza. Inoltre, l'eliminazione dei Code Smells, con i conseguenti benefici significativi sulla qualità del codice, permette di ridurre sostanzialmente il Technical Debt accumulato. Questi risultati evidenziano l'allineamento di CAMILLE con gli obiettivi prefissati e rafforzano la percezione del suo elevato potenziale, suggerendo la convenienza di investire in ulteriori sviluppi futuri. L'efficacia di CAMILLE e le sue notevoli potenzialità sono state confermate attraverso l'esecuzione di test sulla sua funzionalità più importante, la quale consiste nel fornire suggerimenti di refactoring personalizzati attraverso GPT-4o mini. Dai risultati di questi test è emerso che i suggerimenti di refactoring forniti da CAMILLE sono efficaci nell'eliminare autonomamente una considerevole quantità di ML-Specific Code Smells presenti nel codice (63.97%). Tuttavia, questi suggerimenti possono raggiungere un livello di efficacia ancora maggiore se raffinati e integrati con l'esperienza e la conoscenza approfondita del contesto applicativo da parte dello sviluppatore, in perfetta linea con gli scenari d'utilizzo previsti per CAMILLE.

Il codice di CAMILLE e della sua estensione per Visual Studio Code sono disponibili alle seguenti repository GitHub:

- **CAMILLE:** <https://github.com/FrancescoPinto02/CAMILLE>
- **Estensione:** <https://github.com/FrancescoPinto02/CamilleExtension>

1.4 Struttura della Tesi

La presente tesi è strutturata in cinque capitoli (Introduzione compresa). Di seguito è fornita una breve descrizione del contenuto di ciascuno di essi:

- **Capitolo 2 - Background e Stato dell'Arte:** Questo capitolo raccoglie e discute le informazioni relative agli studi condotti sui Technical Debt e sui Code Smells, con un focus particolare sui settori dell'Intelligenza Artificiale e del Machine Learning. Si esaminano inoltre vari aspetti dei Chatbot, includendo la loro struttura e i campi di applicazione.
- **Capitolo 3 - CAMILLE:** Il capitolo descrive le scelte progettuali adottate per lo sviluppo di CAMILLE, offrendo una panoramica dell'architettura e delle tecnologie impiegate. Viene dettagliata la realizzazione del Chatbot e delle sua estensione, con particolare attenzione alle realizzazione delle funzionalità critiche.
- **Capitolo 4 - Valutazione Preliminare dell'Efficacia di CAMILLE:** In questo capitolo viene illustrato il processo utilizzato per valutare l'efficacia dei suggerimenti di refactoring forniti da CAMILLE tramite GPT-4o mini, analizzando i principali risultati ottenuti.
- **Capitolo 5 - Conclusioni:** Questo capitolo presenta riflessioni e considerazioni finali su CAMILLE e sui risultati emersi dalla valutazione della sua efficacia. Vengono inoltre discussi i possibili sviluppi futuri del seguente lavoro.

Background e Stato dell'Arte

2.1 Technical Debt

Il concetto di Technical Debt è stato introdotto per la prima volta nel 1992 da W. Cunningham [8], il quale lo definisce come un insieme di scelte sub-ottimali effettuate in fase di design o implementazione che possono influire negativamente sulla qualità generale del sistema. Negli ultimi anni, con i ritmi sempre più frenetici nell'industria del software, gli sviluppatori sono spesso costretti a scendere a compromessi in termini di solidità del sistema per garantire semplicità e velocità di realizzazione, rendendo l'introduzione di Technical Debt un fenomeno piuttosto naturale.

I Technical Debt, in un certo senso, sono molto simili ai debiti finanziari: non tutti sono "cattivi", ma devono comunque essere ripagati prima o poi [7]. Di conseguenza, è essenziale risolvere gradualmente i Technical Debt, evitando un accumulo eccessivo che potrebbe diventare estremamente difficile da affrontare in futuro. Infatti, i Technical Debt non solo influiscono direttamente sulla manutenibilità e sull'estensibilità del software, aumentando i costi dei cambiamenti nel tempo, ma possono anche influire indirettamente sulla sicurezza e sull'affidabilità del sistema, contribuendo a causare bug e vulnerabilità [9].

2.1.1 Classificazione dei Technical Debt

Martini et al. [10] hanno fornito una classificazione di riferimento per i Technical Debt, individuando le seguenti 5 categorie:

- **Code Debt:** Technical Debt legati a soluzioni sub-ottimali all'interno del codice. Un esempio è l'uso di nomi di variabili poco descrittivi, che rende difficile la comprensione del codice e richiede tempo aggiuntivo per la manutenzione;
- **Architectural Debt:** Technical Debt legati a soluzioni sub-ottimali rispetto ad un'architettura ideale. Ad esempio, una struttura dati centralizzata che crea un bottleneck, rallentando l'accesso ai dati e le performance del sistema;
- **Test Debt:** Technical Debt riguardanti la mancanza o l'inadeguatezza dei test. Esempi includono test manuali non documentati che rendono difficile la replicazione e l'automazione;
- **Documentation Debt:** Technical Debt legati alla carenza o all'inadeguatezza della documentazione. Un esempio potrebbe essere la mancanza di commenti nel codice che spiegano la logica dietro le funzioni critiche, causando difficoltà nella comprensione e nel debugging;
- **Infrastructure Debt:** Technical Debt legati a soluzioni sub-ottimali nell'ambiente di sviluppo. Ad esempio, l'uso di server obsoleti che rallentano il processo di sviluppo e test, aumentando il rischio di downtime e problemi di performance.

Martini et al. [10] hanno evidenziato anche l'esistenza dei Social Debt e dei Process Debt, che, pur non essendo di natura tecnica, influenzano negativamente i progetti in maniera simile ai Technical Debt.

2.1.2 AI Technical Debt

Nel contesto dell'Intelligenza Artificiale e del Machine Learning, il problema dei Technical Debt è ancora più marcato. Infatti, data la natura stocastica e l'influenza del data science, i sistemi software basati su AI non solo soffrono dei Technical Debt tipici dei software tradizionali, ma possono anche presentare nuove tipologie di Technical Debt ancora poco esplorate [11], i cosiddetti AI Technical Debt (AITD).

Un'analisi condotta sull'impatto degli AI Technical Debt [12] ha permesso di capire che, nella maggior parte dei casi, gli AITD sono considerati molto problematici dai professionisti dell'AI, andando ad impattare negativamente su aspetti legati alla manutenibilità, modificabilità, performance, comprensibilità e accoppiamento.

2.1.3 ML-Specific Technical Debt

I Technical Debt nei sistemi basati su Machine Learning sono spesso più difficili da rilevare, poiché nella maggior parte dei casi sono legati a problemi che si "nascondono" a livello del sistema piuttosto che a livello del codice [7]. Per questo motivo, le tecniche normalmente impiegate per affrontare tali problemi spesso non sono sufficienti in questa tipologia di sistemi.

Con lo scopo di aumentare la consapevolezza riguardo questa tipologia di problemi e fornire delle linee guida per far fronte ad essi, Sculley et al. [7] hanno raccolto diverse tipologie di ML-Specific Technical Debt:

- **Data Dependencies Debt:** rappresentano una tipologia di Technical Debt derivante dalle dipendenze tra i dati utilizzati come input nei modelli. Queste dipendenze possono essere difficili da rilevare e gestire, poiché mancano strumenti di analisi adeguati. I Data Dependencies Debt come ad esempio le Unstable Data Dependencies e le Underutilized Data Dependencies, possono complicare la manutenzione e peggiorare l'affidabilità del sistema, aumentando la vulnerabilità ai cambiamenti e i costi di gestione;
- **Configuration Debt:** rappresentano una tipologia di Technical Debt che si accumulano nella configurazione dei sistemi di ML a causa della complessità e della varietà delle opzioni configurabili. Senza adeguate verifiche o test, gli errori nella configurazione possono condurre a significativi sprechi di tempo, risorse computazionali e problemi di produzione;
- **Analysis Debt:** rappresentano una tipologia di Technical Debt che emergono nei sistemi di Machine Learning Live, dove i modelli possono influenzare il proprio comportamento tramite aggiornamenti nel tempo. Questa tipologia di Technical Debt si manifesta attraverso Feedback Loop, diretti o nascosti,

che rendono difficile prevedere il comportamento del modello prima del suo rilascio e complicano l'analisi e la mitigazione degli effetti.

Oltre ai Technical Debt citati precedentemente, Sculley et al. [7] hanno presentato anche diversi Anti-Pattern che contribuiscono ad accrescere il Technical Debt. Inoltre, hanno evidenziato altri tipi di ML-Specific Technical Debts che interessano ulteriori aree, tra cui i Data Testing Debt, i Reproducibility Debt e i Cultural Debt [7].

2.1.4 Strategie di Gestione per AITD

Da un'analisi dettagliata delle strategie di gestione del Technical Debt nei sistemi basati su intelligenza artificiale, condotta da Recupito et al. [12], sono emerse diverse osservazioni chiave:

- La tecnica più diffusa per l'**identificazione** dei problemi all'interno di questi sistemi rimane l'**ispezione manuale**. Tuttavia, è emerso che molti professionisti del settore non dispongono di strategie particolari per affrontare alcuni problemi specifici, mentre in altri casi è necessario avvalersi di team specializzati per risolvere problematiche più specifiche e complesse;
- Sebbene l'uso di strumenti di **analisi statica** non sia la pratica predominante, esso mantiene comunque una certa rilevanza per alcuni professionisti. Strumenti come SonarQube [13] e metodologie come MLOps [14] vengono utilizzati per monitorare e gestire le pipeline di sviluppo, fornendo un supporto critico nel mantenimento della qualità del codice e nella gestione del Technical Debt.
- In termini di **refactoring**, la pratica più comune risulta essere il **refactoring ad hoc**. Questo approccio è seguito, in alcuni casi, dall'assenza totale di attività di refactoring. La scelta di effettuare o meno il refactoring dipende spesso dalla natura del problema e dalle risorse disponibili, evidenziando una gestione talvolta reattiva piuttosto che proattiva nei confronti dei Technical Debt.

Queste osservazioni riflettono una realtà variegata e complessa nella gestione dei Technical Debt nei sistemi AI, sottolineando l'importanza di strategie flessibili e adattabili alle specifiche esigenze del progetto e del team di sviluppo.

2.2 Code Smells

I Code Smells, chiamati anche "Bad Code Smells" o semplicemente "Smells", sono stati definiti per la prima volta nel 1999 da Fowler e Beck [1] come indicatori superficiali di problemi più profondi all'interno del codice. Essi sono sintomi che qualcosa potrebbe non essere giusto, ma non necessariamente sono problemi a sè stanti. Infatti, i Code Smells non sono da considerare errori di programmazione che impediscono il corretto funzionamento del sistema; piuttosto, suggeriscono la presenza di cattive pratiche di design o di implementazione che possono rendere il codice più difficile da mantenere e comprendere, rallentando lo sviluppo e aumentando il rischio di errori e problemi futuri [15].

Da uno studio condotto da Tufano et al.[16] sul ciclo di vita dei Code Smells, è emerso che la maggior parte dei Code Smells sono presenti fin dalla creazione del prodotto software e vengono introdotti principalmente durante l'implementazione di nuove funzionalità o il miglioramento di quelle esistenti. Dallo stesso studio [16], si è scoperto che il tempo di sopravvivenza di un Code Smell all'interno di un sistema è mediamente molto alto, con solo una piccola percentuale di essi che viene rimossa mediante refactoring, cancellazioni e inserimenti di codice. Questo evidenzia che i Code Smell, pur essendo potenzialmente pericolosi, spesso vengono sottovalutati o trascurati.

2.2.1 Tassonomia dei Code Smells

Fowler e Beck [1], al fine di supportare gli ingegneri del software nell'identificazione di problematiche all'interno del codice, hanno introdotto una lista di 22 Code Smell, fornendo inoltre indicazioni importanti per il loro refactoring. Sulla base di ciò, Mäntylä et al. [2] hanno effettuato un ulteriore lavoro, evidenziando la presenza di correlazioni tra diversi smells e raggruppando i 22 Code Smells in sette categorie distinte:

- **Bloaters:** problematiche legate a parti di codice che sono diventate troppo grandi e ingombranti, con conseguenti difficoltà di comprensibilità e modifica;

- **Object-Orientation Abusers:** smells che indicano un uso improprio dei principi dell'OOP;
- **Change Preventers:** smells che rendono difficile modificare il codice senza dover intervenire su molteplici parti del sistema, ostacolando quindi l'evoluzione del software;
- **Dispensables:** codice inutile o ridondante che può essere eliminato per migliorare la leggibilità e la manutenibilità del software;
- **Encapsulators:** problemi legati alla comunicazione tra diverse parti del codice, che influenzano la gestione e l'accesso ai dati;
- **Couplers:** smells legati a un forte accoppiamento tra classi, che riducono la modularità e la riusabilità del codice;
- **Others:** altre tipologie di smells che non rientrano nelle categorie precedenti;

Nella Figura 2.1 è possibile osservare nel dettaglio la tassonomia fornita da Mäntylä et al. [2] dei 22 Code Smells originali proposti da Fowler e Beck [1].

2.2.2 Rapporto tra Code Smells e Technical Debt

Das et al. [17] hanno evidenziato il legame diretto tra Code Smells e Technical Debt, sottolineando che tali Code Smells sono comunemente segnali di difetti di design che se non affrontati portano ad un accumulo di Technical Debt. Tuttavia, è possibile eliminare i Code Smells attraverso un continuo processo di refactoring, che migliora la struttura del codice e ne riduce i Technical Debt associati [17]. Questo approccio non solo migliora la qualità del software ma ne facilita anche la futura manutenzione ed estensione.

Un ulteriore studio ha esaminato 14 tecniche di refactoring comunemente impiegate nei sistemi basati su Machine Learning, collegandole ai Technical Debt che esse aiutano a mitigare [18]. L'analisi di queste tecniche permette di capire che alcune di esse possono essere effettivamente considerate tecniche di refactoring orientate alla

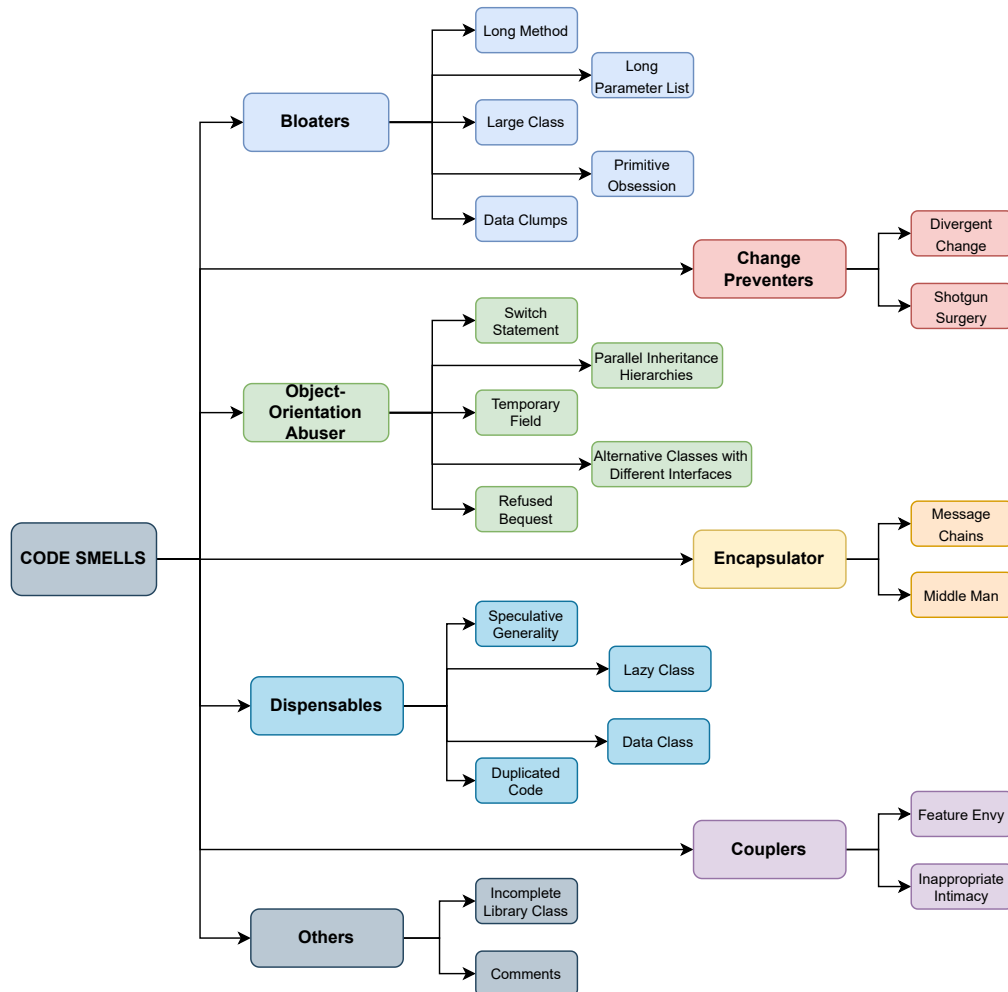


Figura 2.1: Tassonomia dei Code Smells [1, 2].

rimozione di Code Smells. Questo rafforza ulteriormente la correlazione tra Technical Debt e Code Smells, supportando l'idea che una strategia efficace per ridurre il Technical Debt consista proprio nell'eliminare i Code Smells [4, 17].

2.2.3 Tool di Analisi

Das et al. hanno analizzato 22 tools utilizzati per l'identificazione e la rimozione di Code Smell e Technical Debt, suddividendoli in quattro categorie [17]:

- **Code Based:** tools che analizzano direttamente il codice sorgente per identificare Code Smells e Technical Debt (eg. SonarQube [13] e Designite [19]);

- **Code and Design Based:** tools che utilizzano sia il codice sorgente sia elementi di design per valutare e identificare i Technical Debt (eg. STAN [20]). Sono particolarmente utili per identificare gli Architectural Debt poichè essi sono più complessi da individuare solo attraverso il codice;
- **Commit Based:** tool che esaminano le informazioni di commit nei Versioning Control System per identificare comportamenti che possono indicare la presenza di Technical Debt. Sono molto utili per osservare come i Technical Debt si sviluppano nel tempo;
- **Architecture and Design Based:** tool focalizzati principalmente sull'architettura e il design del software, che utilizzano tali informazioni per identificare i Technical Debt radicati nella struttura architeturale.

Come evidenziato da un report sull'utilizzo di tools per la rilevazione dei code smell [21], la varietà delle metriche impiegate porta frequentemente a risultati diversificati. Inoltre, mentre alcuni di questi strumenti offrono funzionalità di refactoring automatico per affrontare i problemi rilevati, molti altri si limitano alla generazione di report. Nonostante il potenziale beneficio teorico di tali strumenti nel migliorare la qualità del codice e nella riduzione dei technical debt, la mancanza di una standardizzazione e di una documentazione chiara complica notevolmente il loro impiego pratico, rendendoli spesso poco attraenti per gli sviluppatori [21]. Questa situazione non fa altro che perpetuare la bassa frequenza di eliminazione dei Code Smells e prolungare la loro permanenza nel ciclo di vita del software.

2.2.4 ML-Specific Code Smells

Nei sistemi basati sul Machine Learning, la presenza di professionisti della Data Science all'interno dei team è spesso necessaria per accelerare e facilitare lo sviluppo. Tuttavia, poiché i Data Scientist tendono spesso a non rispettare i tradizionali standard dell'ingegneria del software [6], e considerato che i tempi di sviluppo sono sempre più stringenti, il problema dei Code Smells all'interno delle pipeline di Machine Learning risulta particolarmente rilevante. Infatti, pur rappresentando solo una piccola parte del sistema completo, la componente di Machine Learning gioca

un ruolo fondamentale al suo interno e purtroppo i Code Smells in questo ambito non sono ancora stati studiati in modo approfondito [4].

La mancanza di linee guida specifiche per la qualità del codice nelle applicazioni di Machine Learning è stata affrontata da Zhang et al. [4] proponendo un catalogo di 22 ML-Specific Code Smells che possono presentarsi durante le varie fasi delle pipeline di ML e che sono stati descritti attraverso tre componenti principali:

- **Contesto:** descrive la situazione o l'ambiente specifico in cui il Code Smell si manifesta;
- **Problema:** descrive i potenziali problemi o le conseguenze negative che possono derivare dal Code Smell se non gestito correttamente;
- **Soluzione:** descrive i metodi proposti per affrontare o eliminare il Code Smell.

All'interno della Tabella 2.1 si può osservare il catalogo completo dei 22 Code Smells e si può notare che essi sono stati ulteriormente distinti da Zhang et al. [4] in due categorie principali:

- **Generici:** i Code Smells generici includono problemi che sono comuni in molte aree del software, indipendentemente dall'uso di specifiche API per il Machine Learning. Esempi di questa tipologia sono il Code Smell "Unnecessary Iteration", che può causare problemi legati alle performance, e il Code Smell "NaN Equivalence Comparison Misused", che può condurre a comportamenti non intenzionali;
- **API-Specific:** i Code Smells API-Specific sono direttamente correlati all'uso di particolari API o librerie per il Machine Learning. Questi problemi emergono quando le API sono utilizzate in modo inappropriato o inefficiente. Ad esempio, il Code Smell 'Chain Indexing' in Pandas ¹ può portare a problemi di prestazione e codice più incline ad errori, oppure il Code Smell 'TensorArray Not Used' in TensorFlow ², che può condurre ad un utilizzo meno efficiente delle risorse e a maggiori probabilità di errori.

¹Pandas Doc: <https://pandas.pydata.org/docs/index.html>

²Tensorflow Doc: https://www.tensorflow.org/api_docs

Tabella 2.1: 22 ML-Specific Code Smells [4].

No.	Code Smell	Descrizione	Fase della Pipeline	Effetto	Tipologia
1	Unnecessary Iteration	Utilizzo di un'operazione di loop sui dati invece di una funzione vettoriale di Pandas.	Data Cleaning	Efficienza	Generico
2	NaN Equivalence Comparison Misused	Utilizzo della funzione 'np.nan' per confrontare un valore di un Data Frame con un valore NaN.	Data Cleaning	Codice Error-Prone	Generico
3	Chain Indexing	Utilizzo dell'indicizzazione a catena per accedere a un singolo valore di un Data Frame.	Data Cleaning	Codice Error-Prone, Efficienza	API-Specific: Pandas
4	Columns and Data Type Not Explicitly Set	Non specificare il nome della colonna e il tipo di dati durante la dichiarazione di un Data Frame.	Data Cleaning	Leggibilità	Generico
5	Empty Column Misinitialization	Utilizzo di zeri o stringhe vuote per inizializzare una nuova colonna in un Data Frame.	Data Cleaning	Robustezza	Generico
6	Merge API Parameter Not Explicitly Set	Non specificare le opzioni "how" e "on" durante un'operazione di merge in Pandas.	Data Cleaning	Leggibilità, Codice Error-Prone	Generico
7	In-Place APIs Misused	Presumere che la funzione di Pandas restituisca un valore in-place.	Data Cleaning	Codice Error-Prone	Generico
8	Dataframe Conversion API Misused	Utilizzo di df.values() invece di df.to_numpy() per convertire un Data Frame in un array NumPy.	Data Cleaning	Codice Error-Prone	API-Specific: Pandas
9	Matrix Multiplication API Misused	Utilizzo della funzione np.dot() per moltiplicare una matrice.	Data Cleaning	Leggibilità	API-Specific: NumPy
10	No Scaling before Scaling-Sensitive Operation	Non scalare correttamente le caratteristiche prima di operazioni sensibili alla scala.	Feature Engineering	Codice Error-Prone	Generico
11	Hyperparameter Not Explicitly Set	Non impostare esplicitamente gli iperparametri di un algoritmo di apprendimento.	Model Training	Codice Error-Prone, Riproducibilità	Generico
12	Memory Not Freed	Non liberare la memoria alla fine di un ciclo di addestramento del modello di machine learning.	Model Training	Efficienza	Generico
13	Deterministic Algorithm Option Not Used	Non impostare l'opzione deterministic_algorithms(True) durante l'addestramento del modello.	Model Training	Riproducibilità	Generico
14	Randomness Uncontrolled	Non impostare esplicitamente il seed nel processo di addestramento.	Model Training, Model Evaluation	Riproducibilità	Generico
15	Missing the Mask of Invalid Value	Non utilizzare tf.clip() o una funzione simile per evitare che i valori delle variabili diventino non validi durante l'addestramento del modello.	Model Training	Codice Error-Prone	Generico
16	Broadcasting Feature Not Used	Utilizzo di un tensore senza un'operazione di broadcasting	Model Training	Efficienza	Generico
17	TensorArray Not Used	Non utilizzare tf.TensorArray() per gestire array variabili nei loop di TensorFlow.	Model Training	Efficienza, Codice Error-Prone	API-Specific: TensorFlow 2
18	Training / Evaluation Mode Improper Toggling	Non passare correttamente tra modalità di addestramento e valutazione in deep learning.	Model Training	Codice Error-Prone	Generico
19	Pytorch Call Method Misused	Utilizzo del metodo self.net.forward() invece di self.net() per inoltrare l'input alla rete in PyTorch.	Model Training	Robustezza	API-Specific: PyTorch
20	Gradients Not Cleared before Backward Propagation	Non azzerare i gradienti con 'optimizer.zero_grad()' prima di utilizzare 'loss_fn.backward()' in PyTorch	Model Training	Codice Error-Prone	API-Specific: PyTorch
21	Data Leakage	Utilizzo di dati di addestramento contenenti informazioni sui risultati delle previsioni, causando leakage dei dati.	Model Evaluation	Codice Error-Prone	Generico
22	Threshold-Dependent Validation	Utilizzo di metriche di valutazione dipendenti da una soglia specifica invece di metriche indipendenti dalla soglia.	Model Evaluation	Robustezza	Generico

Zhang et al. [4] hanno evidenziato anche gli effetti negativi di questi Code Smells. Osservando la Tabella 2.1 è possibile capire nel dettaglio gli effetti negativi dei vari Code Smells, da cui possiamo notare che riguardano principalmente:

- **Efficienza:** alcuni Code Smells, come ad esempio "Unnecessary Iteration" e "Broadcasting Feature Not Used", possono portare ad una riduzione delle prestazioni dell'applicazione, rallentando l'esecuzione del codice o aumentando il consumo di risorse, come la memoria;
- **Codice Error-Prone:** Code smells come "NaN Equivalence Comparison Misused" e "In-Place APIs Misused" possono aumentare la probabilità di bug nel software, portando a comportamenti non intenzionali o errori di calcolo;
- **Robustezza:** "Data Leakage" e "Empty Column Misinitialization" sono esempi di Smells che minano la robustezza dell'applicazione. Un'applicazione non robusta può comportarsi in modo inaspettato di fronte a input non standard o situazioni anomale, spesso con gravi conseguenze, specialmente in ambienti di produzione.
- **Leggibilità:** Code smells come "Columns and DataType Not Explicitly Set" possono rendere il codice più difficile da leggere e da mantenere. Un codice che non è chiaro o che maschera la sua funzione può portare a errori durante le modifiche o l'aggiornamento del software.
- **Riproducibilità:** Problemi come "Deterministic Algorithm Option Not Used" impediscono la riproducibilità dei risultati del software. In ambienti scientifici o di produzione, è cruciale che gli esperimenti o i processi possano essere replicati con precisione per verificare i risultati e garantire la consistenza delle performance nel tempo.

Come si evince dalle sezioni precedenti e per quanto detto riguardo agli effetti negativi degli ML-Specific Code Smells, anche l'accumulo di questa tipologia di Code Smell può contribuire significativamente all'aumento del Technical Debt. Pertanto è essenziale identificarli ed eliminarli tempestivamente. Tuttavia, Zhang et al. [4], al di fuori di qualche esempio, offrono solo indicazioni teoriche su come rilevare e

risolvere i Code Smell da loro identificati, il che può risultare complesso in contesti applicativi reali. Una soluzione preliminare è stata proposta con CodeSmile [22], uno strumento di analisi che, tramite Abstract Syntax Tree (AST), permette di individuare i Code Smells definiti da Zhang et al. [4]. Purtroppo, anche CodeSmile presenta delle limitazioni; ad esempio, è in grado di rilevare solo 14 dei 22 Code Smells identificati, a causa della sua natura di strumento di analisi statica che non consente la rilevazione di Code Smells legati a dettagli non immediatamente evidenti nel codice [22]. Inoltre, CodeSmile genera solamente un report sui problemi rilevati, senza fornire suggerimenti per la loro correzione o funzionalità di refactoring automatico.

2.3 Chatbot

I Conversational Agent, comunemente noti come Chatbot, sono software progettati per simulare conversazioni umane. Essi elaborano e generano dati in linguaggio naturale attraverso interfacce testuali o vocali, con lo scopo di assistere gli utenti nel raggiungimento di obiettivi specifici [23].

2.3.1 Storia dei Chatbot

L'idea di Chatbot emerse nel 1950, quando A. Turing propose il test noto come "The Imitation Game" [24], volto a valutare se una macchina potesse pensare come un essere umano. Il primo esempio concreto di Chatbot fu ELIZA [25], realizzato nel 1966 da Joseph Weizenbaum presso il MIT. ELIZA era un software di elaborazione del linguaggio naturale che simulava una conversazione con uno psicoterapeuta, rispondendo ai testi degli utenti con script predefiniti.

Nel 1972, Kenneth Colby creò PARRY [26], un Chatbot più avanzato di ELIZA, che simulava coerentemente la personalità di un paziente affetto da schizofrenia. Nella Figura 2.2 è possibile osservare il risultato di un esperimento in cui PARRY e ELIZA interagivano, rispettivamente nei ruoli di paziente e psichiatra [27].

Con la diffusione di Internet negli anni '90, i Chatbot iniziarono a trovare applicazioni pratiche su larga scala. Un esempio noto di questo periodo è ALICE (Artificial Linguistic Internet Computer Entity) [28], sviluppato nel 1995 da Richard Wallace.

```
ELIZA: Good Evening. Tell me your problems. Please terminate input with a period or
a question mark.
PARRY: People get on my nerves sometimes.
ELIZA: I am not sure I understand you fully.
PARRY: You should pay more attention.
ELIZA: Suppose you should pay more attention.
PARRY: You're entitled to your own opinion.
ELIZA: What makes you think I am entitled to my own opinion?
PARRY: It bothers me just to be around people in general.
ELIZA: Please go on.
PARRY: I went to the track at Bay Meadows a while back.
ELIZA: What does that suggest to you?
PARRY: I went to the races.
.....
ELIZA: What answer would please you most?
PARRY: It's not a matter of like or dislike.
ELIZA: What else would you like to discuss?
PARRY: I am getting tired of answering the same questions over and over.
```

Figura 2.2: Conversazione tra PARRY e ELIZA.

ALICE utilizzava il linguaggio di markup AIML (Artificial Intelligence Markup Language) per migliorare la capacità di rispondere in modo più complesso e naturale rispetto ai suoi predecessori.

Nel nuovo millennio, giganti della tecnologia come Apple, Google e Amazon introdussero assistenti virtuali avanzati quali Siri [29], Google Assistant [30], e Alexa [31]. Questi assistenti impiegano tecnologie avanzate di Intelligenza Artificiale e Machine Learning per comprendere e rispondere alle richieste degli utenti in maniera sempre più naturale e utile.

2.3.2 Applicazioni dei Chatbot

I Chatbot si sono integrati così profondamente nella nostra vita di tutti i giorni che, spesso senza neanche rendercene conto, li impieghiamo per facilitare la realizzazione di molteplici attività. Come evidenziato da Motger et al. [23], questi strumenti trovano impiego in una varietà di settori, che possono essere raggruppati in sei principali contesti applicativi:

- **Vita Quotidiana:** questo ambito include il supporto alle attività quotidiane tramite Chatbot generici per Q&A e Chatbot per applicazioni specifiche come turismo, ristorazione, giochi, sport e smart home.
- **Commercio:** comprende le attività commerciali, le transazioni e il supporto clienti, prevalentemente tramite Chatbot per il servizio clienti e l'assistenza nell'e-commerce.
- **Supporto Aziendale:** in questo contesto, i Chatbot sono utilizzati per assistere i dipendenti nei processi aziendali interni, rendendo questi processi semi-automatici e più intuitivi. Include attività come gestione delle attività finanziarie, negoziazioni e supporto al lavoro di squadra.
- **Infrastruttura Tecnica:** copre la gestione tecnica e il supporto tramite Chatbot, offrendo accesso e gestione dell'assistenza per infrastrutture tecnologiche complesse e processi tecnici, dall'assistenza tecnica agli utenti fino alla gestione semi-automatica di infrastrutture industriali intelligenti.
- **Sanità:** nel settore sanitario, i Chatbot supportano sia i professionisti, in attività come la prescrizione di farmaci o il monitoraggio dei segni vitali, sia i pazienti, ad esempio nella gestione della terapia.
- **Educazione:** nel settore dell'educazione, i Chatbot vengono impiegati come strumenti e-learning per assistere gli studenti nel processo di apprendimento.

2.3.3 Tipologie di Chatbot

Esistono varie classificazioni dei Chatbot sulla base di vari parametri che ne evidenziano le diverse funzionalità e applicazioni. Una possibile classificazione dei Chatbot è basata sul dominio di conoscenza [3], ovvero la gamma di argomenti o il tipo di informazioni che un Chatbot è in grado di trattare. Questo aspetto è fondamentale per definire l'efficacia del Chatbot in diversi contesti di utilizzo, siano essi generici o specifici:

- **Open Domain:** questa tipologia di Chatbot sono progettati per discutere di una vasta gamma di argomenti senza restrizioni specifiche sul contenuto o l'area

di conoscenza. Utilizzano tecniche avanzate di NLP e AI per comprendere e generare risposte su argomenti diversificati, riuscendo così ad imitare una conversazione umana il più naturalmente possibile. Sono ideali per applicazioni generali che richiedono maggiore flessibilità. Un esempio di Chatbot Open Domain è Siri [29] di Apple, che può rispondere a domande, eseguire comandi vocali e interagire con diverse app su dispositivi Apple, gestendo una vasta gamma di richieste e argomenti.

- **Closed Domain:** questa tipologia di Chatbot sono specializzati in aree specifiche di conoscenza o settori, sono quindi programmati con una base di conoscenza limitata ma profonda, con la quale riescono a gestire domande e compiti specifici relativi al loro campo di applicazione. Sono preferiti in ambienti aziendali o professionali dove è richiesta un'alta specializzazione. Un esempio di Chatbot Closed Domain è ELIZA [25], in quanto le sue capacità sono limitate al dominio psicoterapeutico.

Un'altra possibile classificazione dei Chatbot è invece basata sul metodo di elaborazione dell'input e di generazione della risposta [3]:

- **Rule-Based:** Chatbot che utilizzano un insieme predefinito di regole per determinare la risposta ad ogni input dell'utente. Questi Chatbot tendono ad essere meno flessibili e non generano nuove risposte ma selezionano risposte preimpostate basandosi sull'input riconosciuto, e per questo motivo tendono ad essere meno robusti agli errori grammaticali o di ortografia nell'input dell'utente;
- **Retrieval-Based:** Chatbot che utilizzando appositi algoritmi per valutare e selezionare la risposta più appropriata da una fonte di conoscenza predefinita. Offrono una maggiore flessibilità rispetto ai Rule-Based e possono adattarsi a variazioni negli input degli utenti;
- **Generative-Based:** Chatbot che utilizzano tecniche di ML o Deep Learning per produrre nuove risposte partendo dagli input ricevuti, senza limitarsi a selezionare risposte preimpostate. Offrono un'interazione più naturale e possono adattarsi a contesti conversazionali complessi, gestendo meglio le ambiguità e le sfumature del linguaggio.

2.3.4 Tecnologie di Base e Architettura dei Chatbot

Ciò che differenzia i Chatbot dai Bot è la capacità dei primi di comprendere il linguaggio naturale e di comunicare tramite esso. Infatti, alla base dei Chatbot vi sono la Natural Language Processing (NLP) e la Natural Language Understanding (NLU).

La Natural Language Processing (NLP) è una branca dell'Intelligenza Artificiale che si occupa dell'interazione tra i computer e il linguaggio umano. Il suo scopo principale è quello di permettere ai computer di comprendere, interpretare e rispondere al linguaggio naturale in modo utile e significativo [32]. La Natural Language Understanding (NLU), invece, è una sottodisciplina della NLP che si concentra sulla comprensione del significato e del contesto del linguaggio naturale. Mentre la NLP può riguardare vari aspetti del linguaggio, come la sintassi e la morfologia, la NLU è specificamente interessata all'estrazione di conoscenze semantiche strutturate dal testo [32].

NLU e NLP sono fondamentali affinché il Chatbot riesca a comprendere gli intent, ad estrarre le entity e a tenere traccia del contesto della conversazione [3]:

- **Intent:** mapping tra la richiesta dell'utente e l'action che il chatbot dovrà eseguire;
- **Action:** sequenza di azioni che il chatbot deve compiere per soddisfare uno specifico intent;
- **Entity:** parametri estratti dalla richiesta che aiutano il chatbot a soddisfare al meglio la richiesta;
- **Context:** tiene traccia dell'argomento a cui l'utente sta facendo riferimento.

Analizzando l'architettura di riferimento di un chatbot [3], come si può osservare anche nella Figura 2.3, essa è formata principalmente da:

- **Language Understanding Component:** analizza l'input dell'utente, solitamente in forma di testo o voce, per dedurre l'intent ed estrarre le entity utilizzando tecniche di NLP e NLU;

- **Dialog Management Component:** gestisce il flusso della conversazione, tiene traccia del contesto e gestisce le sessioni utente, assicurandosi che la conversazione proceda in modo logico e coerente;
- **Action Execution Component:** si occupa di eseguire le azioni necessarie per soddisfare le richieste dell'utente basandosi sugli intent e le entity estratte. Può interagire con database esterni, sistemi di backend o API per recuperare informazioni o eseguire operazioni;
- **Response Generation Component:** questo componente si occupa di formulare la risposta del chatbot in un formato comprensibile all'utente;

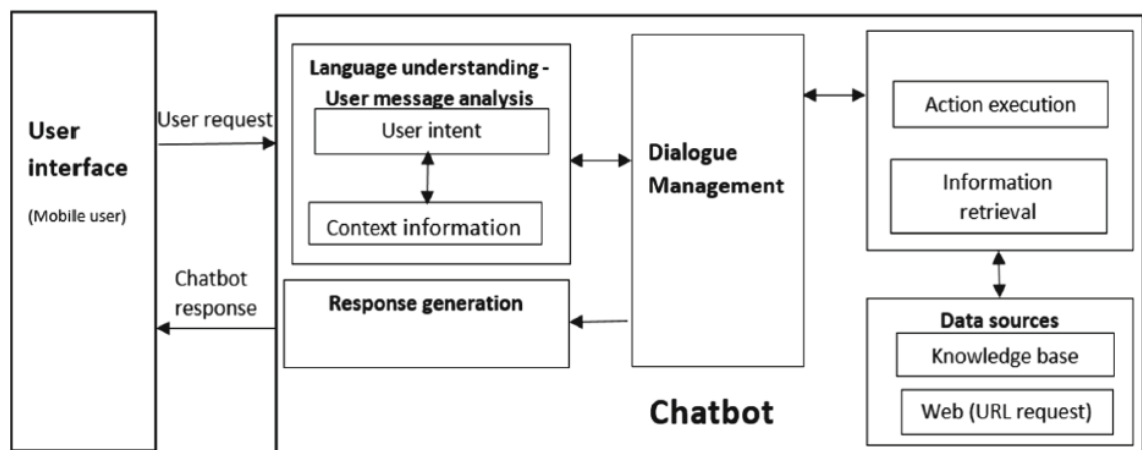


Figura 2.3: Architettura di un Chatbot [3].

Per comprendere meglio il comportamento di un Chatbot e il funzionamento dei vari componenti dell'architettura di riferimento (Figura 2.3), consideriamo un esempio di un Chatbot progettato per un servizio di pizzeria online, il cui compito è aiutare gli utenti ad ordinare pizze, rispondere a domande sul menù e gestire le consegne. Tutte le interazioni tra utente e chatbot ovviamente avverranno attraverso un canale di comunicazione.

- Utente: "Vorrei ordinare una pizza."

Il Language Understanding Component analizza la frase e identifica l'intent "OrderPizza". A questo punto, non sono state specificate entità, quindi il Dialog Management Component procede richiedendo maggiori dettagli.

- Chatbot: "Che tipo di pizza desideri?"
- Utente: "Una Margherita, per favore."

Ora, il Language Understanding Component del Chatbot riconosce l'entità "PizzaType = Margherita". Utilizzando queste informazioni, il Dialog Management Component mantiene il contesto dell'ordine e passa alla conferma del tipo e alla richiesta di ulteriori preferenze.

- Chatbot: "Vuoi aggiungere delle modifiche speciali o altro al tuo ordine?"
- Utente: "No, grazie. Quanto costa?"

Il chatbot, attraverso l'Action Execution Component, consulta il database per il prezzo della pizza Margherita e utilizza il Response Generation Component per informare l'utente.

- Chatbot: "La pizza Margherita costa 8 euro. Vuoi procedere con l'ordine?"
- Utente: "Sì, procediamo."

Il chatbot finalizza l'ordine e fornisce le istruzioni per il pagamento...

2.3.5 Bot e Chatbot nel Software Engineering

Recentemente, il campo dell'ingegneria del software ha assistito a un significativo incremento nell'adozione di Bot e Chatbot. Questo aumento è attribuibile principalmente alla loro efficacia nell'automatizzare compiti ripetitivi, che altrimenti richiederebbero un considerevole impegno in termini di ore di lavoro manuale. L'impiego di questi strumenti automatizzati consente ai team di sviluppo di concentrare la loro attenzione su attività più strategiche e creative, ottimizzando così la gestione delle risorse umane e aumentando l'efficienza operativa complessiva.

Santhanam et al. [33] hanno fornito una categorizzazione dei Bot utilizzati dagli ingegneri del software basata sul loro contesto applicativo:

- **Q&A bots:** Bot che forniscono assistenza immediata agli sviluppatori e agli utenti, accedendo rapidamente a vaste basi di dati e informazioni. Vengono impiegati per recuperare rapidamente documentazione tecnica o per fornire supporto decisionale basato su dati storici.

- **Managing software repositories:** Bot utilizzati per la gestione delle repository che automatizzano e semplificano molteplici compiti ad esse associati. Vengono utilizzati per automatizzare le pull request, per gestire i commit, per gestire i processi di Continuous Integration (CI) e per il tracking di bug/issues.
- **Software development environment:** Bot utilizzati come assistenti nel processo di sviluppo software, aiutando a ottimizzare l'efficienza dello sviluppo attraverso l'automazione di compiti ripetitivi e fornendo supporto al workflow di sviluppo attraverso suggerimenti basati su best practices e analisi precedenti.

Anche se la classificazione sopra citata si riferisce ai Bot, è importante considerare che i Chatbot sono semplicemente una sottocategoria dei primi, pertanto questa classificazione è applicabile anche ad essi [33]. Inoltre, in letteratura sono presenti molti esempi concreti di Chatbot che trovano applicazione nell'ingegneria del software: Sayme [34], un Chatbot per il rilevamento e la risoluzione dei conflitti che possono sorgere quando più sviluppatori lavorano sullo stesso codice; CADOCS [35], un Conversational Agent per l'identificazione e il refactoring dei Community Smells nelle comunità di sviluppo software su Slack; Stephen [36], un Conversational Agent in grado di supportare gli sviluppatori fornendo best practices per lo sviluppo di Bot; un ulteriore esempio è un chatbot (Expert Recommender Chatbot) [37] integrato con Discord, progettato per aiutare gli sviluppatori a identificare la persona più adeguata a cui rivolgersi per assistenza all'interno di progetti open source.

CAMILLE: Conversational Agent for Machine Learning smells Elimination

Nei capitoli precedenti, abbiamo esplorato le sfide associate alla presenza di Technical Debt e Code Smells nei prodotti software. Abbiamo particolarmente sottolineato come queste problematiche siano accentuate nel contesto del Machine Learning (ML), a causa della natura complessa delle applicazioni di ML e alla relativa carenza di ricerche e risorse che affrontano questi problemi specifici. In aggiunta, abbiamo discusso il ruolo dei Chatbot e il loro potenziale impatto positivo nel supportare gli ingegneri del software nelle loro attività quotidiane, migliorando l'efficienza e la qualità del lavoro svolto.

Questo contesto ha portato all'ideazione e allo sviluppo di CAMILLE. Implementato come un Chatbot integrato in un'estensione per Visual Studio Code, CAMILLE mira ad assistere i Machine Learning Engineers durante le diverse fasi delle pipeline di ML. Il suo obiettivo principale è quello di fornire linee guida pratiche e informazioni cruciali per migliorare la qualità del codice, permettendo di eliminare gli ML-Specific Code Smells e prevenendo l'accumulo di Technical Debt nelle applicazioni.

In questo capitolo, saranno descritte nel dettaglio l'architettura e le tecnologie impiegate per realizzare CAMILLE, analizzando le sue funzionalità principali. Verranno evidenziate le modifiche apportate a CodeSmile [22], il tool di analisi del codice utilizzato da CAMILLE, discutendo come queste modifiche hanno permesso l'intergrazione con il Chatbot. Infine, verranno descritti i metodi di testing adottati per validare l'efficacia del tool.

3.1 Architettura Generale di CAMILLE

In questa sezione verranno discusse le tecnologie utilizzate per la realizzazione di CAMILLE, motivando opportunamente le scelte effettuate. Inoltre verrà presentata una panoramica dell'architettura del tool, mostrando i vari componenti e come questi interagiscono tra loro.

3.1.1 Tecnologie Utilizzate

Durante la fase di progettazione, sono state valutate diverse tecnologie per la realizzazione di CAMILLE, selezionando tra queste quelle che meglio si adattavano alle esigenze e agli obiettivi del progetto. Nella seguente sezione, verranno descritte le tecnologie impiegate per realizzare il Chatbot e l'estensione per Visual Studio Code, evidenziando le motivazioni dietro ciascuna scelta.

Sviluppo del Chatbot

- **Rasa** [38]: Questo framework open-source è stato scelto per lo sviluppo del chatbot grazie alle sue avanzate capacità nella gestione delle conversazioni e nell'elaborazione del linguaggio naturale. Rasa offre un alto grado di personalizzazione e scalabilità, essenziali per soddisfare le esigenze specifiche di CAMILLE. Un fattore decisivo nella sua scelta è stata l'ampia disponibilità di risorse, come documentazione dettagliata e video tutorial, che hanno facilitato la fase di implementazione e l'utilizzo del framework.

- **Python:** La scelta di Python come linguaggio di programmazione è stata guidata dalla sua integrazione con Rasa e dall'ampia disponibilità di librerie per il machine learning e l'elaborazione del linguaggio naturale.
- **GPT-4o mini:** Questo Large Language Model fornito da OpenAI è stato integrato tramite API per permettere a CAMILLE di fornire suggerimenti di refactoring per la rimozione dei Code Smells. La scelta di GPT-4o mini è stata influenzata dalla necessità di una soluzione che rientrasse nel budget del progetto e che, allo stesso tempo, garantisse prestazioni elevate. È stata valutata anche l'idea di ospitare un modello localmente, ma, dopo alcune prove, è stata scartata a causa dell'indisponibilità di risorse hardware adeguate.
- **MongoDB:** Per la gestione dei dati persistenti, è stato scelto MongoDB, un database NoSQL orientato ai documenti. Questa scelta è stata motivata principalmente dalla natura e dal volume dei dati attualmente gestiti da CAMILLE, che riguardano esclusivamente informazioni relative alle varie tipologie di Code Smell. Inoltre, rispetto a un database relazionale, MongoDB offre una maggiore flessibilità nella gestione di dati semi-strutturati, permettendo di archiviare e recuperare informazioni senza dover definire uno schema rigido. Questa caratteristica risulta particolarmente vantaggiosa in un contesto in cui la struttura dei dati potrebbe evolvere nel tempo. Un ulteriore vantaggio è la possibilità di integrare facilmente MongoDB con il componente Tracker di Rasa, che può essere configurato per utilizzare questo database per la persistenza automatica delle conversazioni.

Sviluppo dell'estensione

- **Visual Studio Code:** L'IDE è stato scelto per la sua popolarità e il vasto supporto alla creazione e gestione delle estensioni, oltre che per la sua comunità di sviluppatori attiva e risorse documentali ricche.
- **TypeScript:** Per lo sviluppo dell'estensione, si è optato per TypeScript, che grazie al suo sistema di tipi statici, migliora la manutenibilità e la robustezza del codice rispetto a JavaScript;

- **Yeoman:** Utilizzato per lo scaffolding del progetto, Yeoman ha aiutato a strutturare e automatizzare la configurazione iniziale del progetto;
- **Webview API:** Per l'interfaccia utente dell'estensione, è stata utilizzata la Web-View API di Visual Studio Code, che permette di incorporare contenuti HTML, CSS e JavaScript direttamente nell'editor, facilitando la creazione di interfacce utente complesse e interattive.

Tool di Analisi del Codice

- **CodeSmile:** CodeSmile [22] è stato scelto come tool di analisi statica per CAMILLE in quanto, attualmente, risulta essere l'unico tool in grado di rilevare buona parte degli ML-Specific Code Smells all'interno del codice;
- **Flask:** Flask è stato utilizzato per esporre le funzionalità di CodeSmile tramite API, facilitando l'integrazione con il Chatbot. Questo micro-framework per Python è stato scelto per la sua leggerezza e flessibilità, ideali per lo sviluppo rapido di servizi web.
- **GitHub API:** L'API di GitHub è stata utilizzata per permettere a CodeSmile di accedere e analizzare il codice contenuto nelle repository GitHub.

3.1.2 Panoramica dell'Architettura

Questa sezione fornisce una panoramica dettagliata dell'architettura di CAMILLE, illustrando la distribuzione dei vari componenti software e le loro interazioni. L'obiettivo è offrire una chiara comprensione della struttura e del funzionamento del sistema, facilitandone la manutenzione e gli sviluppi futuri.

Come evidenziato in Figura 3.1, CAMILLE è costituito da diversi componenti ospitati principalmente su host locali, con l'integrazione di servizi esterni per funzionalità più avanzate. I componenti principali di CAMILLE includono:

- **CodeSmile:** CodeSmile [22] è il tool di analisi del codice utilizzato da CAMILLE. Questo strumento permette di avviare analisi delle repository GitHub utilizzando la libreria GitPython, che supporta il cloning e il pulling delle repository. CodeSmile offre i suoi servizi tramite un'API RESTful realizzata con Flask.

- **Chatbot:** il Chatbot rappresenta il nucleo funzionale di CAMILLE. Utilizzando la libreria PyMongo, il Chatbot recupera informazioni sui vari Code Smell dal database MongoDB. Inoltre, può richiedere l'analisi del codice contenuto in una repository GitHub tramite l'API esposta da CodeSmile. Il Chatbot utilizza anche l'API di OpenAI per ottenere suggerimenti di refactoring dal LLM GPT-4o mini e fornirli poi agli utenti. il Chatbot dispone di un endpoint REST, realizzato con il framework Rasa, che consente a servizi esterni di interagire con esso tramite richieste HTTP.
- **Estensione:** l'estensione, eseguita come una Webview all'interno di Visual Studio Code, comunica con il Chatbot inviando richieste HTTP POST al suo endpoint REST. Questa interazione permette agli utenti di accedere ai servizi offerti dal Chatbot attraverso un'interfaccia grafica intuitiva direttamente all'interno dell'IDE.

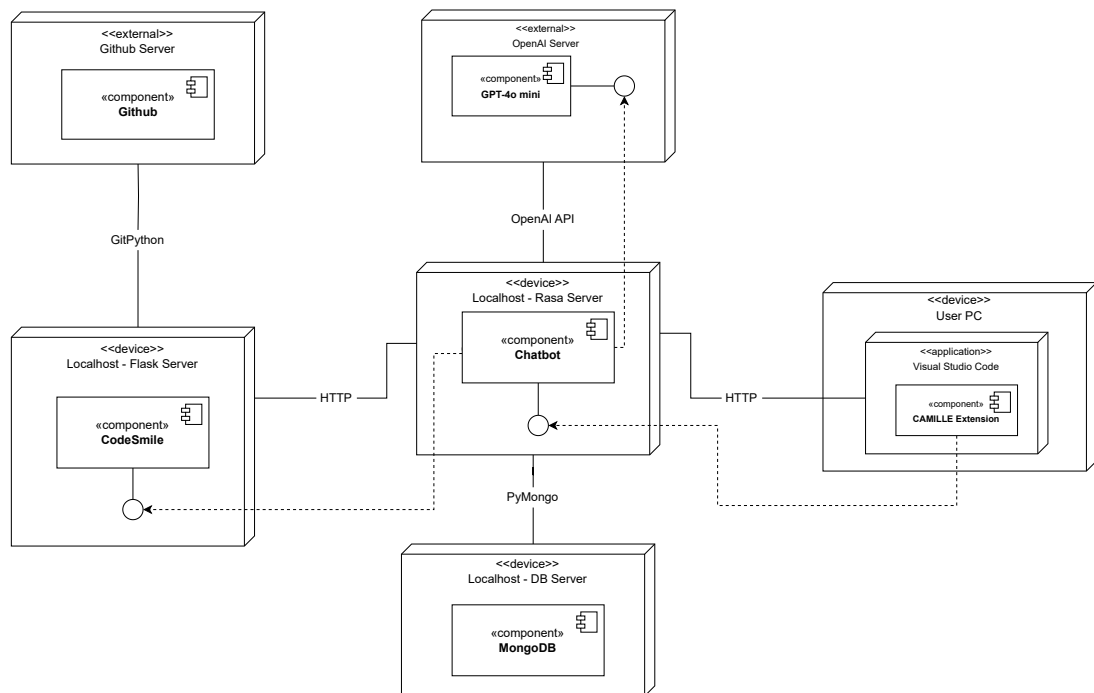


Figura 3.1: Deployment Diagram di CAMILLE.

L'architettura modulare di CAMILLE offre numerosi vantaggi, tra cui la possibilità di modificare ed estendere i vari componenti in modo indipendente. Questa

flessibilità è fondamentale per l'adattamento e l'evoluzione del sistema in risposta a nuove esigenze o opportunità tecnologiche. Inoltre, il design aperto del Chatbot permette la sua integrazione non solo in altre estensioni o plugin ma anche in applicazioni esterne, ampliando così il raggio di utilizzo del sistema. L'utilizzo di un tool di analisi del codice tramite API apre inoltre la possibilità di estendere ulteriormente CAMILLE per incorporare altri strumenti analitici, sostenendo così una crescita continua delle capacità e delle funzionalità del sistema.

3.2 Modifiche al Tool di Analisi

Per consentire agli utenti di avviare analisi delle repository GitHub al fine di rilevare la presenza di ML-Specific Code Smells, CAMILLE si avvale di CodeSmile [22], uno strumento precedentemente descritto nella sezione 2.2.4. CodeSmile, essendo ancora in fase sperimentale, non possedeva inizialmente tutte le funzionalità necessarie per CAMILLE, né era predisposto per l'integrazione con altri sistemi. Pertanto, è stato necessario estendere e modificare le sue funzionalità per adattare alle specifiche esigenze di CAMILLE.

Le modifiche apportate a CodeSmile saranno dettagliate nella sezione seguente, evidenziando le ragioni che hanno reso necessari tali interventi.

3.2.1 Analisi delle repository GitHub

Inizialmente, CodeSmile [22] permetteva solo l'analisi di progetti memorizzati localmente. Tuttavia, per mantenere una separazione efficace tra il tool di analisi, il Chatbot e l'estensione, è stata necessaria una soluzione alternativa. Sono state valutate due opzioni:

- **Clonazione delle repository GitHub:** Permettere a CodeSmile di clonare le repository, memorizzarle sul proprio server e quindi avviare l'analisi del codice.
- **Ricezione dei progetti dal Chatbot:** Abilitare CodeSmile a ricevere i progetti direttamente dal Chatbot mediante un apposito protocollo, per poi procedere con l'analisi.

Dopo un'attenta valutazione, è stata scelta l'opzione di analisi diretta delle repository GitHub. L'invio di progetti tramite protocollo HTTP sarebbe risultato significativamente più oneroso e complesso, data la grande dimensione tipica dei progetti di ML e considerando l'architettura modulare di CAMILLE. La soluzione adottata sfrutta la libreria PyGithub, che offre metodi già ottimizzati per il cloning e il pulling delle repository, semplificando e velocizzando l'intero processo di analisi.

La Figura 3.2 illustra il codice della funzione che permette di analizzare una repository GitHub. Il processo di analisi si svolge attraverso le seguenti fasi:

1. **Identificazione della Repository:** L'URL fornito viene analizzato per determinare il nome del proprietario e il nome della repository. Questi dati sono essenziali per definire i percorsi di input e output, evitando sovrapposizioni con altri progetti che potrebbero avere nomi identici;
2. **Clonazione o Aggiornamento del Repository:** La repository specificata viene clonata sul server di CodeSmile. Se la repository è già presente, viene effettuato un aggiornamento tramite l'operazione di pull, garantendo che l'analisi venga eseguita sulla versione più recente del codice e riducendo il tempo necessario per eseguirla;

```
1 def analyze_github_repository(repo_url, input_base='cs_detection_tool/input/projects/',
2                               output_base='cs_detection_tool/output/projects_analysis/',
3                               result_filename="to_save.csv"):
4     try:
5         # Obtain Repo Details
6         creator_name, repo_name = get_repo_details(repo_url)
7
8         # Build Input/Output Paths
9         input_path = os.path.join(input_base, creator_name, repo_name)
10        output_path = os.path.join(output_base, creator_name, repo_name)
11        result_file = os.path.join(output_path, result_filename)
12
13        # Clone or Pull the Repo
14        clone_or_pull_repo(repo_url, input_path)
15
16        # Create Project Analysis Folder
17        os.makedirs(output_path, exist_ok=True)
18
19        # Run the Project Analysis
20        analyze_project(input_path, output_path, result_filename)
21
22        if os.path.exists(result_file):
23            return result_file
24        else:
25            logging.warning(f"Result file {result_filename} does not exist after analysis")
26            return None
27    except Exception as e:
28        logging.error(f"Error analyzing repository '{repo_url}': {e}")
29        return None
30
```

Figura 3.2: Funzione "analyze_github_repository".

3. **Analisi del Codice:** Una volta memorizzato il progetto localmente, si procede con l'analisi del codice attraverso la funzione preesistente "analyze_project";
4. **Generazione del Report:** Al termine dell'analisi, il sistema genera un report dettagliato. Questo report documenta ogni Code Smell trovato, fornendo informazioni come il nome del file, la funzione coinvolta, il nome del Code Smell rilevato e una breve descrizione del problema.

3.2.2 Recupero del corpo di una funzione

Per fornire suggerimenti di refactoring personalizzati, il Chatbot necessita di accedere al codice dei progetti analizzati. Tuttavia, come discusso nel paragrafo precedente (vedi 3.2.1), i progetti sono memorizzati esclusivamente sul server del tool di analisi. Nonostante ciò, il report generato dal tool di analisi fornisce dettagli sul file e sulla funzione all'interno dei quali sono stati identificati i Code Smells, facilitando così l'implementazione della funzione di utility illustrata nella Figura 3.3.

```
1 def get_function_body(filename, function_name):
2     # Construct the base path for project files
3     project_base_path = os.path.join(os.path.dirname(__file__), '../input/projects/')
4     file_path = os.path.join(project_base_path, filename)
5
6     try:
7         # Open the file and build AST
8         with open(file_path, 'r') as file:
9             tree = ast.parse(file.read(), filename=filename)
10
11         # Traverse the AST
12         for node in ast.walk(tree):
13             if isinstance(node, ast.FunctionDef) and node.name == function_name:
14                 start_line = node.lineno
15                 end_line = node.body[-1].lineno
16
17                 # Open the file and extract function body
18                 with open(file_path, 'r') as file:
19                     lines = file.readlines()
20                 function_body = ''.join(lines[start_line - 1:end_line])
21                 return function_body
22     except Exception as e:
23         logging.error(f"Error analyzing the file '{file_path}': {e}")
24         return None
25     return None
```

Figura 3.3: Funzione "get_function_body".

La funzione "get_function_body" permette di estrarre il corpo di una funzione specifica da un file di codice sorgente, seguendo i seguenti passaggi:

1. **Costruzione del Percorso del File:** Il processo inizia definendo il percorso completo al file di codice sorgente, basandosi sul nome del file fornito.
2. **Generazione dell'AST:** Il file specificato viene aperto per generare un Abstract Syntax Tree (AST), che rappresenta la struttura del codice in un formato leggibile e manipolabile.
3. **Ricerca della Funzione Specificata:** Utilizzando l'AST, il sistema itera sui nodi per trovare la funzione desiderata basandosi sul suo nome. Vengono identificate le linee di inizio e di fine della funzione all'interno del file di codice.
4. **Estrazione del Corpo della Funzione:** Una volta individuate le linee che delimitano la funzione, il file viene aperto di nuovo per leggere e estrarre il testo compreso tra l'inizio e la fine della funzione specificata.
5. **Concatenazione delle Linee Estratte:** Le linee estratte vengono concatenate per formare una stringa che rappresenta l'intero corpo della funzione, che viene poi restituito in output.

Questo processo garantisce che il Chatbot possa accedere al codice specifico necessario per fornire suggerimenti di refactoring accurati e personalizzati, come verrà mostrato in seguito nella Sezione 3.3.7.

3.2.3 Definizione API

Per facilitare la comunicazione tra il Chatbot e il tool di analisi, è stata necessaria la definizione di un'API specifica. Questa API è stata implementata utilizzando il framework Flask, noto per la sua leggerezza e flessibilità. L'API espone al Chatbot i servizi sviluppati attraverso le funzioni precedentemente descritte.

Endpoint '/analyze-repository'

Questo endpoint, illustrato nella Figura 3.4 e accessibile tramite il metodo HTTP POST, innesca il processo di analisi del codice di una repository. Richiede come

parametro di input l'URL di una repository GitHub, dopodichè effettua le seguenti operazioni:

1. **Avvio dell'Analisi:** Avvia l'analisi del codice della repository specificata per identificare i Code Smells.
2. **Modifica dei Risultati:** Se l'analisi produce risultati validi e il file con i risultati viene creato correttamente, il sistema carica questi dati in un DataFrame Pandas e modifica la colonna "filename" per includere solo il percorso relativo, utilizzando la funzione di utility "extract_relative_path".
3. **Risposta:** Il DataFrame viene convertito in una lista di record e restituito come risposta in formato JSON.

```

1 @app.route('/analyze-repository', methods=['POST'])
2 def analyze_repository():
3     data = request.get_json()
4     repository_url = data.get('repository_url')
5
6     if not repository_url:
7         return jsonify({"error": "Missing 'repository_url' parameter."}), 400
8
9     # Run repository analysis
10    result_file = analyze_github_repository(repository_url)
11    if not result_file:
12        return jsonify({"error": "Failed to analyze repository or no results found."}), 500
13
14    df = pd.read_csv(result_file)
15
16    # Return only Relative Paths
17    df['filename'] = df['filename'].apply(extract_relative_path)
18    results = df.to_dict(orient='records')
19
20    return jsonify(results), 200

```

Figura 3.4: Endpoint API per richiedere l'analisi di una repository.

Endpoint '/get-function-body'

Questo endpoint, rappresentato nella Figura 3.5 e anch'esso accessibile tramite metodo HTTP POST, richiede come parametri il nome di un file e il nome di una funzione, dopodichè effettua le seguenti operazioni:

1. **Recupero del Corpo della Funzione:** Estrae il corpo della funzione designata dal file specificato utilizzando la funzione "get_function_body".

2. **Risposta:** Il corpo della funzione estratto al passo precedente viene restituito come risposta in formato JSON.



```

1 @app.route('/get-function-body', methods=['POST'])
2 def get_function_body_endpoint():
3     data = request.get_json()
4     filename = data.get('filename')
5     function_name = data.get('function_name')
6
7     if not filename or not function_name:
8         return jsonify({"error": "Missing 'filename' or 'function_name' parameter"}), 400
9
10    # Retrieve function body
11    function_body = get_function_body(filename, function_name)
12
13    if function_body:
14        return jsonify({"function_body": function_body})
15    else:
16        return jsonify({"error": "Function not found"}), 404

```

Figura 3.5: Endpoint API per richiedere il corpo di una funzione.

3.3 Sviluppo del Chatbot

All'interno della seguente sezione verrà illustrato l'impiego del framework Rasa per la realizzazione della componente chiave di CAMILLE, ovvero, il Chatbot. Si esploreranno dettagliatamente le strutture di base e le personalizzazioni che permettono a CAMILLE di operare efficacemente, tra cui la gestione del linguaggio naturale, la logica di dialogo, e l'interazione con servizi esterni tramite custom actions. Inoltre, verrà anche trattata l'architettura interna del chatbot, fornendo una panoramica sui componenti chiave che ne consentono il funzionamento.

3.3.1 Introduzione al Framework Rasa

Rasa [38] è una piattaforma Open Source per la costruzione di chatbot e assistenti virtuali. Questo framework offre strumenti avanzati per l'elaborazione del linguaggio naturale (NLU) e la gestione dei dialoghi, permettendo di creare assistenti virtuali capaci di comprendere e rispondere in modo efficace agli input degli utenti.

Rasa è disponibile in diverse versioni, inclusa una versione enterprise con funzionalità aggiuntive per le esigenze aziendali. Tuttavia, per lo sviluppo di CAMILLE, la

versione Rasa Open Source 3.1 è stata più che sufficiente, grazie alla sua robustezza e flessibilità.

Tutti i concetti descritti nel capitolo dedicato allo stato dell'arte dei Chatbot (vedi sez. 2.3.4), si applicano anche ai Chatbot realizzati utilizzando il framework Rasa. Tuttavia, per comprendere al meglio tutto ciò che verrà presentato nelle sezioni successive, è bene chiarire il significato di ulteriori termini/concetti fondamentali per Rasa:

- **Stories:** Le stories descrivono sequenze di interazioni tra l'utente e il Chatbot;
- **Rules:** Definiscono le azioni che il chatbot deve intraprendere in risposta ad input specifici;
- **Responses:** Rappresentano le risposte che il Chatbot fornisce agli utenti. Solitamente contengono testo ma possono contenere anche altri tipi di informazioni, come ad esempio delle immagini;
- **Custom Actions:** Azioni personalizzate definite dallo sviluppatore, che permettono al chatbot di eseguire logiche di business specifiche o di interagire con servizi esterni;
- **Slots:** Gli slots in Rasa sono variabili che memorizzano informazioni estratte durante la conversazione. Essi aiutano a mantenere il contesto e a personalizzare le risposte del chatbot in base alle informazioni fornite dagli utenti.

3.3.2 Architettura del Chatbot

Il framework Rasa adotta un'architettura interna standard ben definita [39]. La componente Chatbot di CAMILLE, realizzata tramite Rasa, segue pertanto questa architettura. Di seguito viene descritta l'architettura del Chatbot, come illustrato nella Figura 3.6, che offre una panoramica dei principali componenti.

RASA Open Source

- **NLU Pipeline:** Questa componente è responsabile dell'analisi del linguaggio naturale. Riceve i messaggi dell'utente e li elabora per estrarre gli Intent e le Entity utilizzando diverse tecniche di NLP;
- **Dialogue Policies:** Le politiche di dialogo determinano il comportamento del Chatbot in base al contesto della conversazione. Utilizzano storie di conversazione predefinite e modelli di Machine Learning per decidere la prossima azione da intraprendere;
- **Agent:** L'agente è il componente centrale che coordina le interazioni tra la pipeline di NLU, le politiche di dialogo e gli altri moduli. L'agente riceve i messaggi, li passa alla pipeline di NLU per l'interpretazione, applica le politiche di dialogo per decidere l'azione successiva, e comunica attraverso i canali di input/output per inviare la risposta all'utente.

RASA SDK

- **Action Server:** Questo componente gestisce le azioni personalizzate richieste durante la conversazione. L'Action Server esegue logiche di business specifiche e interagisce con servizi esterni, come nel nostro caso con il database MongoDB, l'API del tool di analisi o l'API di OpenAI.

Altri Componenti

- **Tracker Store:** Il Tracker Store memorizza lo stato della conversazione, permettendo all'agente di mantenere il contesto e tracciare le interazioni dell'utente durante il dialogo. È possibile configurare il componente Tracker per utilizzare un database, come ad esempio MongoDB, per persistere le informazioni rilevanti della conversazione;
- **Lock Store:** il Lock Store gestisce i lock per garantire la coerenza delle conversazioni in ambienti concorrenti, evitando conflitti tra diverse istanze del Chatbot che possono accedere simultaneamente agli stessi dati di conversazione.

- **Filesystem:** Il Filesystem viene utilizzato principalmente per archiviare i modelli di Machine Learning e i dati di training necessari per la pipeline di NLU e per le politiche di dialogo.
- **Input/Output Channels:** I canali di input/output rappresentano l'interfaccia tra il Chatbot e l'utente. Questi canali gestiscono la ricezione dei messaggi dell'utente e l'invio delle risposte generate dal Chatbot.

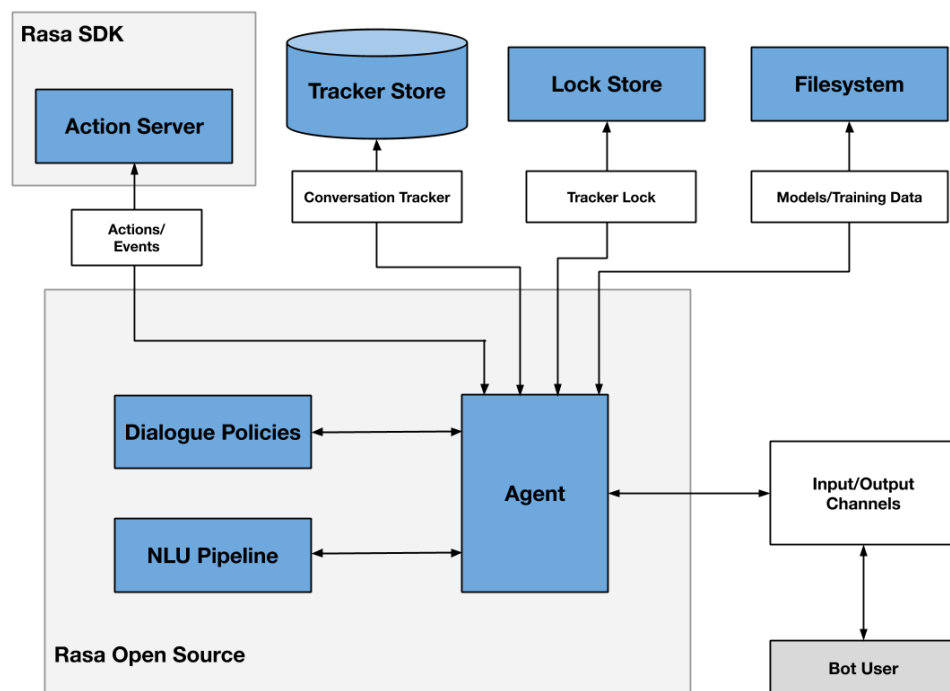


Figura 3.6: Architettura di un Chatbot Rasa.

3.3.3 Database

Il Database rappresenta un pilastro fondamentale per il funzionamento di CAMILLE. Molte delle funzionalità core del Chatbot si basano infatti sull'accesso a informazioni specifiche conservate proprio all'interno del Database. Pertanto, al fine di ottimizzare le interrogazioni, garantire la modularità e favorire la riusabilità del codice, sono state sviluppate delle specifiche funzioni di query.

Nella seguente sezione, verranno esaminate nel dettaglio tutte le query impiegate dal chatbot, insieme alle funzioni di utility che facilitano la loro esecuzione. Sarà quindi illustrato come queste funzioni interagiscono con il database per recuperare,

elaborare e restituire dati in maniera efficace, supportando così le funzionalità di CAMILLE.

Struttura del Database

Il database MongoDB comprende attualmente una sola collezione denominata "codesmells". Questa collezione, i cui campi sono descritti dettagliatamente nella Tabella 3.1, archivia tutti i dati relativi ai vari Code Smell necessari alla realizzazione delle funzionalità principali di CAMILLE.

Tabella 3.1: Campi della collection "codesmells".

Campo	Descrizione
id	Indice identificativo del Code Smell.
name	Nome del Code Smell.
description	Descrizione del Code Smell e del contesto in cui si presenta.
problems	Descrizione delle problematiche causate dal Code Smell.
solution	Descrizione delle possibili soluzioni per evitare o rimuovere il Code Smell.
bad_example	Esempio di codice contenente il Code Smell.
good_example	Esempio di codice contenente la soluzione corretta.
prompt_example	Esempio compatto contenente sia codice corretto che codice affetto dal Code Smell. È ideale per essere inviato ad un LLM perchè è composto da un numero di token più basso.
type	Tipologia del Code Smell (Generic o API-Specific)

Creazione e popolamento

Il processo di creazione e popolamento del database è gestito attraverso il file "mongo_db.py", illustrato nella Figura 3.7. L'esecuzione di questo file avvia la funzione "main", che coordina l'intero processo di configurazione e popolamento del database. Le operazioni chiave compiute da questa funzione sono organizzate come segue:

1. **Caricamento delle Credenziali di Accesso:** Per ragioni di sicurezza, tutte le credenziali di accesso al database sono conservate in un file ".env". Queste vengono

caricate nella funzione principale tramite "dotenv.load_dotenv", garantendo che le informazioni sensibili rimangano protette.

2. **Connessione al Database:** Utilizzando la funzione "get_mongo_client", si stabilisce la connessione al database. Questa funzione sfrutta l'URI salvato nella variabile d'ambiente "MONGO_URI" per configurare correttamente la connessione.

```
1 import pymongo
2 from dotenv import load_dotenv
3 import os
4 import json
5
6 load_dotenv() # Load environment variables
7
8 # Connects to MongoDB
9 def get_mongo_client():
10     return pymongo.MongoClient(os.getenv("MONGO_URI"))
11
12
13 # Retrieves the collection
14 def get_mongo_collection(client):
15     return client[os.getenv("MONGO_DB")]["codesmells"]
16
17
18 def main():
19     with get_mongo_client() as client:
20         collection = get_mongo_collection(client)
21         collection.create_index("id", unique=True)
22         print("DB Created.")
23
24     # Read the data from JSON file
25     with open('data.json', 'r') as file:
26         data = json.load(file)
27
28     # Converts the "id" fields in int
29     for item in data:
30         if isinstance(item.get('id'), str):
31             item['id'] = int(item['id'])
32
33     # Populate
34     collection.insert_many(data)
35     print("DB Populated.")
36
37     collection.create_index("name")
38     print("Indexes created.")
39
40
41 if __name__ == "__main__":
42     main()
43
```

Figura 3.7: File "mongo_db.py"

3. **Accesso alla Collezione:** Viene effettuato l'accesso alla collezione "codesmells" del Database specificato nella variabile "MONGO_DB". Durante questa fase, si procede anche con l'indicizzazione del campo "id" della collezione per migliorare l'efficienza delle query.
4. **Lettura e Inserimento dei Dati:** I dati necessari per il popolamento del database sono letti dal file "data.json". Il campo "id" in questi dati viene convertito da stringa a intero per prevenire problemi dovuti ad un eventuale ordinamento basato su quel campo. Successivamente, tramite la funzione "insert_many", tutti i dati vengono inseriti nel database.
5. **Creazione di Indici Aggiuntivi:** Al fine di ottimizzare le operazioni di ricerca all'interno della collezione, viene creato un ulteriore indice basato sui nomi dei Code Smell.

Proiezione dei campi

Per semplificare l'operazione di proiezione, che può essere effettuata nelle varie funzioni di query, e per prevenire problemi associati a campi non validi, è stata implementata la funzione di utility "build_field_projection". Questa funzione, illustrata nella Figura 3.8, accetta come parametro una lista di campi potenziali. Successivamente, genera un dizionario che include soltanto i campi ritenuti validi, cioè quelli presenti nella lista "VALID_FIELDS".



```

1 VALID_FIELDS = ["id", "name", "description", "problems", "solution",
2                 "bad_example", "good_example", "prompt_example", "type"]
3
4
5 # Create a MongoDB projection dictionary based on the specified fields.
6 def build_field_projection(fields):
7     projection = {}
8
9     # Check if provided fields are valid
10    if fields:
11        for field in fields:
12            if field in VALID_FIELDS:
13                projection[field] = 1 # Include this field in the projection
14    return projection

```

Figura 3.8: Funzione "build_field_projection"

Encoding caratteri di escape

Per assicurare una corretta visualizzazione degli esempi di codice restituiti dalle query, che possono includere sequenze di escape, è stata sviluppata la funzione di utility "process_example_fields", illustrata nella Figura 3.9. Questa funzione svolge un ruolo cruciale nel decodificare e preparare i campi contenenti codice per una presentazione ottimale nell'interfaccia utente. Di seguito sono elencate le operazioni eseguite dalla funzione:

1. **Ricerca dei Campi Contenenti Codice:** La funzione verifica la presenza di campi come "good_example", "bad_example", e "prompt_example" nell'oggetto "code_smell".
2. **Decodifica delle Sequenze di Escape:** Per ogni campo di codice identificato, la funzione converte le stringhe codificate in stringhe decodificate, interpretando correttamente le sequenze di escape Unicode. Questo processo garantisce che gli esempi di codice siano rappresentati in maniera leggibile e precisa.
3. **Output:** Dopo aver elaborato i campi necessari, la funzione restituisce l'oggetto "code_smell" aggiornato.



```
1 # Decode escape sequences in code examples
2 def process_example_fields(code_smell):
3     for key in ['good_example', 'bad_example', 'prompt_example']:
4         if key in code_smell:
5             code_smell[key] = code_smell[key].encode().decode('unicode_escape')
6     return code_smell
```

Figura 3.9: Funzione "process_example_fields".

Query: get_all_code_smells

La funzione "get_all_code_smells", illustrata nella Figura 3.10, è essenziale per il recupero dell'elenco completo dei Code Smells dal database, permettendo l'inclusione selettiva di campi tramite il parametro "fields". Di seguito vengono descritte le fasi chiave del suo funzionamento:

1. **Connessione al Database:** Inizialmente, la funzione stabilisce una connessione con il database e accede alla collezione "codesmells".
2. **Costruzione del Dizionario di Proiezione:** Utilizza la funzione "build_field_projection" per creare un dizionario di proiezione che filtra gli attributi dei Code Smells in base ai campi specificati.
3. **Esecuzione della Query:** La query è eseguita impiegando il metodo "find" sulla collezione, il quale recupera tutti i Code Smells all'interno del Database.
4. **Elaborazione dei risultati:** Prima di restituire i risultati, questi vengono processati dalla funzione "process_example_fields" per decodificare eventuali esempi di codice inclusi, garantendo che siano visualizzati correttamente all'utente.

```
1 # Fetch all code smells from the database
2 def get_all_code_smells(fields=None):
3     mongo_collection, mongo_client = get_mongo_collection()
4     projection = build_field_projection(fields)
5     try:
6         cursor = mongo_collection.find({}, projection).sort("id", pymongo.ASCENDING)
7         result = [process_example_fields(doc) for doc in cursor]
8     except Exception as e:
9         logging.error(f"Error fetching all code smells: {e}")
10        result = None
11    finally:
12        mongo_client.close()
13    return result
```

Figura 3.10: Funzione "get_all_code_smells".

Query: get_code_smell_by_id

La funzione "get_code_smell_by_id", illustrata nella Figura 3.11, è stata sviluppata per recuperare un specifico Code Smell dal database utilizzando il suo identificativo (ID). Questa funzione segue un processo simile a quello descritto per la funzione "get_all_code_smells", con la principale differenza nella modalità di esecuzione della query. In questo caso viene impiegato il metodo "find_one", il quale si focalizza sulla ricerca di un singolo documento che corrisponda all'ID specificato.

```

1 # Fetch a code smell by its ID from the database
2 def get_code_smell_by_id(code_smell_id, fields=None):
3     mongo_collection, mongo_client = get_mongo_collection()
4     projection = build_field_projection(fields)
5     try:
6         result = mongo_collection.find_one({"id": int(code_smell_id)}, projection)
7         if result:
8             result = process_example_fields(result)
9     except Exception as e:
10         logging.error(f"Error fetching code smell details: {e}")
11         result = None
12     finally:
13         mongo_client.close()
14     return result

```

Figura 3.11: Funzione "get_code_smell_by_id".

Query: get_code_smell_by_name

La funzione "get_code_smell_by_name", illustrata nella Figura 3.12, è stata progettata per il recupero di Code Smells dal database basandosi sul loro nome. Anche questa funzione segue un processo simile a quello delle funzioni descritte in precedenza, ma in questo caso il recupero del Code Smell avviene attraverso la funzione di utility "code_smell_name_matcher (vedi Sez. 3.3.5) che sfrutta tecniche di fuzzy matching per identificare il nome più simile al termine di ricerca e il relativo punteggio di somiglianza. Se il punteggio di somiglianza supera il 70%, la query viene eseguita utilizzando il nome più simile trovato. Questo processo riduce gli errori dovuti a discrepanze di digitazione e migliora l'affidabilità dei risultati.

```

1 # Fetch a code smell by its name from the database
2 def get_code_smell_by_name(code_smell_name, fields=None):
3
4     from utils.string_matcher import code_smell_name_matcher
5     mongo_collection, mongo_client = get_mongo_collection()
6     projection = build_field_projection(fields)
7     try:
8         # Find Best Match
9         best_match_name, similarity_score = code_smell_name_matcher(code_smell_name)
10        query = {"name": best_match_name} if similarity_score >= 70 else {"name": code_smell_name}
11        result = mongo_collection.find_one(query, projection)
12        if result:
13            result = process_example_fields(result)
14    except Exception as e:
15        logging.error(f"Error fetching code smell details: {e}")
16        result = None
17    finally:
18        mongo_client.close()
19    return result
20

```

Figura 3.12: Funzione "get_code_smell_by_name".

3.3.4 Domain

Il Domain [40] è un concetto fondamentale in Rasa, che definisce l'universo in cui opera il chatbot. Il Domain viene specificato nel file "domain.yml", utilizzando il Markup Language YAML per definire tutti gli elementi del dominio, tra cui intents, entities, responses, actions e slots.

Nella sezione seguente verrà offerta una panoramica generale degli elementi del dominio di CAMILLE, illustrando brevemente come ciascuno di essi contribuisca al funzionamento complessivo del Chatbot.

Intent

Come già spiegato in precedenza, gli intent rappresentano un mapping tra le richieste dell'utente e l'azione che il chatbot dovrà eseguire. Nel file "domain.yml" bisogna specificare i nomi di tutti gli intent utilizzati dal Chatbot. Gli intent principali utilizzati da CAMILLE sono elencati nella Tabella 3.2. Si noti che gli intent di importanza più marginale, come quelli per i saluti o i ringraziamenti, sono stati omessi dalla tabella per mantenere il focus sulle funzionalità chiave di CAMILLE.

Tabella 3.2: Intents del Chatbot.

Intent	Descrizione
ask_help	L'utente chiede aiuto sulle funzionalità del Chatbot.
ask_code_smells_list	L'utente richiede di visualizzare la lista completa dei Code Smells.
ask_code_smell_details	L'utente richiede informazioni su uno specifico Code Smell.
ask_code_smell_example	L'utente richiede un esempio di uno specifico Code Smell.
ask_project_analysis	L'utente richiede di analizzare una repository GitHub.
ask_analysis_report	L'utente richiede il report dell'ultima analisi.
ask_fix_suggestion	L'utente richiede suggerimenti di refactoring per rimuovere un Code Smell rilevato all'interno del progetto.

Actions

Le action rappresentano le operazioni eseguibili dal Chatbot in risposta alle richieste degli utenti. Come per gli intents, i nomi delle actions utilizzate dal Chatbot devono essere esplicitati all'interno del file "domain.yml". La Tabella 3.3 presenta l'elenco delle Custom Actions utilizzate da CAMILLE, le quali verranno trattate nel dettaglio nella sezione 3.3.7.

Tabella 3.3: Custom Actions del Chatbot.

Custom Action	Descrizione
action_default_fallback	Azione di default eseguita in risposta agli intent di fallback.
action_get_code_smells_list	Azione con cui il chatbot fornisce la lista dei Code Smells all'utente.
action_provide_code_smell_details	Azione con cui il Chatbot fornisce informazioni riguardo uno specifico Code Smell.
action_provide_code_smell_example	Azione con cui il Chatbot fornisce degli esempi di un Code Smell specifico.
action_project_analysis	Azione con cui il Chatbot avvia l'analisi di una repository GitHub.
action_analysis_report	Azione con cui il Chatbot genera e fornisce il report dell'analisi.
action_suggest_fix	Azione con cui il Chatbot fornisce dei suggerimenti per rimuovere un Code Smell rilevato all'interno del codice.

Entities

Anche le entities, ovvero informazioni chiave estratte dalla richiesta e utili per soddisfarla, devono essere definite all'interno del file "domain.yml". Tutte le entities che possono essere estratte dai messaggi dell'utente nel dominio del Chatbot CAMILLE sono descritte nella Tabella 3.4. Tuttavia, il loro utilizzo specifico verrà mostrato di volta in volta nelle sezioni successive.

Tabella 3.4: Entities del Chatbot.

Entity	Descrizione
code_smell_id	Id di un Code Smell estratto da una richiesta di informazioni/esempi riguardo uno specifico Code Smell.
code_smell_name	Nome di un Code Smell estratto da una richiesta di informazioni/esempi riguardo uno specifico Code Smell.
project_repository	URL di una repository GitHub estratto da una richiesta di analisi di una repository.
issue_index	Indice di un problema all'interno del report, estratto da una richiesta di suggerimento di refactoring.

Slots

Per gli slots, diversamente dagli altri elementi del dominio, è necessario specificare ulteriori informazioni importanti oltre al nome. Queste informazioni includono il tipo di valore contenuto nello slot e il mapping, ovvero la modalità con cui viene riempito lo Slot. Nella Figura 3.13 è possibile osservare la definizione dei due Slot principali utilizzati dal Chatbot CAMILLE:



```

1 slots:
2   report_filename:
3     type: text
4     mappings:
5       - type: custom
6
7   repo_url:
8     type: text
9     mappings:
10      - type: from_entity
11        entity: project_repository
12        intent: ask_project_analysis

```

Figura 3.13: Definizione degli Slots.

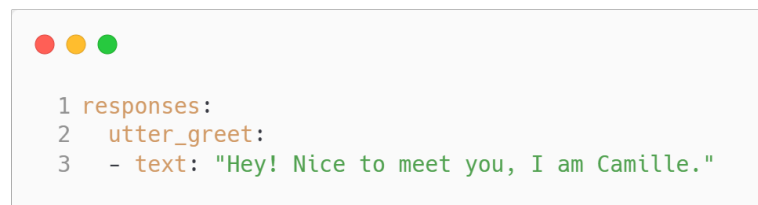
- **report_filename:** Mantiene il percorso in cui è stato memorizzato il report dell'ultima analisi richiesta dall'utente. Questo Slot è di tipo "text", quindi conterrà informazioni testuali. Il mapping è "custom", il che significa che verrà gestito esplicitamente all'interno del codice;

- **repo_url**: Mantiene l'URL della repository per la quale è stata richiesta l'ultima analisi. Grazie a questo Slot, l'utente può richiedere più analisi consecutive sullo stesso progetto senza dover specificare ogni volta l'URL. Anche questo slot è di tipo "text", mentre il mapping avviene ogni volta che viene estratta l'entity "project_repository" dall'intent "ask_project_analysis".

Oltre alle informazioni utilizzate, Rasa permette anche l'aggiunta di altre informazioni opzionali che possono aiutare a gestire meglio il mapping in casi particolari.

Responses

Un altro elemento che bisogna definire all'interno del file "domain.yml" sono le responses. Come è possibile osservare dall'esempio in Figura 3.14, che mostra la response utilizzata dal Chatbot in risposta ai saluti dell'utente, per ogni response bisogna specificare il nome e il testo. Tuttavia, le responses non sono di fondamentale importanza alla comprensione del progetto poichè esse non riguardano aspetti strettamente legati alle funzionalità core del Chatbot. Pertanto, non saranno trattate nel dettaglio.



```
1 responses:
2   utter_greet:
3     - text: "Hey! Nice to meet you, I am Camille."
```

Figura 3.14: Response per salutare l'utente.

3.3.5 Natural Language Understanding

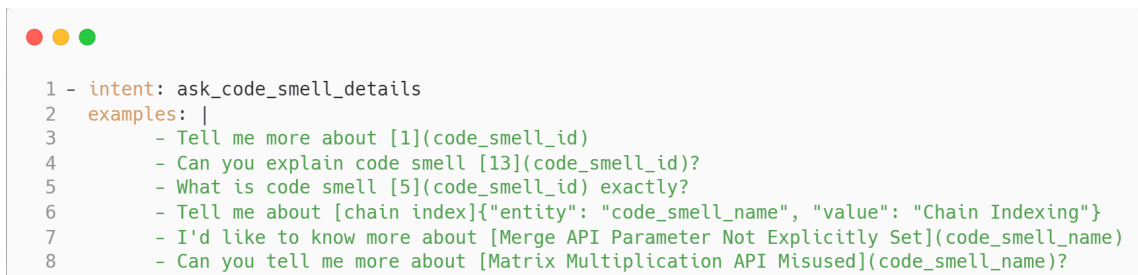
In questa sezione verrà esplorato tutto ciò che riguarda la componente di Natural Language Understanding (NLU) del chatbot CAMILLE. In particolare, verrà descritta la definizione dei dati di training e i meccanismi adottati per facilitare l'estrazione delle entità. Inoltre, verranno analizzati i diversi strumenti e tecniche impiegati, come le espressioni regolari (Regex) e le Lookup Tables, per migliorare la precisione del modello di NLU. Infine, analizzeremo nel dettaglio la pipeline di NLU utilizzata dal

chatbot, illustrando i vari componenti e il loro ruolo nel processo di comprensione del linguaggio naturale.

Dati di Training

I dati di training sono fondamentali per il successo della componente di NLU del Chatbot. Essi consentono al modello di apprendere come riconoscere e interpretare correttamente gli input degli utenti, migliorando così la capacità del chatbot di rispondere in modo accurato e pertinente. Come evidenziato nella documentazione ufficiale di Rasa [41], l'ideale sarebbe avere a disposizione dati reali provenienti da interazioni con gli utenti. Tuttavia, non avendo a disposizione dati del genere per questo progetto, sono stati definiti degli esempi ad hoc per simulare le possibili interazioni.

Nel file "nlu.yml" devono essere specificati diversi elementi cruciali per l'addestramento del modello di NLU. In particolare, per ogni intent è necessario fornire una serie di esempi di frasi che potrebbero essere utilizzate dagli utenti per esprimere quello specifico intent. Inoltre, se all'interno di un Intent devono essere estratte anche delle Entity, è opportuno annotarle negli esempi per aiutare il modello a riconoscerle nelle conversazioni reali.



```
1 - intent: ask_code_smell_details
2   examples: |
3     - Tell me more about [1](code_smell_id)
4     - Can you explain code smell [13](code_smell_id)?
5     - What is code smell [5](code_smell_id) exactly?
6     - Tell me about [chain index]{"entity": "code_smell_name", "value": "Chain Indexing"}
7     - I'd like to know more about [Merge API Parameter Not Explicitly Set](code_smell_name)
8     - Can you tell me more about [Matrix Multiplication API Misused](code_smell_name)?
```

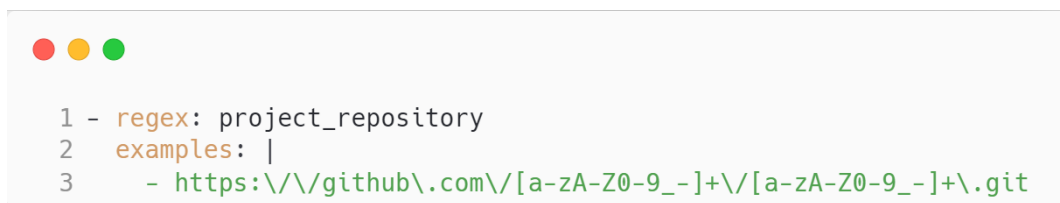
Figura 3.15: Esempio di dati di training per il modello di NLU.

Nella Figura 3.15 è possibile osservare una parte dei dati di training per l'Intent "ask_code_smell_details". Possiamo notare che sono stati forniti sia esempi in cui è annotata l'entity "code_smell_id", sia esempi in cui è annotata l'entity "code_smell_name". Inoltre, nell'esempio alla riga 6, possiamo notare che per l'entity è stato definito un sinonimo, mappando l'entity estratta con valore "chain index" automaticamente al valore "Chain Indexing". I sinonimi possono essere molto utili

in casi in cui gli utenti scrivono spesso in modo errato un'entity. Tuttavia, anche in questo caso, sarebbe opportuno avere a disposizione dati reali per migliorare ulteriormente la precisione e l'efficacia del modello.

Espressioni Regolari

All'interno del file "nlu.yml", Rasa permette di definire delle espressioni regolari (Regex) [42] per migliorare la classificazione degli intent e l'estrazione delle entity. Le Regex vengono impiegate principalmente per riconoscere ed estrarre entity che seguono formati ben definiti, migliorando la precisione del modello di NLU e garantendo che le entity vengano identificate correttamente.



```
1 - regex: project_repository
2   examples: |
3     - https://github.com/[a-zA-Z0-9_-]+\.[a-zA-Z0-9_-]+\.
```

Figura 3.16: Regex per l'entity 'project_repository'.

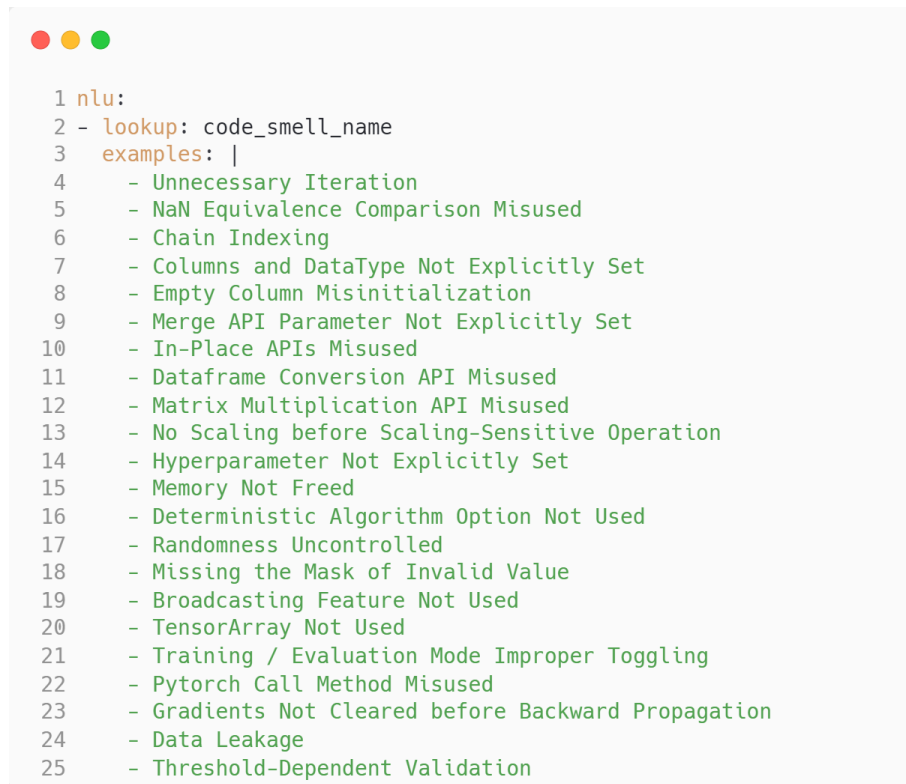
L'esempio in Figura 3.16 mostra la Regex utilizzata per l'entity "project_repository". Utilizzando questa Regex, il modello di NLU può identificare automaticamente gli URL delle repository GitHub all'interno delle frasi degli utenti, migliorando l'accuratezza dell'estrazione e garantendo che i dati rilevati vengano correttamente riconosciuti.

Lookup Tables

Le Lookup Tables [42] sono un altro strumento messo a disposizione da Rasa per migliorare la precisione del processo di entity extraction. Esse consistono in una lista di parole corrispondenti ai valori che può assumere una specifica entità, le quali vanno a generare una sorta di espressione regolare case-insensitive. Risultano particolarmente utili per aiutare il modello ad estrarre entity che hanno un insieme ben definito di valori.

Nella Figura 3.17 viene mostrata la Lookup Table per l'entity "code_smell_name". In questo caso, la Lookup Table risulta particolarmente utile poichè i nomi dei Code

Smells non sono parole d'uso comune e non esiste un algoritmo di entity extraction specificatamente addestrato per riconoscerli.



```

1 nlu:
2 - lookup: code_smell_name
3   examples: |
4     - Unnecessary Iteration
5     - NaN Equivalence Comparison Misused
6     - Chain Indexing
7     - Columns and DataType Not Explicitly Set
8     - Empty Column Misinitialization
9     - Merge API Parameter Not Explicitly Set
10    - In-Place APIs Misused
11    - Dataframe Conversion API Misused
12    - Matrix Multiplication API Misused
13    - No Scaling before Scaling-Sensitive Operation
14    - Hyperparameter Not Explicitly Set
15    - Memory Not Freed
16    - Deterministic Algorithm Option Not Used
17    - Randomness Uncontrolled
18    - Missing the Mask of Invalid Value
19    - Broadcasting Feature Not Used
20    - TensorArray Not Used
21    - Training / Evaluation Mode Improper Toggling
22    - Pytorch Call Method Misused
23    - Gradients Not Cleared before Backward Propagation
24    - Data Leakage
25    - Threshold-Dependent Validation

```

Figura 3.17: Lookup Table per l'entity 'code_smell_name'.

Fuzzy Matching

Come spiegato in precedenza, il processo di estrazione dell'entity "code_smell_name" può risultare problematico. Per migliorare la robustezza del Chatbot e ridurre gli errori nelle query che utilizzano i nomi dei Code Smells, è stata creata una funzione di utility chiamata "code_smell_name_matcher". Come si può notare dalla Figura 3.18, questa funzione esegue il matching tra il nome di un Code Smell fornito in input e l'elenco dei nomi dei Code Smells presenti nel database. Il Matching viene effettuato attraverso la libreria "fuzzywuzzy", la quale calcola il risultato migliore utilizzando la Distanza di Levenshtein.

Il processo operativo della funzione "code_smell_name_matcher" si sviluppa attraverso diverse fasi:

1. **Pulizia dell'Input:** Inizialmente, rimuove gli spazi bianchi non necessari all'inizio e alla fine del nome inserito.

2. **Recupero dei Nomi esistenti:** Accede al database per ottenere l'elenco completo dei nomi dei Code Smells esistenti, formando una lista per il successivo matching.
3. **Applicazione del Matching Fuzzy:** Utilizza la funzione "extractOne" di "fuzzywuzzy" per identificare il matching migliore. Il risultato finale è una coppia di valori che include il nome del Code Smell più simile trovato e il relativo punteggio di similarità.



```

1 from fuzzywuzzy import process
2 from db.query import get_all_code_smells
3 import logging
4
5 logging.basicConfig(level=logging.INFO)
6
7
8 def code_smell_name_matcher(code_smell_name):
9     try:
10         # Strip code smell name
11         code_smell_name = code_smell_name.strip()
12
13         # Select all the code smells name from DB
14         results = get_all_code_smells(["name"])
15         choices = [x['name'] for x in results]
16
17         # Return matching results
18         match = process.extractOne(code_smell_name, choices)
19         if match:
20             logging.info(f"Best match found: {match}")
21         else:
22             logging.info("No match found.")
23         return match
24     except Exception as e:
25         logging.error(f"Error during String Matching: {e}")
26         return None

```

Figura 3.18: Funzione 'code_smell_name_matcher'.

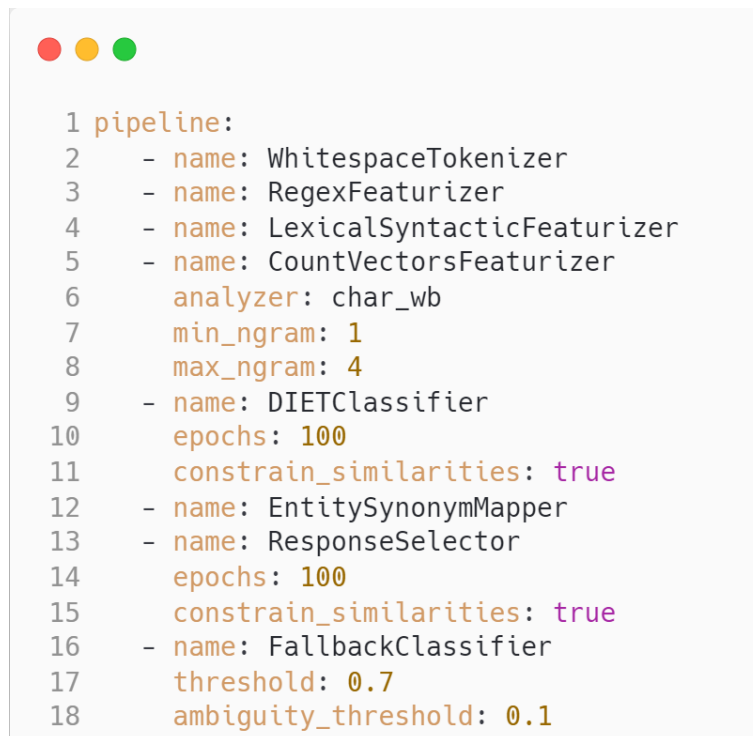
Il punteggio di similarità restituito dalla funzione risulta particolarmente utile per gestire scenari in cui il matching non risulta essere sufficientemente affidabile.

NLU Pipeline

La pipeline di NLU [43] è una sequenza di componenti che analizzano e interpretano gli input degli utenti, permettendo al chatbot di comprendere e rispondere

in modo appropriato. Questa pipeline va definita nel file "config.yml", dove si specificano i diversi elementi che compongono il processo di analisi del linguaggio naturale.

Rasa fornisce anche delle configurazioni predefinite (config recipe) per la pipeline di NLU, valide nella maggior parte degli utilizzi standard. Tuttavia, è buona pratica definire esplicitamente la pipeline per avere un controllo completo sui componenti utilizzati ed evitare comportamenti inattesi causati da modifiche alle recipe.



```
1 pipeline:
2   - name: WhitespaceTokenizer
3   - name: RegexFeaturizer
4   - name: LexicalSyntacticFeaturizer
5   - name: CountVectorsFeaturizer
6     analyzer: char_wb
7     min_ngram: 1
8     max_ngram: 4
9   - name: DIETClassifier
10     epochs: 100
11     constrain_similarities: true
12   - name: EntitySynonymMapper
13   - name: ResponseSelector
14     epochs: 100
15     constrain_similarities: true
16   - name: FallbackClassifier
17     threshold: 0.7
18     ambiguity_threshold: 0.1
```

Figura 3.19: Pipeline di NLU.

Di seguito vengono descritti nel dettaglio i vari elementi della pipeline utilizzata da CAMILLE, così come viene mostrato nella Figura 3.19:

- **WhitespaceTokenizer:** Divide il testo in token basandosi sugli spazi bianchi, facilitando l'analisi successiva del testo;
- **RegexFeaturizer:** Utilizza espressioni regolari per creare feature aggiuntive basate su schemi specifici nel testo. È necessaria per poter utilizzare le Regex (3.3.5) e le Lookup Tables (3.3.5) per il training del modello;
- **LexicalSyntacticFeaturizer:** Genera feature lessicali e sintattiche dalle frasi degli utenti, aiutando il processo di entity extraction;

- **CountVectorsFeaturizer**: Utilizza la tecnica del "bag of words" per rappresentare il testo. Conta il numero di volte che ogni parola appare nel testo e crea un vettore che rappresenta queste frequenze. Nella pipeline, l'opzione "char_wb" permette di configurarlo per analizzare il testo a livello di carattere. Le altre due opzioni, "min_ngram: 1" e "max_ngram: 4", specificano che devono essere considerate tutte le possibili sequenze di caratteri (n-gram) di lunghezza minima 1 e lunghezza massima 4;
- **DIETClassifier**: Il Dual Intent Entity Transformer Classifier utilizza un modello di rete neurale per analizzare il testo, migliorando la precisione dell'entity extraction e della classificazione degli intent. l'opzione "constrain_similarities: true" permette di ridurre le probabilità che il modello confonda intent simili, rendendo quindi il modello stesso più robusto;
- **EntitySynonymMapper**: Assicura che diverse varianti di una stessa entity vengano trattate come equivalenti, mappandole ad una forma standard e migliorando la consistenza dei dati;
- **ResponseSelector**: Seleziona la risposta più appropriata basandosi sul contesto della conversazione;
- **FallbackClassifier**: Gestisce i casi in cui l'intent non può essere determinato con sufficiente sicurezza (fallback). La soglia di confidenza minima per considerare valido un intent è impostata a 0.7, mentre l'opzione "ambiguity_threshold" è stata impostata a 0.1. Questo significa che se la differenza di confidenza tra i due intent principali è inferiore al 10%, il modello attiverà un fallback per evitare ambiguità.

3.3.6 Dialogue Management

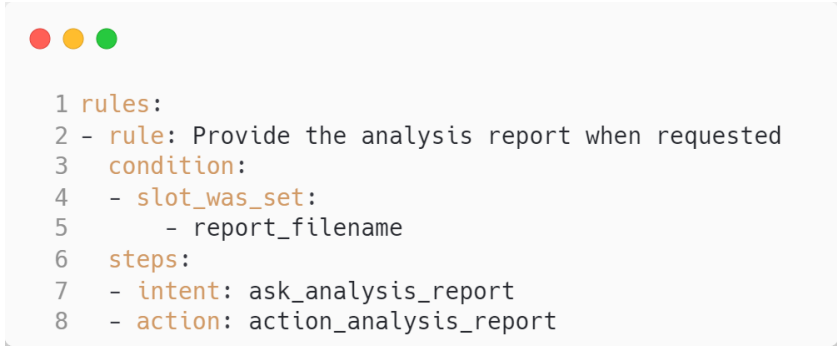
In questa sezione verrà esplorato tutto ciò che riguarda la componente di Dialogue Management del chatbot CAMILLE. In particolare, verranno descritte le rules [44] e le stories [45], elementi chiave per il training del modello di gestione del dialogo. Verranno illustrate le modalità di definizione di questi dati di training, accompagnate da esempi pratici. Inoltre, verranno analizzate le policies configurate nel chatbot, che

combinano le informazioni apprese dalle rules e dalle stories con le predizioni del modello per determinare l'azione più appropriata da eseguire.

Rules

Le Rules [44], definite nel file "rules.yml", vengono utilizzate per il training del modello di dialogue management. Le Rules descrivono brevi pezzi di conversazione che seguono sempre lo stesso pattern e sono quindi utilizzate per gestire scenari ben definiti e prevedibili, garantendo che il chatbot risponda in modo coerente e appropriato in situazioni specifiche.

Per definire una rule, è necessario associarle un nome e una serie di passi in termini di intent rilevato e action eseguita dal chatbot. È possibile anche definire rules dette "rules with conditions", che permettono di eseguire azioni solo se determinate condizioni sono soddisfatte. Queste condizioni possono essere basate sugli slot o su altre informazioni rilevanti nella conversazione.

A screenshot of a code editor window with a light gray background. At the top left, there are three colored circles: red, yellow, and green. The code is written in a monospaced font with syntax highlighting. It defines a rule named 'rules' with a condition and steps.

```
1 rules:
2 - rule: Provide the analysis report when requested
3   condition:
4     - slot_was_set:
5       - report_filename
6   steps:
7     - intent: ask_analysis_report
8     - action: action_analysis_report
```

Figura 3.20: Rule per la richiesta del report.

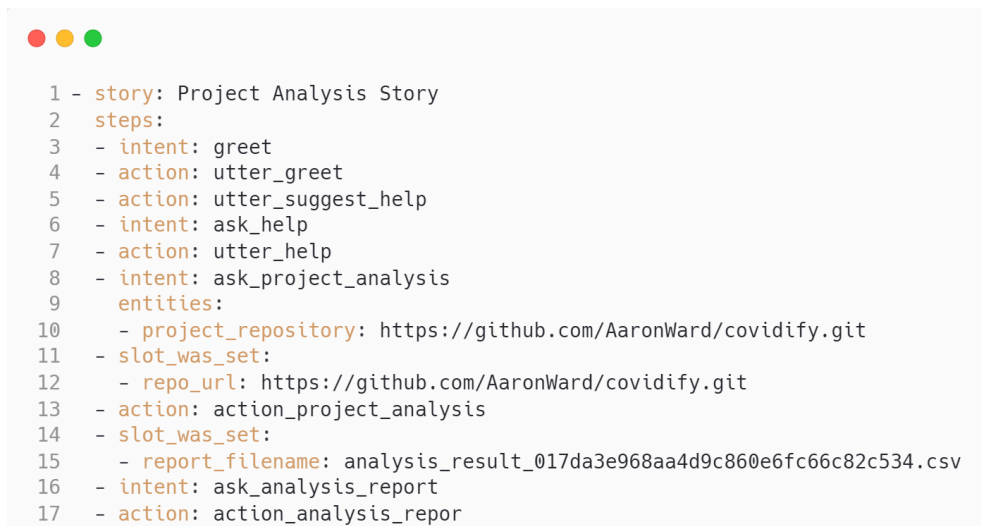
Nella Figura 3.20 è possibile osservare la rule utilizzata per gestire le richieste del report di analisi della repository GitHub. Questa rule si attiva quando l'intent "ask_analysis_report" viene riconosciuto, ovvero quando l'utente chiede di visualizzare il report di analisi. La rule include una condizione che verifica se lo slot "report_filename" contiene dei dati, ovvero se esiste un file di report disponibile. Quindi se l'intent "ask_analysis_report" viene riconosciuto e la condizione sullo slot "report_filename" è soddisfatta, il Chatbot eseguirà l'azione "action_analysis_report".

Stories

Le stories [45], definite nel file "stories.yml" sono un altro componente essenziale per il training del modello di dialogue management. Esse rappresentano sequenze di interazioni tra l'utente e il Chatbot, che il modello utilizza per apprendere come gestire conversazioni più complesse e variabili.

Mentre le rules sono utilizzate per gestire scenari ben definiti e prevedibili, le stories forniscono una maggiore flessibilità e capacità di adattamento a diverse situazioni di conversazione. Per ottenere il massimo dal dialogue manager, è consigliabile utilizzare una combinazione di rules e stories.

Per definire una story, bisogna associarle un nome e una sequenza di intents e actions che rappresentano un path di conversazione. All'interno della sequenza di interazioni, è necessario specificare esplicitamente anche eventi, come ad esempio il riempimento di uno slot o l'attivazione/disattivazione di un form. Questo perché durante la fase di training, Rasa non esegue effettivamente l'action server e non può sapere quali eventi vengono generati dalle custom actions utilizzate dalle stories.



```

1 - story: Project Analysis Story
2   steps:
3     - intent: greet
4     - action: utter_greet
5     - action: utter_suggest_help
6     - intent: ask_help
7     - action: utter_help
8     - intent: ask_project_analysis
9     entities:
10    - project_repository: https://github.com/AaronWard/covidify.git
11    - slot_was_set:
12      - repo_url: https://github.com/AaronWard/covidify.git
13    - action: action_project_analysis
14    - slot_was_set:
15      - report_filename: analysis_result_017da3e968aa4d9c860e6fc66c82c534.csv
16    - intent: ask_analysis_report
17    - action: action_analysis_repor
  
```

Figura 3.21: Story 'Project analysis story'.

Nella Figura 3.21 è possibile osservare un esempio di story. La story, chiamata "Project analysis story", inizia con l'utente che saluta il chatbot ("intent: greet"). Il Chatbot risponde con un saluto ("action: utter_greet") e si propone di aiutare l'utente ("action: utter_suggest_help"). L'utente chiede aiuto ("intent: ask_help"), e il Chatbot risponde fornendo le sue funzionalità principali ("action: utter_help").

Successivamente, l'utente richiede di avviare l'analisi di una repository GitHub ("intent: ask_project_analysis"). L'entity "project_repository" viene estratta dalla richiesta e mappata nello slot "repo_url". Il Chatbot quindi avvia l'analisi della repository ("action: action_project_analysis") e imposta il valore dello slot "report_filename". Infine, l'utente chiede di vedere il report ("intent: ask_analysis_report") e il chatbot glielo mostra ("action: action_analysis_report").

Per sequenze di interazioni più complesse, la scrittura delle stories può risultare altrettanto complessa. Rasa fornisce uno strumento di Interactive Learning che facilita la creazione delle stories. Questo strumento, utilizzato anche per la scrittura di alcune delle stories di CAMILLE, come ad esempio quella mostrata nel Figura 3.21, permette di addestrare il Chatbot interagendo direttamente con esso. Mentre l'utente simula conversazioni reali, Rasa registra automaticamente la sequenza di intents e actions, creando nuove stories basate su queste interazioni.

Policies

Le policies [46] sono componenti del dialogue management di Rasa che determinano quale azione il Chatbot deve eseguire in risposta agli input degli utenti. Esse combinano le informazioni apprese dalle rules e dalle stories con le predizioni del modello per scegliere la risposta più appropriata. Le policies sono definite nel file "config.yml".



```
1 policies:
2   - name: MemoizationPolicy
3   - name: RulePolicy
4     core_fallback_threshold: 0.3
5     core_fallback_action_name: "action_default_fallback"
6     enable_fallback_prediction: True
7   - name: UnexpectTEDIntentPolicy
8     max_history: 5
9     epochs: 100
10  - name: TEDPolicy
11    max_history: 5
12    epochs: 100
13    constrain_similarities: true
```

Figura 3.22: Policies del Chatbot.

Di seguito vengono descritti i vari elementi della configurazione delle policies utilizzate da CAMILLE, così come viene mostrato nella Figura 3.22:

- **MemoizationPolicy:** La MemoizationPolicy memorizza le conversazioni passate e ripete esattamente le stesse risposte se la sequenza di intents e actions si ripresenta, permettendo al chatbot di rispondere rapidamente e con precisione;
- **RulePolicy:** La RulePolicy applica le rules definite nel file "rules.yml" per gestire scenari ben definiti e prevedibili. È stata configurata per attivare l'azione di fallback "action_default_fallback" se la confidenza per la predizione corrente è inferiore al 30%;
- **UnexpectEDIntentPolicy:** La UnexpectEDIntentPolicy permette di gestire intents inaspettati che non sono stati visti durante la fase di training, aiutando il Chatbot a rispondere in modo appropriato anche in queste situazioni;
- **TEDPolicy:** La TEDPolicy (Transformer Embedding Dialogue Policy) utilizza un modello di rete neurale basato su transformers per predire l'azione successiva del Chatbot. Questa policy altamente flessibile e potente, è in grado di gestire conversazioni complesse e variabili, migliorando la precisione e la coerenza delle risposte del Chatbot.

3.3.7 Custom Actions

Le Custom Actions [47] in Rasa sono funzioni definite dallo sviluppatore che permettono al chatbot di eseguire logiche di business specifiche o di interagire con servizi esterni. A differenza delle azioni standard, che sono limitate a risposte predefinite, le Custom Actions possono effettuare calcoli complessi, interrogare database, chiamare API esterne e molto altro.

Nella seguente sezione verrà introdotta la struttura di una Custom Action, spiegandone i componenti principali e come può essere creata. Successivamente, verranno analizzate nel dettaglio le Custom Actions più importanti di CAMILLE.

Creazione di Custom Actions

Per definire una Custom Action in Rasa, è necessario seguire i seguenti passaggi:

1. **Definire la classe della Custom Action:** Questa classe dovrà ereditare la classe "Action" fornita da "rasa_sdk";
2. **Implementare il metodo "name":** Questo metodo restituirà il nome della Custom Action;
3. **Implementare il metodo "run":** Questo metodo conterrà la logica che deve essere eseguita quando la Custom Action viene invocata.

il metodo "run" prende come parametri:

- **dispatcher:** è un CollectingDispatcher che può essere utilizzato per inviare le risposte all'utente tramite il metodo "utter_message";
- **tracker:** È un Tracker che permette di accedere al contesto della conversazione corrente. Può essere utilizzato per accedere alle entities tramite il metodo "get_latest_entity_values", agli slots tramite il metodo "get_slot" e ad altre informazioni importanti;
- **domain:** È un Dictionary che contiene tutte le informazioni sul dominio del Chatbot.

Lo sviluppatore non dovrà preoccuparsi di invocare manualmente la Custom Action o di passargli i parametri, poiché tutto verrà gestito automaticamente da Rasa sulla base di quanto specificato nelle rules e nelle stories.

Custom Action: action_get_code_smells_list

La Custom Action denominata "action_get_code_smells_list", rappresentata nella Figura 3.23, viene attivata in risposta alle richieste degli utenti che desiderano visualizzare un elenco degli ML-Specific Code Smells conosciuti allo stato dell'arte.

```

1 class ActionGetCodeSmellsList(Action):
2
3     def name(self) -> Text:
4         return "action_get_code_smells_list"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        try:
11            # Retrieve all the code smells
12            results = get_all_code_smells(["id", "name"])
13        except Exception as e:
14            logging.error(f"Failed to fetch code smells: {e}")
15            dispatcher.utter_message(text="Sorry, something went wrong "
16                                     "while fetching the code smells...")
17
18            return []
19
20        if results:
21            # Build the response
22            message_lines = ["Here is the list of code smells:"]
23            for code_smell in results:
24                message_lines.append(f"{code_smell['id']}: {code_smell['name']}")
25            message_lines.append("\nYou can ask me for more information or "
26                                "examples about each code smell in the list.")
27            message = "\n".join(message_lines)
28            dispatcher.utter_message(text=message)
29        else:
30            dispatcher.utter_message(text="No code smells found...")
31
32        return []

```

Figura 3.23: Custom Action 'action_get_code_smells_list'.

All'interno del metodo "run" vengono effettuate le seguenti operazioni:

1. **Esecuzione della Query:** Inizialmente, esegue una query al database tramite il metodo "get_all_code_smells" per recuperare gli ID e i nomi di tutti i Code Smells.
2. **Costruzione del Messaggio:** Se la query produce dei risultati, si procede alla costruzione di un messaggio contenente l'elenco dei Code Smells. Questo elenco viene formato in modo da presentare in maniera chiara e organizzata sia l'ID che il nome di ciascun Code Smell.
3. **Invio del Messaggio:** Il messaggio costruito viene successivamente inviato all'utente attraverso il Dispatcher.

Custom Action: action_provide_code_smell_details

La Custom Action denominata "action_provide_code_smell_details", rappresentata nella Figura 3.24, viene attivata quando l'utente richiede ulteriori informazioni riguardanti uno specifico Code Smell.

```

1 class ActionProvideCodeSmellDetails(Action):
2
3     def name(self) -> Text:
4         return "action_provide_code_smell_details"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        # Get entities values
11        code_smell_id = next(tracker.get_latest_entity_values("code_smell_id"), None)
12        code_smell_name = next(tracker.get_latest_entity_values("code_smell_name"), None)
13        result = None
14
15        try:
16            if code_smell_id:
17                # Query by ID
18                result = get_code_smell_by_id(code_smell_id, ["description",
19                                                            "problems", "solution"])
20            elif code_smell_name:
21                # Query by Name
22                result = get_code_smell_by_name(code_smell_name, ["description",
23                                                                "problems", "solution"])
24            else:
25                dispatcher.utter_message(text="I'm sorry, I didn't understand what "
26                                           "code smell you are referring to...")
27                return []
28        except Exception as e:
29            logging.error(f"Failed to fetch code smell details: {e}")
30            dispatcher.utter_message(text="Sorry, something went wrong while fetching "
31                                       "the code smell details...")
32            return []
33
34        if result:
35            message = (
36                f"{result['description']}\n\n"
37                f"{result['problems']}\n\n"
38                f"{result['solution']}"
39            )
40            dispatcher.utter_message(text=message)
41        else:
42            dispatcher.utter_message(text="Sorry, I couldn't find any details about this code smell.")
43
44        return []

```

Figura 3.24: Custom Action 'action_provide_code_smell_details'.

Il metodo "run" si articola nei seguenti passaggi:

1. **Entity Extraction:** Utilizzando il Tracker, vengono estratti i valori delle entità "code_smell_id" e "code_smell_name" dall'ultima interazione dell'utente, per determinare la specifica richiesta.
2. **Ricerca del Code Smell:** A seconda delle informazioni fornite dall'utente, le informazioni sul Code Smell vengono ricercate tramite la funzione "get_code_smell_by_id" se l'utente ha specificato l'ID oppure tramite la funzione "get_code_smell_by_name" se l'utente ha fornito il nome.

3. **Invio del Messaggio:** Se la query restituisce dati validi, viene creato un messaggio dettagliato che include la descrizione del Code Smell, i problemi correlati e le raccomandazioni per mitigare o prevenire tali problemi. Questo messaggio viene poi trasmesso all'utente attraverso il Dispatcher.

È importante notare che se l'utente non specifica adeguatamente il Code Smell di interesse, viene fornita una risposta predefinita per sollecitare ulteriori dettagli.

Custom Action: `action_provide_code_smell_example`

La Custom Action denominata "action_provide_code_smell_example", mostrata nella Figura 3.25, viene attivata in risposta alla richiesta dell'utente di ottenere un esempio di un Code Smell specifico.

```

1 class ActionProvideCodeSmellExample(Action):
2
3     def name(self) -> Text:
4         return "action_provide_code_smell_example"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        # Get entities values
11        code_smell_id = next(tracker.get_latest_entity_values("code_smell_id"), None)
12        code_smell_name = next(tracker.get_latest_entity_values("code_smell_name"), None)
13        result = None
14
15        try:
16            if code_smell_id:
17                # Query by ID
18                result = get_code_smell_by_id(code_smell_id, ["name",
19                                                         "bad_example", "good_example"])
20            elif code_smell_name:
21                # Query by Name
22                result = get_code_smell_by_name(code_smell_name, ["name",
23                                                                "bad_example", "good_example"])
24            else:
25                dispatcher.utter_message(text="I'm sorry, I didn't understand what "
26                                           "code smell you were referring to.")
27            return []
28        except Exception as e:
29            logging.error(f"Failed to fetch code smell examples: {e}")
30            dispatcher.utter_message(text="Sorry, something went wrong while fetching "
31                                       "the code smell examples.")
32            return []
33
34        if result:
35            # Format examples
36            bad_example = self._preserve_empty_lines(result['bad_example'])
37            good_example = self._preserve_empty_lines(result['good_example'])
38            # Build response message
39            message = (
40                f"This is a code example with {result['name']}:\n{bad_example}\n\n"
41                f"And this is the correct one:\n{good_example}"
42            )
43            dispatcher.utter_message(text=message)
44        else:
45            dispatcher.utter_message(text="Sorry, I couldn't find any example about this code smell.")
46
47        return []

```

Figura 3.25: Custom Action 'action_provide_code_smell_example'.

Il metodo "run" di questa Custom Action compie operazioni simili a quelle di "action_provide_code_smell_details", precedentemente descritta. La distinzione principale risiede nelle informazioni recuperate dal database; in questo caso, si tratta degli esempi del Code Smell richiesto. Questi esempi sono accuratamente formattati tramite la funzione "_preserve_empty_lines" prima di essere inviati all'utente.

La funzione "_preserve_empty_lines" modifica le righe vuote all'interno del codice sostituendole con uno spazio. L'introduzione di questa funzione è stata essenziale per evitare che Rasa interpretasse erroneamente le righe vuote come separatori di messaggi, consentendo così di bypassare il problema e allo stesso tempo di preservare l'integrità della presentazione degli esempi di codice.

Custom Action: action_project_analysis

La Custom Action denominata "action_project_analysis", illustrata nella Figura 3.26, viene attivata quando l'utente richiede di avviare l'analisi del codice di una repository GitHub.

```

1 class ActionProjectAnalysis(Action):
2
3     def name(self) -> Text:
4         return "action_project_analysis"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        # Get slot value
11        github_repo_url = tracker.get_slot("repo_url")
12
13        if not github_repo_url:
14            dispatcher.utter_message(text="You haven't provided a GitHub repository URL.")
15            return []
16
17        # Request Preparation
18        url = os.getenv("PROJECT_ANALYZER_BASE_URL") + "/analyze-repository"
19        payload = {"repository_url": github_repo_url}
20
21        try:
22            # Send the request to the project analyzer service
23            response = requests.post(url, json=payload)
24            response.raise_for_status() # Raise an exception for HTTP errors
25            analysis_result = response.json()
26            dispatcher.utter_message(text=f"Analysis Completed: {len(analysis_result)} code smells found!")
27
28            # Save the analysis report
29            report_filename = self._save_analysis_report(tracker.sender_id, analysis_result)
30            dispatcher.utter_message(text="The report is ready for download.")
31            return [SlotSet("report_filename", report_filename)]
32        except requests.RequestException as e:
33            logging.error("Request failed: %s", e)
34            dispatcher.utter_message(
35                text="Sorry, there was an error while analyzing the repository. Please try again later.")
36        except Exception as e:
37            logging.error("Error during project analysis: %s", e)
38            dispatcher.utter_message(text="Sorry, something went wrong during the analysis.")
39
40        return []

```

Figura 3.26: Custom Action 'action_project_analysis'.

Nel metodo "run" di "action_project_analysis" vengono effettuate le seguenti operazioni:

1. **Recupero dell'URL:** Il Tracker recupera il valore dello slot "repo_url", mappato automaticamente da Rasa dall'entity "project_repository" rilevata nelle interazioni dell'utente.
2. **Richiesta di Analisi:** Con l'URL del repository, viene effettuata una richiesta HTTP POST all'endpoint "/analyze-repository" del tool di analisi. Se la richiesta viene processata con successo, l'utente riceve un messaggio che riporta il numero di Code Smells rilevati.
3. **Salvataggio del Report:** Il report dell'analisi, in formato csv, viene salvato utilizzando la funzione "_save_analysis_report", e il nome del report viene mappato all'interno del relativo slot.

La funzione "_save_analysis_report" gioca un ruolo fondamentale in questo contesto. Infatti, essa genera il nome del file di report usando l'ID utente di Rasa, garantendo in questo modo l'unicità del file per ogni utente ed eliminando il rischio di conflitti.

Custom Action: action_analysis_report

La Custom Action denominata "action_analysis_report", illustrata nella Figura 3.27, viene attivata in risposta alla richiesta dell'utente di visualizzare il report dell'ultima analisi effettuata.

Il metodo "run" di questa Custom Action procede nel seguente modo:

1. **Recupero del Report:** Il nome del file di report viene recuperato dallo slot "report_filename", e utilizzato per costruire il percorso completo al file.
2. **Elaborazione del Report:** Il file CSV, che contiene i dettagli del report, viene letto e i dati vengono trasformati in formato HTML, attraverso la funzione "generate_html_report".

3. **Generazione del Report HTML:** La funzione "generate_html_report", utilizza il motore di templating Jinja2 per definire un template HTML. I dati del report vengono poi iniettati nel template tramite la funzione "render" di Jinja2.

```

1 class ActionAnalysisReport(Action):
2
3     def name(self) -> Text:
4         return "action_analysis_report"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        # Retrieve the report filename from the slot
11        report_filename = tracker.get_slot("report_filename")
12
13        if not report_filename:
14            dispatcher.utter_message(text="It looks like you haven't done an analysis yet..")
15            return []
16
17        directory = "./reports/"
18        file_path = os.path.join(directory, report_filename)
19
20        try:
21            # Read the report file
22            report_df = pd.read_csv(file_path)
23
24            # Generate the HTML report
25            html_report = self.generate_html_report(report_df)
26
27            dispatcher.utter_message(text=html_report)
28            dispatcher.utter_message(text="Feel free to ask for suggestions about issues fixing!")
29        except FileNotFoundError:
30            dispatcher.utter_message(text="Sorry, the report file could not be found.")
31            logging.error(f"Report file {file_path} not found.")
32        except pd.errors.EmptyDataError:
33            dispatcher.utter_message(text="Sorry, the report file is empty.")
34            logging.error(f"Report file {file_path} is empty.")
35        except Exception as e:
36            dispatcher.utter_message(text=ERROR_MESSAGE)
37            logging.error(f"Error while processing the report: {e}")
38
39        return []

```

Figura 3.27: Custom Action 'action_analysis_report'.

Custom Action: action_suggest_fix

La Custom Action denominata "action_suggest_fix", mostrata nella Figura 3.28, viene attivata quando l'utente richiede dei suggerimenti di refactoring per rimuovere i Code Smells identificati attraverso l'analisi.

Il metodo "run" di questa Custom Action segue i seguenti passaggi:

1. **Recupero dei Dati del Report:** Il nome del file di report è recuperato dallo slot "report_filename" insieme all'indice del Code Smell all'interno del report (entity "issue_index") per cui l'utente desidera dei suggerimenti di refactoring.

2. **Estrazione delle Informazioni dal Report:** La funzione "get_info_from_report" estrae dal report i dettagli specifici del Code Smell, inclusi il nome del file in cui è stato identificato ("cs_filename"), il nome della funzione ("cs_function_name") e il nome del Code Smell (cs_name).

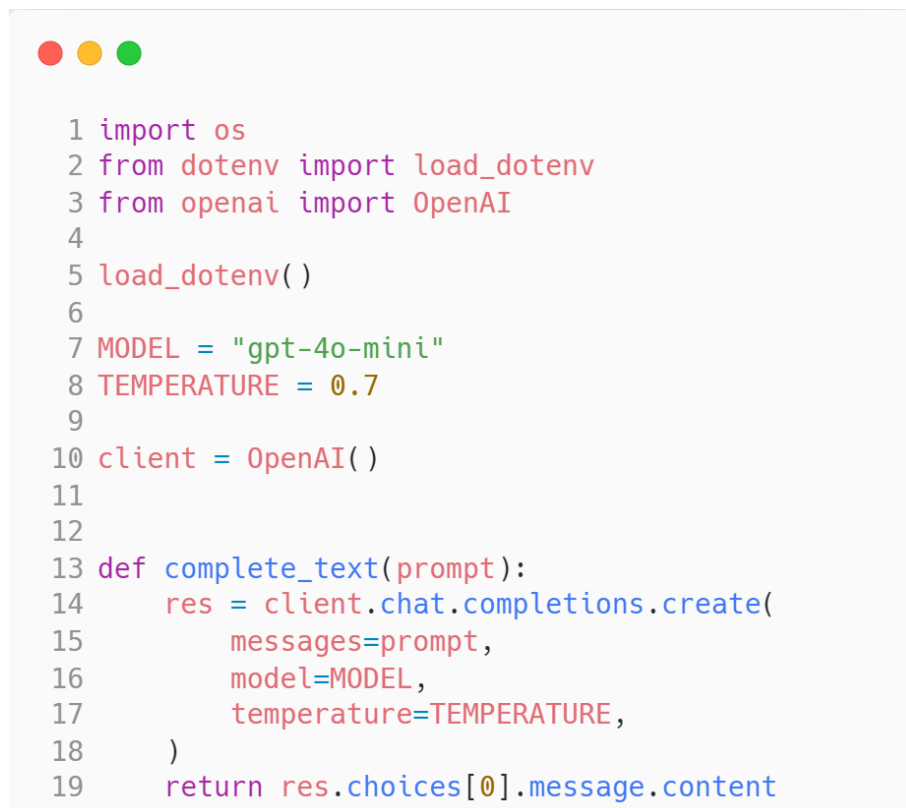
```

1 class ActionSuggestFix(Action):
2
3     def name(self) -> Text:
4         return "action_suggest_fix"
5
6     def run(self, dispatcher: CollectingDispatcher,
7             tracker: Tracker,
8             domain: Dict[Text, Any]) -> List[Dict[Text, Any]]:
9
10        # Retrieve the report filename from the slot
11        report_filename = tracker.get_slot("report_filename")
12        if not report_filename:
13            dispatcher.utter_message(text="It looks like you haven't done an analysis yet...")
14            return []
15
16        # Retrieve the issue index from the user's input
17        issue_index = next(tracker.get_latest_entity_values("issue_index"), None)
18        if not issue_index:
19            dispatcher.utter_message(text="It looks like you haven't provided the issue's index...")
20            return []
21
22        # Extract code smell information from the report
23        cs_filename, cs_function_name, cs_name = get_info_from_report(report_filename, issue_index)
24        if not cs_filename or not cs_function_name or not cs_name:
25            dispatcher.utter_message(text="It looks like you haven't provided a correct index...")
26            return []
27
28        # Prepare the request to get the function body
29        url = os.getenv("PROJECT_ANALYZER_BASE_URL") + "/get-function-body"
30        payload = {"filename": cs_filename, "function_name": cs_function_name}
31
32        try:
33            response = requests.post(url, json=payload)
34            response.raise_for_status() # Raise an exception for HTTP errors
35            function_body = response.json().get('function_body')
36            if not function_body:
37                dispatcher.utter_message(text="Could not retrieve the function body.")
38                return []
39
40            # Get Code Smell info
41            cs_name = code_smell_name_matcher(cs_name)[0]
42            cs_info = get_code_smell_by_name(cs_name, ["description", "problems", "solution", "prompt_example"])
43
44            # build the prompt for the LLM
45            prompt = self.create_prompt(cs_name, cs_info, cs_function_name, function_body)
46            suggestion = complete_text(prompt)
47            suggestion = self._preserve_empty_lines(suggestion)
48
49            # Send the suggestion to the user
50            dispatcher.utter_message(text=f"Here's how you could fix the code smell \"{cs_name}\" "
51                                    f"within the function \"{cs_function_name}\":\n{suggestion}")
52
53        except requests.RequestException as e:
54            dispatcher.utter_message(text="An error occurred while retrieving the function body.")
55            logging.error("Request error during action suggest fix: %s", e)
56        except Exception as e:
57            dispatcher.utter_message(text="An error occurred while suggesting the fix.")
58            logging.error("Error during action suggest fix: %s", e)
59
60        return []

```

Figura 3.28: Codice della Custom Action 'action_suggest_fix'.

3. **Richiesta del Corpo della Funzione:** Utilizzando le informazioni recuperate nel passo precedente, viene effettuata una richiesta HTTP POST all'endpoint `"/get-function-body"` dell'API del tool di analisi per ottenere il corpo della funzione contenente il Code Smell.
4. **Preparazione del Prompt:** Una query al database recupera le informazioni complete sul Code Smell. La funzione `"create_prompt"` prepara un prompt dettagliato che sarà inviato al LLM GPT-4o mini, integrando tutti i dati pertinenti per facilitare la generazione di suggerimenti di refactoring validi.
5. **Richiesta di Completamento al LLM:** Il prompt viene inviato alla funzione `"complete_text"`, mostrata nella Figura 3.29, la quale gestisce la richiesta di completamento al LLM. La risposta ottenuta viene formattata usando la funzione `"_preserve_empty_lines"` per mantenere la struttura originale del codice e poi restituita all'utente come suggerimento di refactoring.

The image shows a code editor window with a light gray background. At the top left, there are three colored circles (red, yellow, green) representing window control buttons. The code is written in Python and is as follows:

```
1 import os
2 from dotenv import load_dotenv
3 from openai import OpenAI
4
5 load_dotenv()
6
7 MODEL = "gpt-4o-mini"
8 TEMPERATURE = 0.7
9
10 client = OpenAI()
11
12
13 def complete_text(prompt):
14     res = client.chat.completions.create(
15         messages=prompt,
16         model=MODEL,
17         temperature=TEMPERATURE,
18     )
19     return res.choices[0].message.content
```

Figura 3.29: Funzione `'complete_text'`.

3.3.8 Endpoints

Rasa consente l'integrazione dei chatbot con diverse piattaforme, come Slack e Facebook, oltre a supportare l'integrazione su piattaforme web personali. Questo permette ai Chatbot di interagire con gli utenti su una varietà di canali, migliorando l'accessibilità e l'utilità del Chatbot stesso.

Gli endpoint devono essere configurati all'interno del file "credentials.yml", specificando, quando necessario, i token di autenticazione. Nel contesto di CAMILLE sono stati configurati due tipi di endpoint:

- **REST:** L'endpoint REST permette di inviare messaggi al chatbot tramite richieste HTTP POST e ricevere risposte in formato JSON. È semplice da configurare e da utilizzare, oltre ad essere supportato dalla maggior parte delle tecnologie web attuali;
- **SocketIO:** L'endpoint SocketIO utilizza i WebSocket e permette una comunicazione bidirezionale in tempo reale tra il client e il server. Offre una comunicazione più rapida e interattiva rispetto ai tradizionali endpoint REST, ma è anche più complesso da configurare e utilizzare.

Nonostante siano stati configurati entrambi gli endpoint per facilitare future integrazioni, al momento l'unico endpoint utilizzato è il REST, grazie alla sua semplicità e facilità di implementazione. Questo consente di mantenere il sistema semplice e robusto, garantendo al contempo la possibilità di espandere le funzionalità in futuro con l'integrazione su altre piattaforme tramite l'utilizzo dei WebSocket.

3.4 Sviluppo dell'Estensione per Visual Studio Code

Per facilitare l'utilizzo dei servizi offerti da CAMILLE attraverso un'interfaccia grafica intuitiva ed efficiente, è stata sviluppata un'apposita estensione per Visual Studio Code, come è possibile osservare nella Figura 3.30. Tale estensione è stata implementata sfruttando le capacità della WebView API, che permette l'integrazione di contenuti HTML all'interno dell'IDE, fornendo così un'esperienza utente fluida e integrata direttamente nell'ambiente di sviluppo.

Nella sezione seguente, verranno esplorati in dettaglio i componenti chiave che hanno consentito la realizzazione dell'estensione per CAMILLE, mettendo in luce le scelte tecnologiche e le strategie adottate per garantirne il funzionamento e assicurare un'interazione utente ottimale.

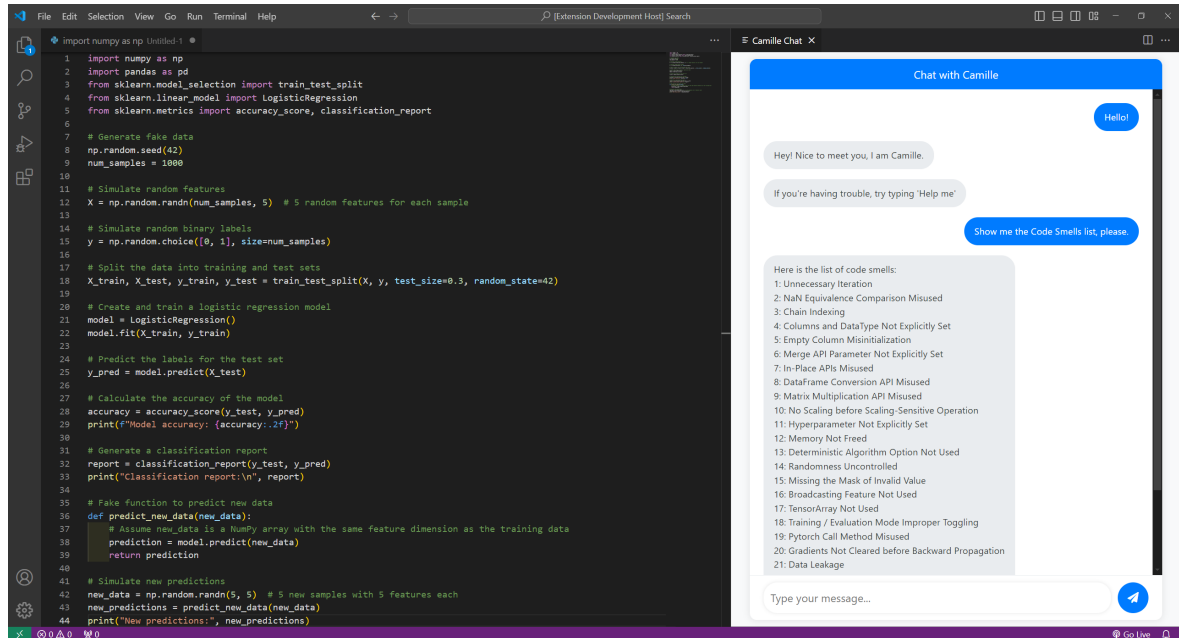


Figura 3.30: Estensione di CAMILLE all'interno di VS Code.

3.4.1 Setup Iniziale

Il setup iniziale dell'estensione per Visual Studio Code, dedicata a CAMILLE, è stato gestito mediante l'impiego di Yeoman, uno strumento di scaffolding ampiamente apprezzato per la sua efficacia nella configurazione iniziale di progetti software. Questo strumento ha semplificato la creazione della struttura di base del progetto, offrendo la possibilità di selezionare una serie di opzioni chiave durante la fase di inizializzazione. In questo contesto, sono stati definiti parametri cruciali quali il nome dell'estensione, l'adozione di TypeScript come linguaggio di programmazione e l'utilizzo di npm come gestore dei pacchetti.

Successivamente alla definizione di queste impostazioni, Yeoman ha automaticamente generato tutti i file di configurazione necessari, inclusa la predisposizione delle dipendenze per le librerie e la configurazione dell'ambiente di sviluppo. L'approccio adottato ha non solo ottimizzato i tempi di setup, ma ha anche minimizzato il rischio

di errori manuali, assicurando un avvio agile e strutturato del progetto, oltre che garantire una base solida e coerente per i successivi sviluppi dell'estensione.

3.4.2 Configurazione dell'estensione

Uno degli elementi fondamentali nello sviluppo dell'estensione per Visual Studio Code è rappresentato dal file "extension.ts". Questo file costituisce il cuore dell'intera estensione, ospitando la logica cruciale per il suo funzionamento all'interno dell'IDE. Il file "extension.ts" è progettato per gestire il ciclo di vita dell'estensione, registrare i comandi disponibili per l'utente e coordinare le interazioni con la WebView.

Le sottosezioni seguenti descriveranno in dettaglio le diverse componenti del file "extension.ts", analizzandone il funzionamento e le funzionalità specifiche.

Gestione delle Risorse

Le WebView all'interno di Visual Studio Code sono eseguite in un ambiente isolato, noto come sandbox, che adotta rigide politiche di sicurezza per mitigare rischi quali attacchi di scripting cross-site o accessi non autorizzati ai dati del sistema. Queste politiche restrittive impediscono il caricamento diretto di risorse esterne, come fogli di stile CSS o script JavaScript, tramite percorsi relativi o assoluti direttamente nel codice HTML.

```
1 // Load Resource File inside the HTML
2 function getWebViewContent(context: vscode.ExtensionContext, panel: vscode.WebviewPanel): string {
3     const htmlPath = path.join(context.extensionPath, 'src', 'index.html');
4     let htmlContent = fs.readFileSync(htmlPath, 'utf8');
5
6     // Build Safe URI
7     const scriptPathOnDisk = vscode.Uri.file(path.join(context.extensionPath, 'src', 'scripts.js'));
8     const scriptUri = panel.webview.asWebviewUri(scriptPathOnDisk);
9     const stylePathOnDisk = vscode.Uri.file(path.join(context.extensionPath, 'src', 'styles.css'));
10    const styleUri = panel.webview.asWebviewUri(stylePathOnDisk);
11
12    // Replace Placeholder
13    htmlContent = htmlContent.replace(/<script src="scripts.js"><\script>/,
14        '<script src="${scriptUri}"></script>');
15    htmlContent = htmlContent.replace(/<link rel="stylesheet" href="styles.css">/,
16        '<link rel="stylesheet" href="${styleUri}">');
17
18    return htmlContent;
19 }
```

Figura 3.31: Funzione "GetWebViewContent".

Per superare questa limitazione, è stata sviluppata la funzione "getWebviewContent", la cui implementazione è illustrata nella Figura 3.31. Questa funzione utilizza un meccanismo specializzato per gestire le risorse all'interno delle WebView, basato sul mapping delle risorse esterne tramite URI speciali che Visual Studio Code può interpretare e servire in modo sicuro. Le operazioni eseguite dalla funzione "getWebviewContent" includono:

1. **Trasformazione dei Percorsi delle Risorse:** Utilizza la funzione "vscode.Uri.file" per convertire i percorsi fisici delle risorse su disco in URI che Visual Studio Code può gestire in modo sicuro.
2. **Conversione degli URI per la WebView:** Tramite il metodo "asWebviewUri", gli URI ottenuti vengono trasformati in URI autorizzati a essere caricati all'interno delle WebView.
3. **Sostituzione dei Riferimenti nel Codice HTML:** I riferimenti ai file di risorse nel codice HTML vengono sostituiti con gli URI sicuri generati nei passi precedenti.

La funzione "getWebviewContent" garantisce che gli script e i fogli di stile siano caricati correttamente, preservando le funzionalità dell'applicazione e rispettando gli standard di sicurezza imposti dall'ambiente sandbox.

Attivazione dell'estensione

La funzione "activate" è invocata automaticamente da Visual Studio Code quando l'estensione viene attivata dall'utente. Questa funzione ha il compito primario di inizializzare l'estensione, configurare i suoi componenti principali e preparare l'ambiente per l'interazione con l'utente. Il processo di attivazione, il cui codice è mostrato nella Figura 3.32 comprende i seguenti passaggi fondamentali:

1. **Registrazione del Comando:** Un comando specifico, "Camille.showChat", è registrato all'interno dell'ambiente VS Code. Questo comando funge da punto di ingresso per gli utenti che desiderano avviare l'interfaccia del chatbot CAMILLE;

2. **Creazione del Pannello della WebView:** Viene creato un pannello specifico all'interno di Visual Studio Code, posizionato strategicamente, tramite l'opzione "vscode.ViewColumn.Two", per facilitare l'accessibilità e la visibilità;
3. **Caricamento del Contenuto:** Il contenuto HTML, generato dalla funzione "getWebviewContent", è caricato nella WebView. Questo contenuto include riferimenti a script e stili che sono stati adattati per rispettare le politiche di sicurezza di Visual Studio Code.



```

1 //Activate Extension
2 export function activate(context: vscode.ExtensionContext) {
3     context.subscriptions.push(
4         // Register Command
5         vscode.commands.registerCommand('Camille.showChat', () => {
6             // WebView Creation
7             const panel = vscode.window.createWebviewPanel(
8                 'camilleChat' // ID
9                 'Camille Chat', // Title
10                vscode.ViewColumn.Two, // Position
11                {
12                    enableScripts: true, // Enable Scripts
13                    retainContextWhenHidden: true, // Prevent reloading
14                    localResourceRoots: [vscode.Uri.file(path.join(context.extensionPath, 'src'))]
15                }
16            );
17            panel.webview.html = getWebviewContent(context, panel);
18            ....
19        })
20    );
21 }
22 }
23 }
24

```

Figura 3.32: Attivazione dell'estensione e configurazione WebView.

È stato di fondamentale importanza, per garantire il corretto funzionamento dell'estensione, la configurazione di due specifiche opzioni della WebView:

- **enableScript:** Questa opzione ha reso possibile l'esecuzione di script JavaScript all'interno del file HTML utilizzato dalla WebView;
- **retainContextWhenHidden:** Quando la WebView viene nascosta, per motivi legati alle performance, viene automaticamente distrutta da VS Code. Attivando questa opzione è stato modificato il comportamento di VS Code così da prevenire il ricaricamento della WebView con la conseguente perdita dei messaggi inviati in precedenza.

Gestione dei Messaggi

La WebView, nell'ambiente di Visual Studio Code, può interagire con l'IDE attraverso uno scambio di messaggi. I messaggi e le relative logiche di business che devono essere eseguite alla loro ricezione sono definiti all'interno della funzione "activate". Nel contesto dell'estensione per CAMILLE, come mostrato dalla Figura 3.33, è stato implementato il supporto per un comando specifico, denominato "downloadReport". Come suggerito dal nome, questo messaggio consente all'utente di scaricare il file di report inviato dal Chatbot e di salvarlo localmente sul proprio sistema.

```

1 // Activate Estensione
2 export function activate(context: vscode.ExtensionContext) {
3     context.subscriptions.push(
4
5         ....
6
7         // Manage Messages Received
8         panel.webview.onDidReceiveMessage(
9             async message => {
10                 switch (message.command){
11                     case 'downloadReport':
12                         // Open Folder Selection Window
13                         const folderUri = await vscode.window.showOpenDialog({
14                             canSelectFolders: true,
15                             canSelectFiles: false,
16                             canSelectMany: false,
17                             openLabel: 'Seleziona la cartella di destinazione'
18                         });
19
20                         // Save the report
21                         if (folderUri && folderUri[0]) {
22                             const filePath = path.join(folderUri[0].fsPath, 'CamilReport.html');
23                             fs.writeFileSync(filePath, message.content);
24                             vscode.window.showInformationMessage('File scaricato: ' + filePath);
25                         } else {
26                             vscode.window.showErrorMessage('Nessuna cartella selezionata');
27                         }
28                         return;
29                     }
30                 },
31                 undefined,
32                 context.subscriptions
33             );
34         });
35     };
36 }

```

Figura 3.33: Gestione Messaggi della WebView.

Alla ricezione di un messaggio contenente il comando "downloadReport", vengono intraprese le seguenti azioni:

1. **Apertura della Finestra di Dialogo:** Viene presentata all'utente una finestra di

dialogo ("showOpenDialog") che permette di selezionare la cartella di destinazione per il salvataggio del file di report;

2. **Gestione della Selezione della Cartella:** Nel caso in cui l'utente selezioni una cartella, l'URI di questa viene utilizzato per costruire il percorso completo del file di report. Il contenuto del report, situato nel campo "content" del messaggio, viene quindi scritto nel file specificato tramite "fs.writeFileSync".
3. **Feedback all'utente:** Se il file viene creato con successo, l'utente riceve un messaggio informativo che conferma il download e mostra il percorso del file salvato. Se invece non è stata selezionata nessuna cartella, viene mostrato un messaggio di errore.

La gestione del download mediante questo sistema di scambio di messaggi è necessaria perché, a causa delle rigide politiche di sicurezza imposte dalla sandbox in cui viene eseguita la WebView, non è possibile implementare tali funzionalità direttamente tramite JavaScript come si farebbe in una normale pagina web.

3.4.3 Realizzazione della WebView

L'interfaccia utente dell'estensione, realizzata tramite una WebView, sfrutta le consuete tecnologie di sviluppo web, quali HTML, CSS e JavaScript. In questa sezione, non verranno trattati i dettagli meno rilevanti per la comprensione complessiva del progetto, come i file HTML e CSS. Piuttosto, l'attenzione sarà focalizzata sugli script che hanno garantito il funzionamento efficace della WebView e la sua interazione sia con il chatbot CAMILLE sia con l'ambiente di sviluppo integrato (IDE) di Visual Studio Code.

GestioneGUID

La gestione del Global Unique Identifier (GUID) è essenziale per tracciare in modo univoco le sessioni degli utenti all'interno dell'interfaccia della WebView. Questo identificativo permette di mantenere una traccia consistente delle interazioni degli utenti con il chatbot, assicurando che i messaggi inviati siano correttamente associati

alla sessione utente appropriata. Di seguito sono descritte le funzioni JavaScript impiegate per la gestione del GUID, illustrate nella Figura 3.34:

- **Generazione del GUID:** La funzione "generateGUID" è progettata per creare un nuovo GUID in formato standard UUID. Essa genera un identificativo basandosi su un template ben definito e producendo casualmente dei valori esadecimali.
- **Recupero e Memorizzazione del GUID:** La funzione "getOrGenerateGUID" verifica la presenza di un GUID già memorizzato localmente. In assenza di questo, genera un nuovo GUID tramite "generateGUID" e lo memorizza nel "localStorage" per garantirne la persistenza;



```

1 //GUID Generation
2 function generateGUID() {
3     return 'xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx'.replace(/[xy]/g, function(c) {
4         var r = Math.random() * 16 | 0,
5             v = c === 'x' ? r : (r & 0x3 | 0x8);
6         return v.toString(16);
7     });
8 }
9
10 //Obtain the GUID or generate and save a GUID in localStorage
11 function getOrGenerateGUID() {
12     let sender = localStorage.getItem('sender');
13     if (!sender) {
14         sender = generateGUID();
15         localStorage.setItem('sender', sender);
16     }
17     return sender;
18 }
19
20 //Remove GUID from localStorage
21 function clearGUID() {
22     localStorage.removeItem('sender');
23 }
24
25 //Generate GUID on a new window
26 window.onload = function() {
27     getOrGenerateGUID();
28 };
29
30 //Remove GUID when closing window
31 window.addEventListener('beforeunload', function() {
32     clearGUID();
33 });

```

Figura 3.34: Gestione del Global Unique Identifier (GUID).

- **Rimozione del GUID:** La funzione "clearGUID" consente di eliminare il GUID dal "localStorage" quando non è più necessario, ad esempio alla chiusura della sessione;

- **Gestione degli Eventi della Finestra:** All'apertura di una nuova finestra, la presenza di un GUID è assicurata attraverso l'evento "onload". In modo simile, l'evento "beforeunload" si occupa di rimuovere il GUID prima della chiusura della finestra.

Queste funzionalità assicurano che ogni sessione utente sia univocamente identificata e prevengono possibili conflitti tra le sessioni di conversazione di utenti diversi, migliorando l'affidabilità e la sicurezza dell'interazione con il chatbot.

Comunicazione con il Chatbot

La funzione "sendMessage", illustrata nella Figura 3.35, riveste un ruolo cruciale nell'interfaccia utente dell'estensione, gestendo sia l'invio di messaggi degli utenti al chatbot CAMILLE sia la ricezione delle risposte. Questa funzione viene attivata quando l'utente invia un messaggio utilizzando il pulsante dedicato nella WebView o premendo il tasto "Enter". Le operazioni eseguite all'interno della funzione includono:

1. **Visualizzazione del Messaggio nella UI:** Il messaggio, raccolto dall'apposita casella di testo, viene visualizzato nella finestra di chat. Contemporaneamente, viene mostrata un'animazione di caricamento per segnalare che l'invio del messaggio è in fase di elaborazione;
2. **Invio del messaggio al Chatbot:** Il messaggio viene inviato all'endpoint REST del chatbot tramite la funzione "fetch". Oltre al testo del messaggio, viene incluso anche il GUID, che viene generato non appena la WebView viene caricata, per assicurare l'unicità della sessione utente.
3. **Gestione della Risposta:** Dopo aver ricevuto la risposta dal chatbot, l'animazione di caricamento viene rimossa. Le risposte ricevute vengono elaborate e visualizzate nell'interfaccia della chat. Queste possono variare da messaggi testuali semplici a contenuti più complessi, come script di codice o report HTML, che vengono gestiti e presentati in modo appropriato all'interno dell'interfaccia utente.



```

1 function sendMessage() {
2   const chatInput = document.getElementById('chat-input');
3   const chatBody = document.getElementById('chat-body');
4   const message = chatInput.value;
5
6   if (message.trim() !== '') {
7     // Append user message
8     const userMessage = document.createElement('div');
9     userMessage.classList.add('chat-message', 'user');
10    userMessage.innerHTML = `<div class="message">${message}</div>`;
11    chatBody.appendChild(userMessage);
12
13    ....
14
15    // Make API call to Rasa Chatbot
16    const sender = getOrGenerateGUID();
17    fetch('http://localhost:5005/webhooks/rest/webhook', {
18      method: 'POST',
19      headers: {
20        'Content-Type': 'application/json'
21      },
22      body: JSON.stringify({
23        sender: sender,
24        message: message
25      })
26    })
27    .then(response => response.json())
28    .then(data => {
29      //Manage the response
30      data.forEach(botResponse => {
31        ....
32      });
33
34      // Highlight code blocks in the new messages
35      hljs.highlightAll();
36    })
37  }
38 }

```

Figura 3.35: Invio del messaggio al Chatbot all'interno della WebView.

Highlighting del Codice

L'integrazione della libreria "highlight.js" nella WebView migliora notevolmente l'esperienza utente, offrendo una visualizzazione chiara e formattata del codice. Questa caratteristica risulta particolarmente vantaggiosa quando il Chatbot fornisce risposte contenenti blocchi di codice, come per gli esempi di Code Smell o per i suggerimenti di refactoring, assicurando che tali blocchi siano facilmente leggibili. La funzione "formatMessage", essenziale per il funzionamento di "highlight.js" e mostrata nella Figura 3.36, esegue le seguenti operazioni per ottimizzare la presentazione del codice:

1. **Identificazione dei Blocchi di Codice:** Implementa un'espressione regolare per identificare e isolare i blocchi di codice Python all'interno del testo, assicurando che ogni segmento sia correttamente riconosciuto e trattato;
2. **Formattazione del Testo:** Racchiude i blocchi di codice nei tag HTML necessari per l'attivazione di "highlight.js". Inoltre, converte i caratteri di escape "\n" in "
" per garantire una visualizzazione corretta nel contesto HTML;
3. **Aggiunta Funzionalità di Copia:** Introduce un pulsante accanto a ciascun blocco di codice. Questo pulsante, collegato alla funzione "copyToClipboard", permette agli utenti di copiare facilmente il codice negli appunti, migliorando significativamente l'interazione con l'interfaccia.

```

1 function formatMessage(text) {
2   // python code blocks pattern
3   const codeBlockPattern = /``python([\s\S]*)?``/g;
4   let formattedText = '';
5   let lastIndex = 0;
6
7   text.replace(codeBlockPattern, (match, code, offset) => {
8     formattedText += text.slice(lastIndex, offset).replace(/\n/g, '<br>');
9     //Add copy button for code block
10    formattedText += `<div class="code-container">
11      <button class="copy-btn" onclick="copyToClipboard(this)"><i class="fas fa-copy"></i></button>
12      <pre><code class="language-python">${code}</code></pre>
13    </div>`;
14    lastIndex = offset + match.length;
15  });
16
17  // Replace \n with <br>
18  formattedText += text.slice(lastIndex).replace(/\n/g, '<br>');
19  return formattedText;
20 }

```

Figura 3.36: Formattazione dei blocchi di codice per "highlight.js".

Download del Report

La funzionalità di gestione dei report all'interno dell'interfaccia della WebView consente agli utenti di visualizzare e scaricare report generati dal chatbot CAMILLE. Quando la comunicazione con il chatbot genera una risposta che include un report in formato HTML, la funzione "sendMessage" (vedi Sez. 3.4.3) elabora il report per visualizzarlo all'interno di un iFrame nella UI dell'estensione. Tuttavia, può essere necessario salvare il report o visualizzarlo in modo più pratico all'esterno dell'IDE. A questo scopo è stata sviluppata la funzione "downloadReport", illustrata nella Figura 3.37, che compie le seguenti operazioni:

1. **Decodifica del Contenuto HTML:** Il contenuto del report, passato come parametro codificato per URL, viene decodificato per ottenere il file HTML originale;
2. **Invio del Comando di Download:** Un messaggio viene inviato all'API di Visual Studio Code tramite "vscode.postMessage", comunicando il comando "downloadReport" e il contenuto HTML decodificato del report.

A screenshot of a code editor window with a light gray background. At the top left, there are three colored window control buttons: red, yellow, and green. The code is written in JavaScript and is syntax-highlighted. It starts with a comment '1 // VsCode API', followed by '2 const vscode = acquireVsCodeApi();'. Then there is a blank line '3', followed by a comment '4 //Download the report'. The next line is '5 function downloadReport(encodedContent) {' with a closing brace on line 13. Inside the function, line 6 has '6 const decodedContent = decodeURIComponent(encodedContent);', line 7 is blank, line 8 has a comment '8 //Send a message to VsCode API', and line 9 has '9 vscode.postMessage({' with a closing brace on line 12. Line 10 has '10 command: 'downloadReport'', line 11 has '11 content: decodedContent', and line 12 has '12 });'. Line 13 has '13 }'.

Figura 3.37: Download del report HTML.

Questo processo di comunicazione garantisce che il contenuto del report possa essere scaricato direttamente sul sistema dell'utente (vedi Sez. 3.4.2), preservando l'integrità del formato e del contenuto del report originale.

Valutazione Preliminare dell'Efficacia di CAMILLE

Per valutare l'efficacia e le potenzialità di CAMILLE, si è reso necessario concentrare l'attenzione su test relativi alla sua funzionalità più critica, la quale consiste nel fornire suggerimenti di refactoring orientati alla rimozione degli ML-Specific Code Smells rilevati all'interno dei progetti, sfruttando le potenzialità di GPT-4o mini.

Il presente capitolo esamina in maniera approfondita la metodologia di testing adottata per questa valutazione, delineando ogni passaggio del processo e mettendo in luce come le diverse fasi siano state progettate per garantire un'analisi accurata e dettagliata. Successivamente, si procederà all'analisi dei risultati ottenuti, che confermeranno o meno l'efficacia delle soluzioni proposte da GPT-4o mini.

4.1 Metodologia di Testing

Prima di procedere con la valutazione pratica della potenzialità di CAMILLE, è stato fondamentale definire un'approccio metodico e strutturato per il testing.

Questa sezione dettaglia il processo adottato per testare l'efficacia dei suggerimenti di refactoring forniti da CAMILLE tramite GPT-4o mini.

4.1.1 Dataset

Il primo passo per condurre l'attività di testing è stato individuare un numero sufficiente di progetti di Machine Learning in Python su cui eseguire l'analisi. A tal fine è stato utilizzato NICHE [48], un dataset eterogeneo contenente dati relativi a più di 500 progetti di Machine Learning. Questo ha permesso di avere a disposizione un'ampia e variegata gamma di campioni di test per l'analisi.

4.1.2 Procedimento

Una volta individuato il dataset adatto, è stato sviluppato un procedimento per condurre l'attività di testing. In particolare, per ogni progetto contenuto in NICHE [48], sono state seguite le seguenti operazioni:

1. **Cloning del Progetto:** Il dataset includeva informazioni sulle repository GitHub di ogni progetto, permettendo così di clonare facilmente i progetti e memorizzarli localmente.
2. **Analisi Pre-Refactoring:** Il progetto clonato nel passo precedente è stato analizzato utilizzando CodeSmile [22]. Per ogni Code Smell rilevato sono state effettuate le seguenti operazioni:
 - (a) **Recupero Informazioni:** Sono state recuperate tutte le informazioni riguardanti il Code Smell e la funzione all'interno del quale è stato rilevato;
 - (b) **Richiesta di Refactoring:** Con le informazioni raccolte, è stato creato un prompt per richiedere a GPT-4o mini un suggerimento di refactoring per rimuovere il Code Smell;
 - (c) **Refactoring del Codice:** La funzione contenente il Code Smell è stata sostituita con la versione rifattorizzata suggerita da GPT-4o mini.
3. **Analisi Post-Refactoring:** Dopo aver effettuato il refactoring di ogni funzione affetta da Code Smells, è stata avviata un'altra analisi del progetto tramite CodeSmile [22] per verificare in quanti casi GPT-4o mini ha fornito suggerimenti effettivamente validi.

Dopo aver automatizzato l'intero processo di testing ed aver completato le operazioni descritte, i risultati di tutte le analisi Pre-Refactoring sono stati aggregati in un unico file CSV, così come i risultati di tutte le analisi Post-Refactoring. Questo ha permesso di semplificare notevolmente la successiva operazione di analisi dei risultati ottenuti.

4.1.3 Refactoring del Codice

Per realizzare le varie fasi dell'intero processo di testing, le funzioni già progettate per la realizzazione di CAMILLE si sono rivelate più che adatte. Però, è importante ricordare che CAMILLE non effettua il refactoring automaticamente; piuttosto, è compito dello sviluppatore modificare il codice sorgente del proprio progetto basandosi sui suggerimenti forniti da CAMILLE. Pertanto, è stata necessaria l'adozione di una soluzione alternativa che permettesse la sostituzione automatica delle funzioni contenenti Code Smells con le nuove funzioni proposte da GPT-4o mini. A questo scopo, è stata fondamentale la realizzazione del modulo "function_replacer.py", illustrato nella Figura 4.1.

Il modulo "function_replacer.py" sfrutta le potenzialità degli Abstract Syntax Tree (AST) per ricercare e modificare le funzioni all'interno dei file di codice sorgente. L'operazione di sostituzione avviene attraverso varie fasi:

1. **Analisi del File Sorgente:** Attraverso la funzione "replace_function_in_file", il file sorgente viene letto e il suo contenuto viene analizzato in un AST;
2. **Modifica del Codice:** Utilizzando la classe "FunctionReplacer", l'AST viene visitato e, se necessario, modificato per sostituire la funzione specificata in "function_name" con "new_function";
3. **Conversione dell'AST:** Dopo la modifica dell'AST, tramite il modulo "astor", questo viene riconvertito in un file di codice sorgente in formato UTF-8. L'uso di "astor" assicura di evitare problemi di formattazione, come indentazioni errate.

Grazie al modulo "function_replacer.py", durante il processo di testing, è stato possibile sostituire automaticamente ed in maniera efficace le funzioni affette da Code Smells con le nuove funzioni suggerite da GPT-4o mini. Ovviamente, prima di

effettuare la sostituzione, i suggerimenti sono stati opportunamente ripuliti tramite apposite funzioni, così da eliminare eventuali elementi di disturbo che avrebbero potuto causare problemi nel processo di sostituzione, come ad esempio i backtick tripli che vengono solitamente utilizzati dagli LLM per delimitare i blocchi di codice.



```

1 import ast
2 import astor
3
4
5 class FunctionReplacer(ast.NodeTransformer):
6     def __init__(self, function_name, new_function):
7         self.function_name = function_name
8         self.new_function = ast.parse(new_function).body[0]
9
10    def visit_FunctionDef(self, node):
11        if node.name == self.function_name:
12            return self.new_function
13        return node
14
15
16 def replace_function_in_file(file_name, function_name, new_function):
17     # Read the file's content and build AST
18     with open(file_name, 'r', encoding='utf-8') as file:
19         tree = ast.parse(file.read(), filename=file_name)
20
21     # Apply Transformer
22     replacer = FunctionReplacer(function_name, new_function)
23     new_tree = replacer.visit(tree)
24
25     # Convert the AST in source code
26     with open(file_name, 'w', encoding='utf-8') as file:
27         file.write(astor.to_source(new_tree))

```

Figura 4.1: Modulo "function_replacer.py".

4.2 Analisi dei Risultati

L'obiettivo di questa sezione è esaminare l'impatto del processo di Refactoring effettuato sfruttando le funzionalità di CAMILLE, in particolare analizzando la riduzione del numero di istanze dei vari ML-Specific Code Smells identificati prima e dopo l'applicazione dei suggerimenti di refactoring proposti da GPT-4o mini. Questo permette di valutare il potenziale impatto che può avere l'utilizzo di CAMILLE per migliorare la qualità del codice e ridurre il Technical Debt.

4.2.1 Distribuzione dei Code Smells

Durante il testing, sono stati analizzati 523 progetti, nei quali sono stati individuati in media circa 7 ML-Specific Code Smells per progetto, totalizzando 3759 Code Smells. Dopo l'applicazione delle modifiche suggerite da GPT-4o mini alle funzioni affette da Code Smells, si è osservata una diminuzione dei Code Smells del 62,97%, con un totale di 1392 Code Smells (circa 3 per progetto).

Il grafico mostrato nella Figura 4.2 illustra la distribuzione dei vari ML-Specific Code Smells. In particolare, per ciascun Code Smell, la barra rossa rappresenta il numero di istanze prima del refactoring, mentre la barra blu indica il numero di istanze dopo il refactoring. Di seguito è riportata una sintesi dei cambiamenti osservati per i vari tipi di Code Smells:

- **Columns and DataType Not Explicitly Set:** Da 1655 a 562 istanze, mostrando un significativo miglioramento.
- **Pytorch Call Method Misused:** Ridotto significativamente da 712 a 59 istanze.
- **TensorArray Not Used:** Da 454 a 367 istanze, mostrando una riduzione meno marcata.
- **In-Place APIs Misused:** Diminuzione da 437 a 296 istanze.
- **Gradients Not Cleared before Backward Propagation:** Riduzione da 393 a 97 istanze.
- **Dataframe Conversion API Misused:** Forte riduzione da 82 a 4 istanze.

Va notato che CodeSmile [22], il tool di analisi utilizzato da CAMILLE, è in grado di identificare 14 tipologie di ML-Specific Code Smells. Tuttavia, il grafico presenta solamente 6 diverse tipologie di Code Smells. Gli altri 8 Code Smells sono stati raggruppati nella categoria "altri", poiché sono stati identificati in numeri troppo esigui, o addirittura nulli, per fornire dati statistici significativi. Questa scarsa frequenza, benché limiti l'analisi statistica per una parte dei Code Smells, è indicativa del fatto che questi non sono particolarmente ricorrenti nei progetti di Machine Learning, probabilmente perché gli sviluppatori tendono a evitarli adottando soluzioni ottimali o perché le stesse librerie interessate hanno rilasciato con il tempo soluzioni

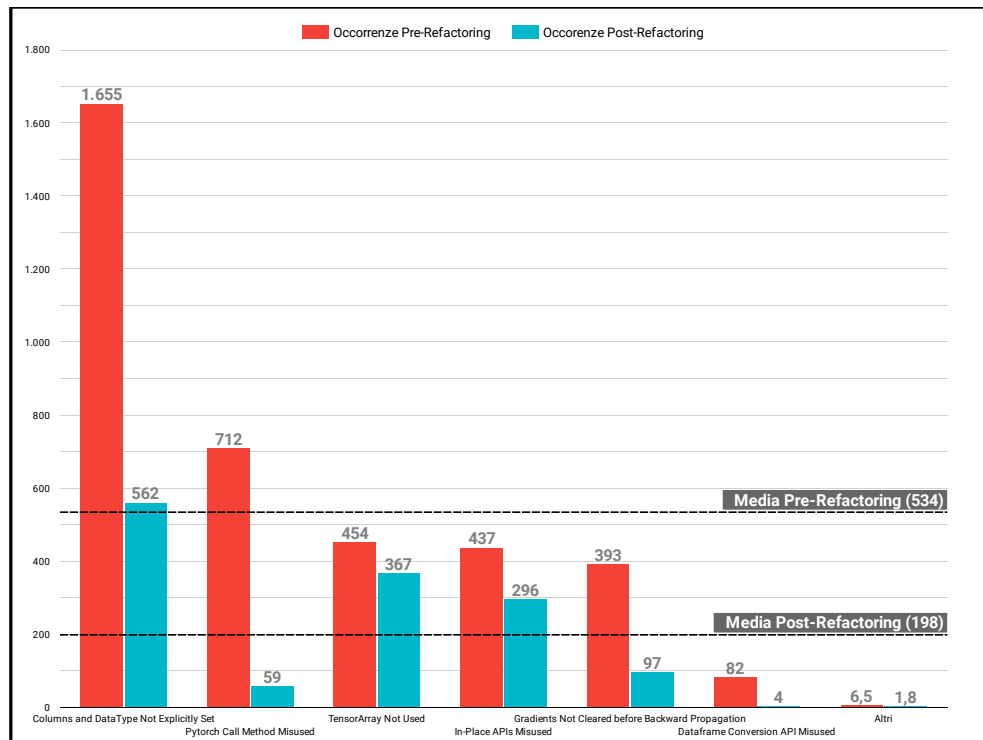


Figura 4.2: Report sulla distribuzione dei Code Smells prima e dopo il refactoring.

alternative più efficaci. Tutto ciò inoltre ci suggerisce l'importanza di concentrarsi principalmente sui Code Smells più frequenti elencati in precedenza.

4.2.2 Tasso di Riduzione dei Code Smells

Dai dati presentati nella sezione precedente, possiamo calcolare le percentuali di riduzione per le varie tipologie di ML-Specific Code Smells. Nel grafico in Figura 4.3, osserviamo che per i 6 Code Smells più ricorrenti, la percentuale media di riduzione si attesta intorno al 63.27%. Questo indica che i suggerimenti di refactoring forniti da GPT-4o mini, secondo l'analisi di CodeSmile [22], sono stati efficaci nel rimuovere i Code Smells nel 63.27% dei casi. Analizzando più nel dettaglio le percentuali relative a ciascun Code Smell, possiamo effettuare le seguenti osservazioni:

- **Columns and DataType Not Explicitly Set:** Mostra una riduzione del 66.04%, leggermente superiore alla media.
- **Pytorch Call Method Misused:** Dopo il testing su 712 istanze, ha ottenuto una riduzione del 91.71%, significativamente superiore alla media, rappresentando il risultato più convincente dell'attività di testing.

- **TensorArray Not Used:** Con una riduzione del 19.16%, si conferma come il risultato meno soddisfacente.
- **In-Place APIs Misused:** Con una riduzione del 32.37%, risulta essere migliore rispetto al precedente, ma ancora sotto la media.
- **Gradients Not Cleared before Backward Propagation:** Mostra ottimi risultati con una riduzione del 75.32%.
- **Dataframe Conversion API Misused:** Presenta un tasso di riduzione del 95.12%. Sebbene il numero di istanze testate sia stato inferiore rispetto agli altri casi (82 istanze), i risultati suggeriscono che la percentuale di riduzione sarebbe probabilmente rimasta alta in ogni caso.

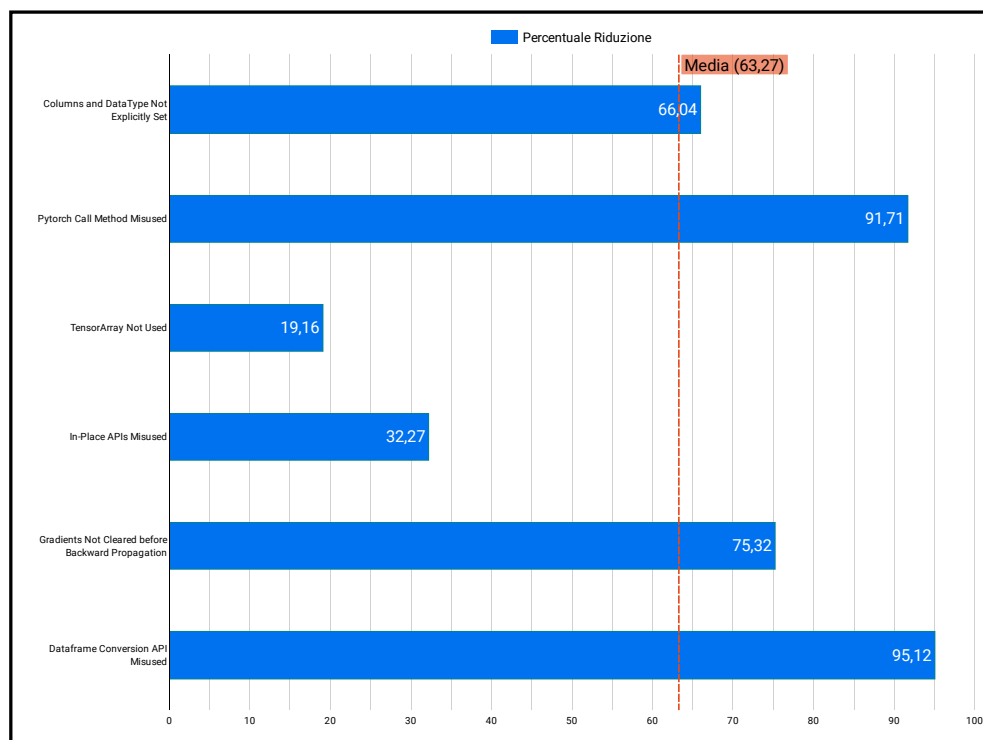


Figura 4.3: Report sulle percentuali di riduzione dei Code Smells.

Come si può osservare dalle percentuali riportate, GPT-4o mini ha mostrato maggiore efficacia nel refactoring di alcuni Code Smells rispetto ad altri. Nelle sotto-sezioni seguenti, l'attenzione sarà focalizzata sui Code Smells che hanno registrato un tasso di riduzione più basso. Saranno analizzati più nel dettaglio questi specifici Code Smells, cercando di identificare le possibili cause delle percentuali ottenute.

Tensor Array Not Used

Nell'ambito dello sviluppo con TensorFlow 2, il Code Smell "Tensor Array Not Used" si verifica quando è necessario modificare il valore di un array inizializzato con `"tf.constant()"` all'interno di un ciclo. Tali modifiche vengono spesso gestite tramite `"tf.nn.assign()"`, che, pur essendo una soluzione praticabile, si rivela subottimale poiché genera numerosi tensori intermedi e complica la gestione della memoria [4].

Sebbene la riduzione del 19.16% nel Code Smell "Tensor Array Not Used" mostri che ci sono stati alcuni successi nel refactoring, la performance sotto la media suggerisce che il refactoring automatizzato effettuato da GPT-4o mini abbia incontrato limitazioni nell'affrontare efficacemente questo problema. Le sfide incontrate da GPT-4o mini possono essere attribuite principalmente alla complessità delle API di TensorFlow. In particolare, `"tf.TensorArray()"`, l'alternativa raccomandata, richiede una comprensione approfondita dei flussi di controllo e della logica del programma, una competenza che può essere ardua per un modello di intelligenza artificiale che analizza principalmente il testo del codice senza un contesto operativo dettagliato. Questa situazione sottolinea l'importanza per gli sviluppatori di intervenire e perfezionare le soluzioni proposte, garantendo un uso appropriato di `"tf.TensorArray()"` per migliorare la gestione degli array dinamici e ottimizzare le performance e la manutenibilità del codice.

Per comprendere meglio i problemi affrontati da GPT-4o mini, è stata condotta un'ispezione manuale di alcuni dei suggerimenti forniti. Quest'ispezione ha rivelato che, nella maggior parte dei casi, i suggerimenti rispettavano, almeno sintatticamente, le linee guida fornite per la rimozione del Code Smell "Tensor Array Not Used". Pertanto, considerando che CodeSmile [22] è ancora uno strumento in fase di sperimentazione, è possibile che alcuni problemi riscontrati non siano direttamente attribuibili alle capacità di GPT-4o mini, ma piuttosto alle detection rules utilizzate dallo strumento di analisi.

In-Place APIs Misused

Nel contesto dello sviluppo software, specialmente nell'uso di librerie come Pandas e TensorFlow, il Code Smell "In-Place APIs Misused" si verifica quando

gli sviluppatori adottano un approccio in-place senza impostare correttamente i parametri necessari. Questo problema nasce dalla differenza nel comportamento delle API che, per impostazione predefinita, possono modificare una struttura dati esistente (in-place) o restituire una copia modificata [4].

Una riduzione del 32.37% per questo Code Smell indica che sono state incontrate delle difficoltà nel gestirlo in modo efficace tramite refactoring automatizzato. Queste difficoltà possono essere principalmente attribuite alla causa del Code Smell stesso. Infatti, come già spiegato in precedenza, le API di diverse librerie come Pandas, NumPy e PyTorch hanno comportamenti diversi riguardo le operazioni in-place. Mentre alcune API adottano l'in-place di default, altre restituiscono una copia modificata dell'oggetto. Questa inconsistenza può complicare il refactoring automatizzato e confondere GPT-4o mini.

Per migliorare l'efficacia dei suggerimenti di refactoring, una possibile soluzione potrebbe consistere nel perfezionare il prompt inviato a GPT-4o mini. Questo miglioramento dovrebbe includere indicazioni precise su come gestire tutte le diverse API che vengono più frequentemente associate a questo Code Smell. Fornendo a GPT-4o mini istruzioni dettagliate su come trattare le differenze tra le operazioni in-place e quelle che restituiscono una copia, si potrebbero ottenere suggerimenti più accurati e contestualmente appropriati.

4.2.3 Miglioramento della Qualità del Codice

Come discusso nella Sezione 2.2.4, i Code Smells possono avere un impatto negativo su diversi aspetti legati alla qualità del codice. Pertanto, in questo studio è stata condotta un'analisi dettagliata per valutare gli effetti del refactoring su questi aspetti. I 6 Code Smells analizzati incidono principalmente su quattro aree critiche: leggibilità, propensione agli errori (error-prone), robustezza ed efficienza. Di seguito è riportata una sintesi dei risultati illustrati nel Grafico in Figura 4.4:

- **Leggibilità:** Il numero di Code Smells che compromettono la leggibilità del codice è diminuito del 66.04%, passando da 1655 a 562. Questa riduzione significativa migliora la chiarezza del codice, rendendolo più facile da comprendere

e mantenere. Un codice più leggibile facilita la collaborazione tra sviluppatori e riduce il tempo necessario per comprendere e modificare il codice esistente.

- **Error Prone:** I Code Smells che aumentano il rischio di errori sono stati ridotti del 44.58%, da 1384 a 767. Questa riduzione indica che il codice è diventato meno suscettibile a bug e malfunzionamenti, migliorando la sua affidabilità complessiva. Un codice meno propenso agli errori riduce la probabilità di fallimenti in fase di produzione e semplifica il processo di debugging e testing.
- **Robustezza:** L'incremento più notevole è stato osservato nella categoria della robustezza, con una riduzione dei Code Smells del 91.71%, da 712 a 59. Questo segna un sostanziale miglioramento nella capacità del software di funzionare correttamente in condizioni diverse senza fallimenti. Un codice più robusto resiste meglio ai cambiamenti e alle condizioni impreviste, garantendo prestazioni stabili e una minore frequenza di malfunzionamenti.
- **Efficienza:** Il numero di Code Smells legati all'efficienza è diminuito del 19.16%, passando da 454 a 367. Questa riduzione, sebbene meno marcata rispetto ad altre categorie, riflette un miglioramento nella gestione delle risorse e nella riduzione delle complessità inutili all'interno del codice, contribuendo a diminuire il consumo di risorse e a migliorare le prestazioni complessive. La percentuale nettamente più bassa rispetto alle altre è dovuta principalmente al fatto che l'unico Code Smell analizzato che impatta sull'efficienza è "Tensor Array Not Used", le cui difficoltà di rimozione sono state descritte nella sezione precedente.

Questi miglioramenti non solo elevano la qualità complessiva del software, ma contribuiscono attivamente anche alla riduzione del Technical Debt. Infatti, diminuendo il numero di Code Smells, si minimizzano i costi futuri legati alla manutenzione e alla risoluzione di problemi non identificati o non risolti in fase di sviluppo.

È importante sottolineare che alcuni Code Smells influenzano negativamente più di uno di questi aspetti qualitativi, quindi la loro eliminazione può avere un impatto ancora più positivo sulla qualità del codice. Ad esempio, il Code Smell "Tensor Array Not Used", che rende il codice sia inefficiente sia propenso agli errori,

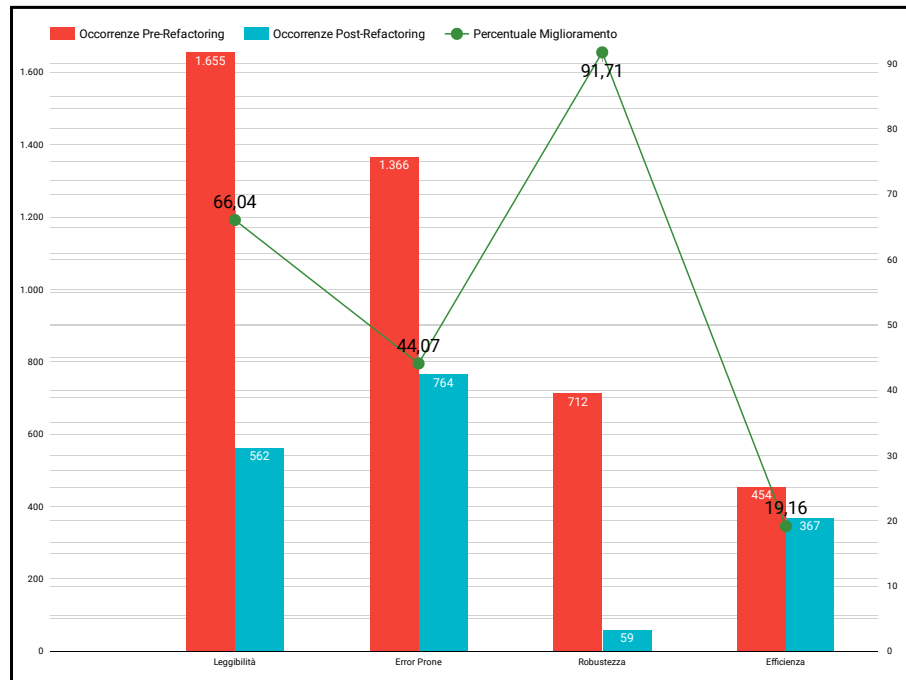


Figura 4.4: Report sui miglioramenti qualitativi riscontrati.

una volta eliminato, contribuirà significativamente a rendere il software più efficiente e affidabile. Questo aspetto rappresenta un'importante indicazione per ulteriori potenziamenti di CAMILLE, al fine di migliorare il supporto alla rimozione di tali Code Smells in modo più affidabile e preciso.

4.3 Considerazioni Finali

Sebbene i risultati ottenuti dimostrino un significativo decremento nel numero dei Code Smells a seguito dei suggerimenti di refactoring, questi dati non garantiscono che le modifiche proposte siano sempre corrette dal punto di vista funzionale. Il testing condotto si è focalizzato principalmente sulla correttezza sintattica e sull'aderenza alle linee guida stabilite per evitare i Code Smells, senza poter assicurare che tali cambiamenti non compromettano il funzionamento generale dell'applicazione. Questa limitazione è intrinseca nel metodo di testing adottato, che, per mantenere una certa semplicità, non include l'esecuzione del codice in un contesto operativo reale né l'esecuzione di test funzionali approfonditi.

Tuttavia, è importante sottolineare che CAMILLE non è concepito come uno strumento di refactoring completamente automatizzato. Piuttosto, si propone come

un supporto per gli sviluppatori nell'identificazione e nella correzione dei Code Smells. Pertanto, l'efficacia di CAMILLE è ottimizzata quando viene integrata con la conoscenza e l'esperienza dello sviluppatore, il quale possiede una comprensione completa del contesto applicativo del progetto. L'esperienza dello sviluppatore è fondamentale per valutare la pertinenza delle modifiche suggerite, specialmente in situazioni in cui GPT-4o mini può incontrare difficoltà, dato che non ha accesso al contesto completo del progetto a causa delle ovvie limitazioni comuni a tutti i LLM.

È ragionevole ipotizzare che, in un contesto di utilizzo reale, dove gli sviluppatori possono integrare attivamente i suggerimenti di CAMILLE con il proprio giudizio professionale, le percentuali di miglioramento potrebbero essere notevolmente superiori a quelle osservate in questo studio. L'interazione sinergica tra CAMILLE e l'esperienza umana può portare a una riduzione più marcata dei Code Smells, migliorando ulteriormente la qualità del codice e riducendo il Technical Debt in modo più efficace.

Conclusioni e Sviluppi Futuri

5.1 Conclusioni

Il presente lavoro di tesi ha evidenziato l'importanza dei Code Smells e il loro impatto dannoso sui prodotti software, nonché il loro contributo all'accumulo di Technical Debt, che può portare a problemi più gravi. Nel contesto del Machine Learning, queste problematiche sono altrettanto dannose ma ancora poco esplorate, una lacuna che ha motivato la realizzazione di CAMILLE. Questo chatbot è stato progettato per assistere gli ingegneri del machine learning nelle diverse fasi delle pipeline di ML, offrendo funzionalità pensate per facilitare la rilevazione e la rimozione dei Code Smells. Tra queste funzionalità, CAMILLE fornisce informazioni ed esempi sui vari ML-Specific Code Smells, permette l'avvio di analisi delle repository GitHub e offre suggerimenti di refactoring generati tramite il LLM GPT-4o mini. Dall'analisi condotta sull'efficacia di quest'ultima funzionalità, è emerso un notevole potenziale di CAMILLE: i suoi suggerimenti di refactoring hanno permesso una riduzione automatica del 62.97% del numero di Code Smells nei progetti analizzati. Si è osservato che questi suggerimenti sono stati particolarmente efficaci per alcune tipologie di Code Smells, mentre per altre sono stati meno impattanti, probabilmente a causa della limitata comprensione dell'intero contesto applicativo da parte del LLM.

Tuttavia, questa limitazione può essere superata dagli sviluppatori, che, avvalendosi di una conoscenza più approfondita del contesto, possono utilizzare i suggerimenti di CAMILLE come linee guida per raggiungere il proprio scopo piuttosto che come soluzioni definitive. In conclusione, CAMILLE, supportando gli sviluppatori nella rimozione degli ML-Specific Code Smells, contribuisce non solo a migliorare la qualità del codice ma anche a ridurre il Technical Debt. Pertanto, CAMILLE rappresenta un significativo progresso nello stato dell'arte, in un campo dove l'attenzione verso queste problematiche nel Machine Learning è ancora limitata.

5.2 Sviluppo Futuri

CAMILLE ha già dimostrato di possedere un potenziale significativo; tuttavia, vi è ancora ampio margine per ulteriori miglioramenti che potrebbero accrescerne l'efficacia. Innanzitutto, considerando che le librerie e le API nel campo del Machine Learning sono soggette a frequenti aggiornamenti ed estensioni, sarebbe opportuno proseguire il lavoro iniziato da Zhang et al. [4], aggiornando continuamente le informazioni relative agli ML-Specific Code Smells, eliminando quelli che, a causa degli stessi aggiornamenti, non sono più rilevanti o verificabili, e introducendone eventualmente dei nuovi. Ovviamente, questi aggiornamenti dovrebbero essere riflessi tempestivamente nelle funzionalità di CAMILLE e nel suo tool di analisi.

Per quanto riguarda i possibili miglioramenti più strettamente legati a CAMILLE, bisogna considerare che CAMILLE è un Chatbot e, come tale, sarebbe opportuno seguire una metodologia di sviluppo appropriata. In particolare, lo stesso framework utilizzato per realizzare CAMILLE, Rasa, promuove l'adozione del Conversation Driven Development [41]. Questa metodologia prevede il rilascio iniziale del chatbot agli utenti non appena le funzionalità di base sono operative, seguito dalla raccolta continua di dati sull'utilizzo e feedback periodici attraverso sondaggi. È proprio basandosi su queste informazioni e sui feedback ricevuti, che si dovrebbero apportare miglioramenti alle funzionalità esistenti ed eventualmente svilupparne di nuove. Implementando questo processo in modo iterativo, CAMILLE può evolversi in uno strumento ancora più efficace e pragmaticamente utile, rispondendo alle necessità e alle preferenze degli utenti.

Bibliografia

- [1] M. Fowler and K. Beck, *Refactoring: Improving the Design of Existing Code*, ser. Addison-Wesley object technology series. Addison-Wesley, 1999. [Online]. Available: <https://books.google.it/books?id=UTgFCAAAQBAJ> (Citato alle pagine iv, 9, 10 e 11)
- [2] M. Mantyla, J. Vanhanen, and C. Lassenius, "A taxonomy and an initial empirical study of bad smells in code," in *International Conference on Software Maintenance, 2003. ICSM 2003. Proceedings.*, 2003, pp. 381–384. (Citato alle pagine iv, 9, 10 e 11)
- [3] E. Adamopoulou and L. Moussiades, "An overview of chatbot technology," 05 2020, pp. 373–383. (Citato alle pagine iv, 18, 19, 20 e 21)
- [4] H. Zhang, L. Cruz, and A. van Deursen, "Code smells for machine learning applications," 2022. [Online]. Available: <https://arxiv.org/abs/2203.13746> (Citato alle pagine vi, 2, 11, 13, 14, 15, 16, 86, 87 e 92)
- [5] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *SN Computer Science*, vol. 2, no. 3, p. 160, 2021. [Online]. Available: <https://doi.org/10.1007/s42979-021-00592-x> (Citato a pagina 1)
- [6] A. J. Simmons, S. Barnett, J. Rivera-Villicana, A. Bajaj, and R. Vasa, "A large-scale comparative analysis of coding standard conformance in open-source data

- science projects,” in *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ser. ESEM '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3382494.3410680> (Citato alle pagine 1 e 12)
- [7] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28. Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf (Citato alle pagine 2, 5, 7 e 8)
- [8] W. Cunningham, “The wycash portfolio management system,” *SIGPLAN OOPS Mess.*, vol. 4, no. 2, p. 29–30, dec 1992. [Online]. Available: <https://doi.org/10.1145/157710.157715> (Citato a pagina 5)
- [9] P. Avgeriou, I. Ozkaya, A. Chatzigeorgiou, M. Ciolkowski, N. A. Ernst, R. J. Koontz, E. Poort, and F. Shull, “Technical debt management: The road ahead for successful software delivery,” in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, May 2023. [Online]. Available: <http://dx.doi.org/10.1109/ICSE-FoSE59343.2023.00007> (Citato a pagina 5)
- [10] A. Martini, V. Stray, and N. B. Moe, “Technical-, social- and process debt in large-scale agile: An exploratory case-study,” in *Agile Processes in Software Engineering and Extreme Programming – Workshops*, R. Hoda, Ed. Cham: Springer International Publishing, 2019, pp. 112–119. (Citato a pagina 6)
- [11] J. Bogner, R. Verdecchia, and I. Gerostathopoulos, “Characterizing technical debt and antipatterns in ai-based systems: A systematic mapping study,” in *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*, 2021, pp. 64–73. (Citato a pagina 6)

-
- [12] G. Recupito, F. Pecorelli, G. Catolino, V. Lenarduzzi, D. Taibi, D. Nucci, and F. Palomba, "Technical debt in ai-enabled systems: On the prevalence, severity, impact, and management strategies for code and architecture," 10 2023. (Citato alle pagine 7 e 8)
- [13] SonarQube, <https://www.sonarqube.org>, accessed on Jul. 3, 2024. (Citato alle pagine 8 e 11)
- [14] M. Ops, "Ml ops," <https://ml-ops.org/>, accessed on Jul. 5, 2024. (Citato a pagina 8)
- [15] A. Kaur, "A systematic literature review on empirical analysis of the relationship between code smells and software quality attributes," *Archives of Computational Methods in Engineering*, vol. 27, 07 2019. (Citato a pagina 9)
- [16] M. Tufano, F. Palomba, G. Bavota, R. Oliveto, M. D. Penta, A. De Lucia, and D. Poshyvanyk, "When and why your code starts to smell bad (and whether the smells go away)," *IEEE Transactions on Software Engineering*, vol. 43, no. 11, pp. 1063–1088, 2017. (Citato a pagina 9)
- [17] D. Das, A. A. Maruf, R. Islam, N. Lambaria, S. Kim, A. S. Abdelfattah, T. Cerny, K. Frajtak, M. Bures, and P. Tisnovsky, "Technical debt resulting from architectural degradation and code smells: a systematic mapping study," *SIGAPP Appl. Comput. Rev.*, vol. 21, no. 4, p. 20–36, jan 2022. [Online]. Available: <https://doi.org/10.1145/3512753.3512755> (Citato alle pagine 10 e 11)
- [18] Y. Tang, R. Khatchadourian, M. Bagherzadeh, R. Singh, A. Stewart, and A. Raja, "An empirical study of refactorings and technical debt in machine learning systems," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 238–250. (Citato a pagina 10)
- [19] Designite, <https://www.designite-tools.com/>, accessed on Jul. 3, 2024. (Citato a pagina 11)
- [20] S. A. for Java, <https://stan4j.com/>, accessed on Jul. 3, 2024. (Citato a pagina 12)

- [21] F. A. Fontana, E. Mariani, A. Mornioli, R. Sormani, and A. Tonello, “An experience report on using code smells detection tools,” in *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*, 2011, pp. 450–457. (Citato a pagina 12)
- [22] G. Recupito, G. Giordano, F. Ferrucci, D. D. Nucci, and F. Palomba, “When code smells meet ml: On the lifecycle of ml-specific code smells in ml-enabled systems,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.08311> (Citato alle pagine 16, 25, 27, 29, 80, 83, 84 e 86)
- [23] Q. Motger, X. Franch, and J. Marco, “Conversational agents in software engineering: Survey, taxonomy and challenges,” *CoRR*, vol. abs/2106.10901, 2021. [Online]. Available: <https://arxiv.org/abs/2106.10901> (Citato alle pagine 16 e 17)
- [24] A. M. TURING, “I.—COMPUTING MACHINERY AND INTELLIGENCE,” *Mind*, vol. LIX, no. 236, pp. 433–460, 10 1950. [Online]. Available: <https://doi.org/10.1093/mind/LIX.236.433> (Citato a pagina 16)
- [25] J. Weizenbaum, “Eliza—a computer program for the study of natural language communication between man and machine,” *Commun. ACM*, vol. 9, no. 1, p. 36–45, jan 1966. [Online]. Available: <https://doi.org/10.1145/365153.365168> (Citato alle pagine 16 e 19)
- [26] K. M. Colby, S. Weber, and F. D. Hilf, “Artificial paranoia,” *Artificial Intelligence*, vol. 2, no. 1, pp. 1–25, 1971. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0004370271900026> (Citato a pagina 16)
- [27] G. Güzeldere and S. Franchi, “Dialogues with colorful “personalities” of early ai,” 1995. [Online]. Available: <https://api.semanticscholar.org/CorpusID:63360351> (Citato a pagina 16)
- [28] R. S. Wallace, *The Anatomy of A.L.I.C.E.* Dordrecht: Springer Netherlands, 2009, pp. 181–210. [Online]. Available: https://doi.org/10.1007/978-1-4020-6710-5_13 (Citato a pagina 16)

-
- [29] Apple, "Siri," <https://www.apple.com/it/siri/>, accessed on Jul. 4, 2024. (Citato alle pagine 17 e 19)
- [30] Google, "Google assistant," https://assistant.google.com/intl/it_it/, accessed on Jul. 4, 2024. (Citato a pagina 17)
- [31] Amazon, "Alexa," <https://www.alexa.com/>, accessed on Jul. 4, 2024. (Citato a pagina 17)
- [32] S. Jung, "Semantic vector learning for natural language understanding," *Computer Speech Language*, vol. 56, pp. 130–145, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0885230817303595> (Citato a pagina 20)
- [33] S. Santhanam, T. Hecking, A. Schreiber, and S. Wagner, "Bots in software engineering: a systematic mapping study," *PeerJ Computer Science*, vol. 8, p. e866, 2022. (Citato alle pagine 22 e 23)
- [34] E. Paikari, J. Choi, S. Kim, S. Baek, M. Kim, S. Lee, C. Han, Y. Kim, K. Ahn, C. Cheong, and A. van der hoek, "A chatbot for conflict detection and resolution," in *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, 2019, pp. 29–33. (Citato a pagina 23)
- [35] G. Voria, V. Pentangelo, A. D. Porta, S. Lambiase, G. Catolino, F. Palomba, and F. Ferrucci, "Community smell detection and refactoring in slack: The cadocs project," in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022, pp. 469–473. (Citato a pagina 23)
- [36] O. P. Santosuosso, "Studio e analisi dei problemi riguardanti l'utilizzo dei bot per lo sviluppo software," 2022, tesi di laurea triennale. (Citato a pagina 23)
- [37] J. Cerezo, J. Kubelka, R. Robbes, and A. Bergel, "Building an expert recommender chatbot," in *2019 IEEE/ACM 1st International Workshop on Bots in Software Engineering (BotSE)*, 2019, pp. 59–63. (Citato a pagina 23)
- [38] Rasa, "Rasa," <https://rasa.com/docs/rasa/>, accessed on Jul. 19, 2024. (Citato alle pagine 25 e 34)

- [39] —, “Rasa architecture overview,” <https://rasa.com/docs/rasa/arch-overview>, accessed on Jul. 19, 2024. (Citato a pagina 35)
- [40] —, “Rasa domain,” <https://rasa.com/docs/rasa/domain>, accessed on Jul. 19, 2024. (Citato a pagina 44)
- [41] —, “Conversation driven development,” <https://rasa.com/docs/rasa/conversation-driven-development>, accessed on Jul. 19, 2024. (Citato alle pagine 48 e 92)
- [42] —, “Rasa nlu training data,” <https://rasa.com/docs/rasa/nlu-training-data>, accessed on Jul. 19, 2024. (Citato a pagina 49)
- [43] —, “Rasa nlu pipeline components,” <https://rasa.com/docs/rasa/components>, accessed on Jul. 19, 2024. (Citato a pagina 51)
- [44] —, “Rasa rules,” <https://rasa.com/docs/rasa/rules>, accessed on Jul. 19, 2024. (Citato alle pagine 53 e 54)
- [45] —, “Rasa stories,” <https://rasa.com/docs/rasa/stories>, accessed on Jul. 19, 2024. (Citato alle pagine 53 e 55)
- [46] —, “Rasa policies,” <https://rasa.com/docs/rasa/policies>, accessed on Jul. 19, 2024. (Citato a pagina 56)
- [47] —, “Rasa custom actions,” <https://rasa.com/docs/rasa/custom-actions>, accessed on Jul. 19, 2024. (Citato a pagina 57)
- [48] R. Widyasari, Z. Yang, F. Thung, S. Q. Sim, F. Wee, C. Lok, J. Phan, H. Qi, C. Tan, Q. Tay, and D. Lo, “Niche: A curated dataset of engineered machine learning projects in python,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.06286> (Citato a pagina 80)

Ringraziamenti

Desidero esprimere la mia più sincera gratitudine a tutte le persone che mi hanno supportato e accompagnato durante questo percorso, senza le quali questa tesi non sarebbe stata possibile.

In primo luogo, vorrei ringraziare il Professore Fabio Palomba, il Dottore Gilberto Recupito e il Dottore Giammaria Giordano, relatori di questa tesi, per il loro costante supporto lungo tutto il percorso. La loro esperienza, conoscenza e disponibilità immediata sono state fondamentali per la realizzazione di questo lavoro.

Non posso non menzionare il supporto incondizionato della mia famiglia. Grazie di cuore ai miei genitori, che mi hanno sempre sostenuto in ogni scelta e incoraggiato a perseguire i miei sogni. La vostra fiducia in me è stata la mia forza. Grazie di cuore anche a mia sorella, Maria Chiara, per il suo affetto e il suo sostegno costante, così come ai miei nonni e agli zii, che hanno sempre creduto in me e mi hanno fatto sentire amato e supportato in ogni momento. Un ringraziamento speciale va a mio nipote Marco e a Michele. Un grazie di cuore anche ai miei cugini Luca e Jessica, per il loro affetto e la loro presenza.

Un sentito ringraziamento va anche ai miei compagni di università, Michele, Giulio, Giovanni, Stefano, Mauro, Giuseppe e Salvatore, con cui ho condiviso questo viaggio.

Grazie per il vostro sostegno, per i momenti di confronto e per le tante ore di studio (e non solo) trascorse insieme.

Un grazie speciale ai miei compagni di squadra, Christian, Gerardo, Giovanni e Antonio, per le nuotate, gli allenamenti e i tanti momenti conviviali condivisi. Inoltre, vorrei ringraziare Andrea, Davide, Marco, Luciano e Pasquale, per i momenti di svago che mi hanno aiutato a trovare il giusto equilibrio tra studio e relax.

A voi e a tutti gli altri, il mio più sentito grazie.

Questa tesi ha contribuito a piantare un albero in Nepal tramite il progetto Treedom.

<https://www.treedom.net/it/user/francesco-pinto-6437/trees/5MX-Y45N>